

第一部分 · 纯八股

120 道原理题 · 每题包括机制、推演、条件与追问

C 与 C++：对象、内存与语法规则

操作系统：执行、同步与内存

Linux I/O 与事件驱动

帧协议、可靠传输与幂等

TCP、MQTT、HTTP 与网络边界

存储、日志与运行期生命周期

Cortex-M3 与 STM32 系统基础

外设、DMA 与传感器

FreeRTOS 内核与任务通信

时间、低功耗与故障恢复

排障、验证与软件工程

跨模块设计与定量分析

FreeRTOS 移植：源码、异常与验收

C 与 C++：对象、内存与语言规则

T001 原理问答

RAII 为什么比到处手动 `close` 更可靠？

RAII 把资源的释放时机绑定到对象生命周期。关键不是“用了析构函数”，而是任何时刻都有明确所有者，取得资源后立即交给它；这样正常返回、提前返回和异常展开都走同一套释放规则。

原理与成立条件

RAII 的核心不变量

资源获取即初始化，把资源有效期绑定到对象有效期。成功构造的对象拥有资源，析构时负责释放；离开作用域、提前 `return` 或异常栈展开都走同一释放路径。资源不仅是堆内存，还包括 `fd`、互斥锁和连接。与 `goto cleanup` 相比，责任随类型封装，调用者不必逐条记住释放动作。

RAII 不能替代业务关闭协议

析构能释放内存，却不保证后台线程已退出、队列已排空或数据库提交成功。成员按声明顺序的逆序销毁，也要符合依赖关系。线程仍使用某成员时就析构该成员，即使每个成员都用了智能指针，整体设计仍可能错误。先建立停止顺序，再让 RAII 承担最后的资源回收。

推演与反例

为什么只靠成对调用仍会泄漏

假设函数先 `open` 文件，再分配缓冲，最后启动线程。第二步失败时，第三步从未发生，但第一步已经需要清理。逐条 `goto` 或 `catch` 可以做对，却会随路径增加而变复杂。用 `fd` 包装器、`vector` 和线程管理对象分别接管资源，已构造完成的成员在失败时逆序析构；尚未构造完成的对象自身不会调用析构，因此构造中的裸资源尤其应先交给已完整构造的子对象。

独占所有权怎样影响接口

资源包装器通常禁用复制，允许移动：移动后原对象进入“不持有资源”的状态，最终只关闭一次。借用者可使用 `fd` 或指针，但不能擅自释放，也不能活得比所有者长。线程资源还多一步：析构前先提出停止、唤醒阻塞点并 `join`，只有释放句柄不足以保证后台代码已经停止访问成员。

继续追问

为什么不能依赖析构函数自动提交数据库事务？

析构缺乏清晰的失败返回通道，而且异常退出时往往应回滚。一般让提交显式发生，析构负责在未提交时安全撤销或关闭。

RAII 能保证断电时也执行清理吗？

不能。进程被强制终止、断电或异常终止路径不保证正常栈展开。RAII 解决进程内资源生命周期；持久状态仍靠事务、日志及恢复协议。

依据与核对范围

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

T002 原理问答

unique_ptr、shared_ptr、weak_ptr 如何选择？

先决定谁拥有对象，再选智能指针。唯一拥有者用 `unique_ptr`；确实有多个独立生命周期共同拥有时用 `shared_ptr`；只想观察共享对象而不延长寿命，用 `weak_ptr`。裸指针也可表达借用，重点是接口是否说清释放责任。

原理与成立条件

循环引用与析构线程

A 强持 B、B 强持 A 会让计数无法归零，可将非拥有方向改为 `weak_ptr`。回调按值捕获 `shared_ptr` 也可能形成隐蔽环。最后一个强引用在哪个线程释放，析构就可能在哪个线程执行；若析构耗时或必须在某个线程进行，需要额外的销毁调度设计。

推演与反例

控制块和对象是两个层次

`shared_ptr` 通过控制块管理强弱引用计数、删除器等。强引用归零时销毁对象；控制块还要供 `weak_ptr` 判断失效，通常到相关弱引用也释放后才能回收。`weak_ptr::lock` 把“是否还活着”和“取得一份强引用”组合为安全操作，不能先 `expired` 再使用一个原始指针，因为两步之间对象可能被销毁。

原子引用计数没有保护哪些东西

两个不同 `shared_ptr` 实例可共同管理一个对象并分别调整引用；但一个 `shared_ptr` 变量被一个线程赋值、另一个线程普通读取，仍涉及该变量自身的竞争。C++17 可以使用针对 `shared_ptr` 的原子自由函数或锁；`atomic<shared_ptr<T>>` 是 C++20 的接口。无论哪种方式，被指对象的业务字段仍需单独同步。

继续追问

`weak_ptr.expired()==false` 后直接用旧裸指针安全吗？

不安全，检查后对象仍可能销毁。应 `lock` 取得一个覆盖整个访问过程的强引用，再检查业务对象是否处于可用状态。

`make_shared` 总比分别 `new` 更好吗？

通常减少分配次数并简化异常安全；但大对象与控制块合并分配时，残留 `weak_ptr` 可能使整块分配迟迟不能归还。自定义删除器、外部资源或特殊分配要求也会影响选择。

依据与核对范围

C++ 工作草案: `shared_ptr` — 共享所有权及线程安全边界。项目按 C++17，未引入新标准专属特性。

T003 原理问答

`std::move` 做了什么，移动后对象还能用吗？

`std::move` 只是把表达式转换成可被移动重载接受的值类别，真正转移资源的是被选中的移动构造或移动赋值。它本身既不搬运字节，也不清空原对象。

原理与成立条件

转移所有权不等于端到端零拷贝

队列中移动 `vector` 可转移其缓冲，但编码时、内核 I/O 和硬件搬运仍可能复制。移动也不会让指向原对象子对象的任意引用自动有效，具体失效规则看类型。`return` 局部对象通常让编译器进行复制消除或隐式移动，随意加 `std::move` 可能妨碍 NRVO。

推演与反例

一次容器插入究竟发生什么

把一个 `vector` 移进另一个对象，在通常的分配器条件下可以转交数据区指针、大小和容量，避免逐个复制元素；但小对象、内嵌存储或不兼容分配器可能仍需逐项操作。对 `const` 对象 `std::move` 得到的仍带 `const`，常规 T&& 移动构造不能接收它，因此最终可能调用复制构造。判断是否发生移动必须看类型及重载，而不是看到 `move` 字样就下结论。

移动后如何写正确代码

标准库对象通常保证有效但未指定状态：可析构、赋新值，也能调用满足前置条件的成员；不能普遍假设字符串一定为空或容器一定保留旧内容。自定义类型应明确移动后的不变量。容器扩容还会考虑移动是否 `noexcept`；当移动可能抛异常且可复制时，`vector` 可能用复制维持异常保证。

继续追问

怎么验证某次操作究竟复制还是移动？

先看重载签名和值类别，再用可控示例或调试观察构造路径；不要仅凭源码出现 `std::move` 判断，也不要打打印改变时序后的单次测量宣称性能提升。

返回局部变量时都写 `return std::move(x)` 吗？

通常不需要，它可能阻碍命名返回值优化。应优先让返回值优化和隐式移动发挥作用，再在确需转移已有对象时显式 `move`。

依据与核对范围

C++ 工作草案：move / forward — 值类别转换与移动语义。项目按 C++17，未引入新标准专属特性。

T004 原理问答

vector 扩容、迭代器失效和字符串缓冲有什么坑？

容器操作首先要问“对象有没有搬家”和“元素位置有没有变化”。`vector` 连续存储，扩容会搬迁元素；即使没有扩容，中间插入或删除也可能使后面的元素移动，因此不同操作的引用失效范围不同。

原理与成立条件

view 与裸指针都不会保活

`string_view`、`span` 类视图以及 `data()` 返回地址都依赖原存储；容器扩容、销毁或修改相应区域后，视图可能悬空或含义改变。异步回调拿着一个 `view` 时，必须让拥有者在回调结束前保持稳定，而不是只保证局部 `view` 对象还存在。

消费缓冲怎样避免重复搬移

每处理一个字节就 `erase(begin)` 会反复搬动尾部，累计可能接近平方级成本。常用读写偏移标记已消费区域，空间不足时再集中压缩或扩容；但压缩后原地址同样失效。优化复杂度时还需设总容量上限，否则慢速消费者仍会拖垮内存。

推演与反例

用三个元素说明失效

`vector` 有 A、B、C，保存了指向 B 的指针。若 `push_back` 导致重新分配，旧数据区释放，所有元素指针都失效；若容量足够，既有元素指针仍有效，但旧 `end` 已改变。若 `erase(A)`，B、C 被前移，即使底层地址没换，原 B 位置也不能继续当作原来的 B。`reserve` 只减少扩容，不能解决擦除引起的身份变化。

字符串缓冲与异步发送

`data` 或 `c_str` 返回的是对象内部存储的借用地址，后续修改、扩容、移动和析构可能使地址或内容不再满足要求。把这个地址交给异步接口后马上复用字符串，错误可能在很久后才出现。应让发送任务持有整个字符串/独占缓冲，或在提交时复制，而不是仅延长一个裸指针变量的寿命。

继续追问

提前 **reserve** 足够大就永远不会有失效问题吗？

只能在不超容量且相应操作不移动目标元素的条件下减少失效。**erase**、中间插入、容器销毁与超出预估的增长仍需处理。

把 **vector** 改成 **list** 就没有生命周期问题了吗？

list 的节点地址通常更稳定，但擦除对应节点后仍失效，遍历缓存局部性和分配成本也不同。稳定地址不能代替所有权与删除时机设计。

依据与核对范围

C++ 工作草案: `vector modifiers` — 扩容、擦除和失效规则。项目按 C++17，未引入新标准专属特性。

T005 原理问答

引用、指针、**const** 和回调捕获该怎样解释？

指针和引用描述访问关系，**const** 描述通过某个表达式允许做什么，它们都不自动延长普通被引用对象的生命周期。回调捕获尤其要区分“保存值”和“保存对原对象的借用”。

原理与成立条件

lambda 捕获的真实内容

[**x**] 复制 **x**，若 **x** 是裸指针，只复制地址；[**&x**] 借用局部变量，不能越过其寿命；[**this**] 捕获当前对象地址，不复制对象。C++17 的 [***this**] 才是对象副本语义。异步保存的 **lambda** 应逐项分析捕获数据及其指向对象的寿命，而不是看到“按值捕获”就认为安全。

接口如何表明生命周期

短同步读取可借用 **const** 引用；接管资源可用独占拥有者或值参数移动；真正共享可用 **shared_ptr**；可选但不拥有可用指针并写清有效期。返回局部对象引用是立即悬空的典型错误，延长某些临时对象寿命的语言规则也不会普遍沿多层引用传递。

推演与反例

回调返回后为何容易悬空

函数中创建局部配置，用 `[&]` 把它交给后台线程，函数返回后局部配置销毁，回调再访问就是悬空。改成值捕获只复制被捕获的对象；若复制的是裸指针，底层对象仍没有被保活。捕获 `this` 通常也是保存指针，不会复制整个对象。可让回调持有必要数据副本，或捕获 `weak_ptr`，运行时 `lock` 取得访问期间的所有权。

`const` 的三种常见误读

`const T*` 限制通过该指针修改 `T`，`T* const` 限制指针本身改指向，两者可以同时存在。`const` 成员函数限制通过 `this` 修改普通成员，但 `mutable` 成员仍可变，也不意味着线程安全。对真正 `const` 对象强行去 `const` 后写入有未定义行为；不能把 `const` 当“优化器不会动它”的设备同步手段。

继续追问

回调按值捕获一个含指针的结构体安全吗？

结构体副本只复制成员值，不自动深拷贝指针目标。仍需看目标由谁拥有，以及回调何时结束。

引用一定不为空，因此就一定安全吗？

合法绑定时应引用有效对象，但对象之后仍可能销毁，引用同样会悬空。引用没有“自动检查仍然存活”的运行期能力。

依据与核对范围

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

C++ 工作草案: `lambda capture` — 捕获形式与生命周期。项目按 C++17，未引入新标准专属特性。

T006 原理问答

结构体为什么不能直接当作串口协议发送？

结构体是编译器在内存中的对象布局，协议是双方约定的字节格式。二者不能混为一谈：协议必须明确字段宽度、符号、端序、单位、长度和校验范围，再逐字段编码解码。

原理与成立条件

显式编解码的顺序

先验证整帧最小长度和字段范围，再按字节构造数值。例如大端 32 位值逐字节先转为 `uint32_t` 再左移组合，避免 `int` 提升造成符号或移位问题。解析失败应返回明确错误且不读取后续越界区域；未知版本与未知类型要有清晰兼容策略。

协议必须约定的内容

除字段顺序，还需明确长度是否含头尾、校验覆盖范围、单位和比例、有符号性、保留字段以及合法值。CRC 自身字节序可以与业务字段不同，只要两端一致。跨平台测试应比较固定字节向量，而非仅用同一份错误编码再喂给同一份解码获得“互通”。

推演与反例

对齐怎样改变 `sizeof`

在一种常见 ABI 中，含 `uint8_t`、`uint32_t`、`uint16_t` 的结构体可能在第一字段后补 3 字节、尾部再补 2 字节，`sizeof` 为 12 而有效字段仅 7 字节；这是示例布局，不能作为所有平台的固定保证。直接发送整块对象会把填充字节也带出去，还可能泄露未初始化内容。`packed` 可减少填充，但会引入未对齐访问，依然没有解决端序、版本和数值表示问题。

解码应当先检查再组合

读取一个大端 `u16` 时，先证明至少还剩两个字节，再用无符号类型执行 `(uint16_t(p[0]) << 8) | p[1]`。不要把任意 `uint8_t*` 强转成 `uint16_t*` 解引用：地址可能未对齐，访问还可能不满足语言对象/别名规则，且仍依赖主机字节序。编码过程应与解码互逆，并用固定向量跨平台核对。

继续追问

`memcpy` 到本地整数就能解决端序吗？

`memcpy` 可避免部分对齐和别名问题，但取得的是本机表示，仍需按协议端序转换，也仍要先保证源长度足够。

`memcpy` 到结构体能解决所有问题吗？

它可避免某些未对齐解引用问题，但只有布局、字节序、对象表示等契约都匹配时才可能正确。外部格式仍应显式解析。

依据与核对范围

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

T007 原理问答

整数溢出、符号转换和时间回绕有哪些隐蔽问题？

整数问题常发生在表达式计算时，而不是赋值时。先确定每个操作数经过整数提升后的类型，再判断溢出、移位和符号转换；最后才看目标变量是否放得下结果。

原理与成立条件

长度计算如何防溢出

判断 $\text{offset} + \text{len} \leq \text{capacity}$ 前，加法可能已回绕；更稳妥是先保证 $\text{offset} \leq \text{capacity}$ ，再判断 $\text{len} \leq \text{capacity} - \text{offset}$ 。分配 n 个元素前检查 $n \leq \text{SIZE_MAX}/\text{sizeof}(T)$ 。网络字段即使范围很小，也应在转换为本机大小类型前验证，不能把强制转换当作输入校验。

经过时间与先后顺序不同

同宽无符号 `now-start` 可以处理计数回绕，但要求真实经过时间未达到一个完整计数周期，否则无法知道绕了几圈。比较两个环形时间戳的先后，通常还需半周期以内的距离约束或明确代数；直接转有符号的实现习惯也要考虑语言和平台契约。

数值例子帮助建立边界

32 位毫秒计数约 49.7 天一圈；72 MHz 的 32 位周期计数约 59.65 秒一圈。要延时几十微秒可以用差值，但不能拿后者无扩展地计一天。秒乘 1000、微秒乘时钟频率还可能在乘法阶段溢出，因此应先选择足够宽类型，再校验最大允许输入。

推演与反例

为什么扩大目标类型仍会溢出

两个 32 位有符号数先相乘，再赋给 `int64_t`，乘法可能早已在 32 位中产生未定义溢出。应在运算前扩大一个操作数。无符号减法按 2^N 取模，可用 `elapsed = now - start` 处理计数器的一次回绕，但必须保证实际经过时间没有超过能唯一解释的范围；超过一整圈后，仅凭两个端点无法知道经过了几圈。

时间先后与经过时间不是同一问题

若采用 N 位序号的半空间比较，只能在差值严格小于 $2^{(N-1)}$ 的约束下区分新旧；正好半圈存在歧义。

`elapsed` 是否达到一个有限 `duration`，与任意比较两个远距离绝对时间点，是不同算法。毫秒转 `Tick` 还要检查乘法宽度和向零截断，短延时可能被换算为 0。

继续追问

序号 255 后变 0，为什么不能只判断 `new > old`?

普通数值顺序不表达环形序号顺序；必须限制可比较距离，或增加会话/代数明确区分新旧。半空间规则及寿命约束见 T038、T040。

有符号整数溢出会像硬件一样自然回绕吗？

标准 C/C++ 不能这样假设。无符号运算有明确模规则；需要饱和、范围拒绝或宽位计算时应明确实现，不能依赖优化前看到的现象。

依据与核对范围

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

RFC 1982: Serial Number Arithmetic — 通用模序号比较背景；项目 SR 具体策略以源码为准。

T008 原理问答

malloc/free、new/delete、堆栈与碎片有什么区别？

存储取得、对象构造和对象销毁是三件事。malloc/free 管理原始存储；new 通常取得存储后构造对象，delete 先析构再释放。栈是自动生命周期的一种典型实现，堆适合运行期变化的生命周期，但需管理容量和碎片。

原理与成立条件

存储期与对象生命周期

栈常承载自动对象，函数返回时局部对象结束；堆允许存储跨作用域存在，但必须明确拥有者。malloc/free 管原始存储，new/delete 还执行构造与析构；placement new 可以在已有存储中构造，不再次分配。new[]/delete[] 与单对象形式也需配对，不能混用释放接口。

内存池如何改变问题

固定大小块池通过空闲链表等结构提供有界分配和回收，代价是最大并发数需要预估，块大小不匹配会浪费。不同大小对象可分池；池耗尽必须明确拒绝、等待或降级。静态分配消除一部分运行期失败，但不会阻止数组越界、栈溢出或错误地提前复用块。

嵌入式预算为何不能只看链接表

静态段占用只是部分 RAM；动态任务栈、内核对象、C 库堆、ISR 主栈和 DMA 缓冲都可能来自不同区域。递归、printf、大局部数组和最坏中断嵌套会放大栈需求。需要静态调用分析、运行水位与异常路径覆盖，不能以正常运行一次没有复位作为容量证明。

推演与反例

内部碎片与外部碎片怎么区分

分配器按对齐或大小类别给出比请求更大的块，未使用部分属于内部碎片；空闲空间被已分配块隔开，总量足够却没有足够大的连续块，是外部碎片。例如空闲 100、占用 100、空闲 100，总空闲 200 仍不能直接满足连续 150 的请求。合并相邻空闲块只消除相邻边界，不能跨过仍在使用的块。

实时系统关注最坏路径

一次分配可能包含链表遍历、锁竞争、页分配甚至缺页，因此平均很快不能保证截止时间。启动时预分配、固定大小块池和静态对象可以缩小运行期不确定性，但会固定容量。栈上大数组虽然不需要堆锁，却可能造成任务栈溢出；优化应同时算静态区、堆、每个任务栈和中断栈。

继续追问

还有 **2 KiB** 空闲堆，申请 **1 KiB** 为什么失败？

可能最大连续空闲块不足，或还需要块头、对齐开销；也可能查询的是另一分配器的堆。先确认分配器与最大块统计口径。

new[] 的结果为什么必须 **delete[]**？

数组释放需要按匹配的数组分配协议销毁元素并释放存储，混用 **delete**、**free** 等不匹配接口会产生未定义行为。

依据与核对范围

FreeRTOS: *Memory management* — heap_1..5 与静态分配。

C++ 工作草案: **new** — 对象构造和存储分配。项目按 C++17，未引入新标准专属特性。

T009 原理问答

编译、链接、静态库、extern C 和交叉编译分别做什么？

编译把每个翻译单元变成目标文件，链接把符号引用和段布局组合成最终映像；交叉编译还要保证指令集、ABI、库和链接脚本都与目标一致。能在 PC 上通过编译，不代表固件能在 MCU 正确启动。

原理与成立条件

四个阶段各处理什么

预处理展开包含、宏和条件编译；编译进行语义检查并生成目标架构代码表示；汇编形成带符号和重定位信息的目标文件；链接解析跨目标文件符号并确定最终地址。找不到头文件通常属于前期，**undefined reference** 多发生在链接阶段，运行时找不到共享库则又是装载问题。

静态库与动态库

静态库是目标文件归档，链接器通常按未解析符号需求抽取成员，因此库顺序和循环依赖可能影响结果；共享库一般由装载器映射，并按平台机制解析符号。静态链接方便分发却可能增加镜像体积，动态链接方便共享与独立更新却引入版本和部署依赖。

extern C 与交叉编译

`extern "C"` 指定语言链接，解决 C++ 名称修饰等与 C 接口的匹配；不会把实现自动限制为 C，也不保证结构体 ABI 在任意平台一致。交叉编译必须匹配指令集、字节序、浮点调用约定、运行库及 `sysroot`，不能只换一个编译器可执行文件名。

链接脚本和启动代码如何配合

链接脚本给 `.text`、`.data`、`.bss` 等安排运行地址与装载地址。初始化数据常在 Flash 保存初值、在 RAM 运行；启动代码按符号复制 `.data` 并清零 `.bss`，再调用运行库初始化及 `main`。若脚本和启动符号不一致，可能构建成功却在首次使用全局变量时出错。

推演与反例

undefined reference 与重复定义的根本

头文件只声明外部函数而没有把实现目标文件加入链接，会出现未定义引用；把非 `inline` 外部变量的定义放在被多个源文件包含的头文件中，可能出现多重定义。静态库按需要抽取目标文件，传统链接器的库顺序还会影响是否找到依赖。先检查最终链接命令、符号表和 `map`，比盲目添加 `include` 更有效。

extern C 到底改变什么

它让 C++ 声明采用 C 语言链接规则，便于与 C 目标文件中的符号匹配；不会让 C++ 类、异常或 STL 容器变成 C ABI。跨边界更适合固定宽度数据、显式长度、错误码和不透明句柄。ARM 固件还要核对 Thumb、CPU 类型和浮点 ABI，目标相同名称不代表二进制调用约定相同。

继续追问

声明已经写进头文件，为什么仍 undefined reference?

声明只承诺存在定义；还要把包含定义的目标文件链接进来，并确认签名、命名空间、语言链接及条件编译一致。

静态库与动态库主要差在哪里?

静态库中的被选对象通常并入最终映像；动态库在装载/运行阶段建立共享对象依赖。动态库可独立更新，但增加部署和 ABI 兼容要求，不能只比较文件大小。

依据与核对范围

GCC: [Overall Options](#) — 预处理、编译、汇编与链接阶段。

GNU ld: [Linker Scripts](#) — 段布局、装载地址与运行地址。

如何回答未定义行为、越界与优化后才出现的 bug?

未定义行为表示语言标准不再约束程序结果，编译器可据此前提优化。越界、悬空访问、数据竞争和某些整数运算错误都可能在低优化时偶尔正常，在高优化或不同布局下暴露。

原理与成立条件

UB 与其他不确定行为

未定义行为意味着标准不再约束程序结果；实现定义行为要求实现说明其选择，未指定行为则允许若干结果而不要求固定选择。三者不能混为“随机”。越界、悬空访问、部分非法移位、有符号溢出和 C++ 数据竞争都是需要优先排查的来源。

典型内存错误怎样识别

数组形参会调整为指针，函数内部 `sizeof(param)` 通常不是原数组长度；`memcpy` 的源和目的区间不能重叠，重叠搬移使用 `memmove`；二进制数据不保证以零终止，`strlen` 可能越界。先验证长度再读字段，并同时证明源可读范围、目的可写范围和对象寿命。

推演与反例

优化为什么会放大已有错误

如果程序用有符号溢出是否变小来判断溢出，编译器可基于“合法执行不会发生有符号溢出”删去该判断。越界写也可能原先碰巧落在未用空间，换一版编译器后就覆盖对象的长度或返回地址。应修复语言规则和边界证明，而不是把关闭优化当作最终方案。

检测工具覆盖不同错误

ASan 更适合越界和释放后使用，UBSan 检测一部分未定义运算，TSan 发现被执行路径上的数据竞争；它们改变时序且不覆盖所有问题。MCU 上可结合 MPU、栈检查、填充哨兵、硬件观察点和错误现场。一次工具无报错只说明此次执行没有命中其可检测条件。

继续追问

Sanitizer 都过了，能断言没有 UB 吗？

不能，只能说明已执行路径上未发现所启用检测能识别的错误；未覆盖路径、外设语义和工具盲区仍需分析。

memcpy 与 memmove 什么时候不能互换？

`memcpy` 的源与目的区域不得重叠；存在重叠时应使用 `memmove` 或证明安全的专用算法。不能依据某个 libc 版本恰好复制成功推导通用合法性。

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

GCC: Instrumentation Options — ASan/UBSan/TSan 的用途与限制。

T011 原理问答

static、extern、const、inline 和宏最容易错在哪里？

这些关键字分别控制链接、存储期、访问限制或多重定义规则，不能混成“性能优化关键字”。先区分作用域、链接性和存储期，再讨论编译器会不会内联或常量折叠。

原理与成立条件

static 的含义取决于位置

C 文件作用域 **static** 通常提供内部链接，让名字仅在该翻译单元可见；函数内 **static** 则让对象跨调用保持存储期，而作用域仍局限函数。C++ 类 **static** 成员属于类层面，不是每个实例各一份。存储期、作用域和链接性是三个不同问题。

局部静态初始化与线程安全

C++11 起局部 **static** 的初始化具有并发保护，但初始化之后对象的普通修改并不自动同步。嵌入式 C 的同名 **static** 不提供相同的 C++ 运行库初始化承诺。工具链关闭相关运行库能力或使用特别编译选项时，还需核实目标构建契约。

推演与反例

头文件中的定义为什么危险

文件作用域 **static** 通常给予内部链接，每个包含它的翻译单元可能有自己的对象；**extern** 声明通常指向别处定义。C 与 C++ 对文件作用域 **const** 的默认链接规则不同，应按语言判断。C++ **inline** 函数允许满足规则的同一定义出现在多个翻译单元中，不要求机器码一定展开；C++17 **inline** 变量也有明确的 ODR 用途。

宏展开与函数调用的本质差异

宏是预处理文本替换，例如 `MIN(i++, j++)` 可能重复求值，运算符优先级和语句上下文也可能改变含义。内联函数或模板有类型检查、作用域和一次求值语义，通常更适合普通运算。宏仍用于条件编译、字符串化等预处理任务，不能简单宣称应全部消灭。

继续追问

const 指针变量所在内存能否被别名改写？

要区分指针自身 `const`、指向对象 `const`，以及对象真实是否为 `const`；通过强转修改真实 `const` 对象可能 UB，不能把 `const_cast` 当成通用取消保护。

函数内 `static` 初始化线程安全吗？

C++11 起局部静态对象的初始化具有线程安全保证，但对象初始化后的所有业务访问不会自动加锁；C 语言和禁用相应实现机制的环境也不能直接套用这一结论。

依据与核对范围

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

C++ 工作草案: `inline` — 语言定义与优化内联的区别。项目按 C++17，未引入新标准专属特性。

T012 原理问答

虚函数、多态、函数指针和模板如何取舍？

虚函数把实现选择放到运行期，模板通常在编译期确定类型与实现，函数指针提供显式运行期回调。选择取决于变化发生在哪一层，以及是否需要跨语言、二进制接口和运行期替换。

原理与成立条件

虚函数提供运行期分派

通过基类指针/引用调用虚函数，根据对象动态类型选择最终覆盖函数。常见实现使用虚表与隐藏指针，但精确布局属于 ABI，不是语言强制每个对象固定增加某个字节数。构造/析构期间的动态分派受当前构造阶段限制，不能期待调用尚未构造完成的派生行为。

析构与对象切片

经普通基类指针 `delete` 派生对象时，基类通常需要虚析构以正确销毁完整对象；若禁止经基类删除，可采用受保护非虚析构等明确契约。把派生对象按值复制到基类会发生切片，只保留基类部分，不能再期待原来的多态效果。

函数指针与上下文参数

C 常用函数指针加 `void* context` 实现回调；函数指针负责行为，上下文指向状态。生命周期、类型匹配和重入安全由接口设计保证。C++ `std::function` 可包装多种可调用对象，方便但可能涉及类型擦除和分配；不能在有严格实时预算时未经测量就忽略开销。

模板的静态多态

模板让编译时根据类型生成和选择实现，有利于内联及消除分派，但可能增加实例化代码体积与构建复杂度。运行期插件类型未知时虚函数或回调更自然；类型已知且资源紧时模板可能合适。选择要看扩展边界、代码尺寸、测试与 **ABI**，不是单纯“虚函数慢，模板快”。

推演与反例

动态分派会增加哪些约束

通过多态基类接口调用覆盖函数，常见 **ABI** 使用虚表等机制，但标准不规定必须怎样布局。若通过基类指针 **delete** 派生对象，基类一般需要虚析构；构造和析构期间的虚调用还有当前构造层次的限制。不能把“有虚表指针”当成所有 **C++** 对象的共同结构，也不能直接序列化多态对象。

模板与回调的代价不同

模板可暴露实现给优化器、避免某些间接调用，但每组实例可能增加代码体积并扩大编译依赖。函数指针通常需要额外 **context** 指针表达状态，调用者必须保证 **context** 活到回调结束。嵌入式选择要同时衡量 **Flash**、执行时间、测试替换与接口稳定性，而不是只比较一条调用指令。

继续追问

有虚函数就一定无法内联吗？

编译器若能证明动态类型，可能去虚化并内联；反之模板中的大函数也不一定展开。标准语义与优化结果应分别说明。

用函数指针替换虚函数一定更快吗？

不一定，两者都可能是间接调用，优化器也可能去虚化。应结合生成代码和实际调用频率分析；布局、所有权与可维护性往往更重要。

依据与核对范围

C++ 工作草案: [virtual functions](#) — 运行期多态与覆盖。项目按 **C++17**，未引入新标准专属特性。

C++ Core Guidelines — [R.1/资源管理](#)、[CP/并发与接口生命周期](#)。

操作系统：执行、同步与内存

T013 原理问答

进程、线程和 FreeRTOS 任务有什么区别？

进程主要提供资源与地址空间的隔离，线程和 RTOS 任务主要描述可调度的执行流。判断差别要看共享哪些资源、切换保存哪些状态以及故障能否被隔离，不能只记“进程重、线程轻”。

原理与成立条件

进程隔离什么，线程共享什么

进程是资源与保护边界的抽象，线程是执行流的抽象。在常见 Linux 多线程进程中，线程共享地址空间、代码、堆、全局变量和文件描述符表，各自有寄存器现场、栈、线程局部存储和调度状态。线程栈虽按用途分开，却仍在共同地址空间内；一个线程持有另一线程栈地址时，硬件不会替它阻止越界访问。

切换成本为什么不同

线程切换要保存和恢复寄存器及调度状态；跨地址空间的进程切换还涉及页表上下文，并可能增加 TLB 与缓存失效成本。现代 CPU 有地址空间标识等优化，不能简单说“进程切换必然清空全部缓存”。真正成本受工作集、缓存命中和调度频率影响，不宜背固定微秒数。

RTOS 任务是不是进程

RTOS 任务同样有任务控制块、独立栈和寄存器上下文，但许多小型单片机应用让所有任务共享物理地址空间，没有 Linux 式进程隔离。MPU 可以提供区域权限保护，但 MPU 不等于支持地址翻译的 MMU。任务越界可能直接破坏其他任务、内核链表或外设状态。

选型怎样推导

需要强隔离、独立升级和故障遏制时，考虑多进程；频繁共享状态、资源紧张时，多线程或 RTOS 多任务较方便，但必须额外保证同步与生命周期。并发表示多个工作在时间上交错推进；并行要求有多个执行资源同时运行。单核抢占式多任务主要提供并发，并不把 CPU 算力变成多份。

推演与反例

从一次切换看到真实成本

同一进程的两个线程切换时，需要换寄存器、栈指针和调度状态，但通常不用换地址空间；跨进程还要改变地址翻译上下文。缓存和 TLB 是否保留取决于架构与标识机制，工作集改变仍可能导致大量未命中。因此大量细碎任务即使各只计算几微秒，也可能把时间花在调度与缓存重新变热上；成本必须结合平台和负载测量。

隔离为什么不是线程库能补出来的

同进程线程可访问彼此堆和栈地址，一个越界写可能破坏任意共享对象；互斥量只协调合法访问，无法拦住非法地址。常见无 MPU 隔离的 MCU 任务也如此。需要容错边界时，可考虑独立进程或 MPU 区域，代价是通信、资源复制和权限管理；把任务拆得更多并不会自动增加可靠性。

继续追问

一个线程崩溃是否只影响自己？

普通线程的非法内存访问可能让整个进程终止；即使没有立即终止，共享数据也可能已损坏。“各有自己的栈”不等于故障隔离。

一个任务阻塞会不会让整个单核停住？

若由调度器把它置为 **Blocked**，其他 **Ready** 任务可运行；若它关中断忙等、持有全局关键锁或阻塞的是唯一事件循环，则可能拖住更多工作。关键是阻塞发生在哪个调度层。

依据与核对范围

`pthread(7)` — 进程内共享与线程独立状态。

`fork(2)`: 进程资源 — 进程隔离与继承。

FreeRTOS 官方内核书: **Task Management** — 第 4 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 `main` 完全相同。

T014 原理问答

用户态、内核态、系统调用、中断和上下文切换是什么关系？

用户态与内核态是权限边界，上下文切换是执行流切换。系统调用、异常和中断提供进入处理代码的路径，但每次进入内核并不一定更换线程。

原理与成立条件

为什么区分用户态和内核态

权限级别使普通程序不能随意修改页表、处理中断或访问任意设备。用户程序通过系统调用请求内核服务，内核检查参数与权限，再操作受保护资源。C 库函数并不必然对应系统调用，例如字符串长度计算通常全部在用户态完成，缓冲式文件 I/O 也可能暂时不进入内核。

三类入口的区别

系统调用是程序主动发出的同步请求；缺页、非法指令等异常由当前指令触发；外部中断由设备等异步事件触发。进入处理程序时处理器保存必要返回状态，切换到相应入口与权限环境，软件再补充保存寄存器。异常与中断的命名在不同架构中略有差别，应先说明讨论平台。

模式切换不等于线程切换

`read` 进入内核后，如果数据已经可取，可以复制数据并返回原线程，只发生权限环境切换。若线程需要等待，内核将其置为睡眠状态并调度另一个线程，才发生执行流切换。中断返回也可能继续原线程，或因更高优先级工作就绪而触发调度；不能把每次中断都计成任务切换。

性能成本从哪里来

成本不仅是几条入口指令，还包括参数检查、用户地址访问、缓冲复制、锁竞争和缓存扰动。减少无效系统调用、批量处理和降低唤醒频率可能有效，但批量过大会增加尾延迟。对耗时先区分线程在运行、可运行但等 CPU，还是在等待 I/O，才能选对优化方向。

推演与反例

对比三条 `read` 路径

数据已经在内核缓冲时，`read` 检查参数、复制并返回原线程；阻塞 `fd` 暂时无数据时，线程进入等待，调度器运行其他线程；非阻塞 `fd` 无数据时，返回 `EAGAIN`，让用户决定何时重试。三者都可能进入内核，只有需要等待且另选执行流的路径必然涉及调度。一次系统调用耗时包含等待和抢占，不能全部归因于复制。

用户指针为何不能直接相信

系统调用收到的是用户提供的地址和长度，内核要处理地址是否可访问、范围是否越界以及访问过程中可能发生的缺页。权限切换并不使用户指针天然可信。类似地，设备协议的外部长度也必须先验证，再访问数组；底层代码和上层协议都需要在信任边界检查输入。

继续追问

`read` 很慢就能断定内核复制慢吗？

不能。耗时可能主要来自等数据或调度；应结合系统调用耗时、线程状态和 CPU 采样，分离等待时间与实际执行时间。

中断时保存了寄存器，是否就叫一次线程切换？

可以称为异常上下文保存，但不等同调度到另一个线程。面试中应把异常进入/退出与调度切换分开描述，避免把中断数直接当任务切换数。

依据与核对范围

`syscalls(2)` — 系统调用接口与内核边界。

T015 原理问答

虚拟内存、页表、TLB 和缺页分别解决什么问题？

虚拟内存让程序使用独立、受保护的地址空间；页表给出映射，TLB 缓存翻译结果，缺页异常处理当前映射无法直接完成的访问。TLB 未命中不等于缺页，缺页也不一定读磁盘。

原理与成立条件

虚拟地址怎样变成物理地址

程序使用虚拟地址；MMU 按当前地址空间的页表，把虚拟页号映射到物理页框，页内偏移保持不变，同时检查读写、执行和特权权限。多级页表只为实际使用的地址范围分配下级结构，减少稀疏地址空间的管理开销。连续虚拟地址不要求对应连续物理内存。

TLB 未命中与缺页不是一回事

TLB 缓存地址翻译结果。未命中时需要查页表，若页表已有有效映射，填充 TLB 后继续执行，不必从磁盘读取。缺页异常表示这次访问无法按现有映射直接完成：可能是首次按需分配、写时复制、文件页尚未驻留，也可能是权限错误或非法地址。后两类不能一概通过“调入一页”解决。

分配成功为何不等于物理内存已到位

malloc 主要向分配器申请虚拟内存范围；按需分页下，某些物理页直到首次访问才实际建立。匿名页首次写入可能需要分配并清零；文件映射缺页可能涉及存储 I/O。Linux 的内存承诺策略还会影响分配与后续访问失败的时机，所以“malloc 返回非空”不是实时访问时延的保证。

保护、共享与实时性的联系

独立页表可将不同进程相同虚拟地址映射到不同物理页，也可显式映射同一物理页实现共享。共享映射仍需同步。实时路径可预分配、预触页并在适当权限和资源预算下锁页，但锁页不消除锁竞争、调度和所有硬件延迟。必须分清虚拟容量、驻留内存与峰值工作集。

推演与反例

一次写入匿名页可能经过什么

程序先申请一段虚拟地址，第一次写某页时发现尚无可写物理映射，触发异常；内核检查该地址属于合法区域，分配或准备物理页、清零并更新页表，随后重新执行该条指令。若地址不合法或权限不允许，则走错误路径。页表已有有效映射而 TLB 没记录时，只需重新建立翻译缓存，这一过程通常不涉及存储 I/O。

写时复制与实际内存占用

fork 后父子进程最初可以共享物理页并把相关映射设为只读；一方首次写入触发写保护异常，内核为它复制该页，使两边后续修改独立。复制单位是页，不是被修改的一个字节。大量随机写会扩大驻留内存及缺页成本；RSS、虚拟地址空间大小和程序申请总量因此不能互相替代。

继续追问

有 MPU 的单片机就支持 Linux 式虚拟内存吗？

不一定。MPU 主要检查区域权限，MMU 提供页表翻译；二者能力不同，不能由“有保护单元”推出按需分页或进程地址空间。

mlock 后是否就能保证访问延迟？

锁页可减少被换出导致的等待，但仍需预触页、控制工作集及权限资源；缓存未命中、调度、中断和锁竞争依然存在，它不是完整硬实时保证。

依据与核对范围

mmap(2) — 虚拟内存与文件映射。

proc(5) — 进程、内存与运行期诊断入口。

T016 原理问答

阻塞、非阻塞、同步、异步如何区分？

阻塞描述调用是否等待，异步描述完成如何交付，两者不是同一坐标。非阻塞 I/O 通常仍由调用者在就绪后继续推进；异步接口则提交操作并在之后报告完成。

原理与成立条件

两组概念分别回答什么

阻塞与非阻塞描述调用在暂时不能取得进展时是否等待；同步与异步描述操作的完成如何交付、由谁等待并推进。讨论前必须限定接口语境：POSIX I/O 的同步定义，与把“后台线程执行”口头称为异步不是同一层概念。非阻塞 read 立即返回 EAGAIN，仍由调用者稍后重试并取得数据。

就绪通知和完成通知

select/poll/epoll 通知某个方向可能取得进展，应用仍要执行读写；这是典型 Reactor。异步提交接口则先接受操作描述，之后报告完成及结果，常称 Proactor 风格。完成前缓冲必须保持有效，取消也需要确认它不再被使用。异步并不等于没有复制，更不等于零延迟。

为什么非阻塞仍然可能占满 CPU

若不断 `read` 并在 `EAGAIN` 后立即重试，线程没有阻塞却在忙轮询。正确做法通常是暂存状态，通过事件机制等待条件变化；对小而明确的硬件等待，也可有意采用有限忙等。采用哪种方式，要比较等待长度、允许抖动、上下文切换成本与功耗。

看一个发送例子

要发送 100 字节，`write` 返回 40，只完成本次调用的 40 字节提交，余下 60 字节仍归应用管理。可写事件只表示再次尝试有意义，不能推断全部 100 字节已发送。即使接口报告本地发送操作完成，也要看它定义的是缓冲复制、DMA 结束、线路结束，还是对端确认。

事件循环数量与业务线程是两个维度

单 Reactor 单线程通常由同一线程处理 I/O 与业务；单 Reactor 多线程把业务交给其他线程；多 Reactor 多线程让多个循环分别管理一部分 I/O。多个循环可以共用业务执行器，并不要求每个循环固定配一套业务线程。单进程可以包含上述任一种布局，进程数不能替代线程数。主从 Reactor 按连接分配，按协议或设备分工也是一种多循环布局。

库的接口风格与底层实现

Boost.Asio 的 `async_read/async_write` 通过完成回调交付结果，属于 Proactor 风格接口；Linux 上可由 `epoll` 就绪机制推进底层读写。`io_context::run` 的调用线程执行处理函数，异步 API 不会自动另开业务线程。描述架构时既要说明库对应用的完成接口，也要说明事件循环由哪些线程运行，不能只凭 `async` 或 Reactor 一个词判断执行上下文。

推演与反例

读一个完整消息为什么需要状态

一次非阻塞 `read` 得到头部的前 3 字节，应用必须保留这 3 字节及解析位置，等待后续就绪。若把“返回了数据”当“消息已经完成”，协议会在合法分包时失败。完成式接口也未必替你完成业务消息定界，它可能只完成一次指定长度的底层读操作。接口完成单位与协议消息单位要分别确认。

缓冲生命周期怎样变化

同步写接口若已把数据复制进内核，调用返回后通常可复用用户缓冲；DMA 或异步接口可能继续读取提交的内存，直到其明确完成。取消请求同样可能与完成竞态，不能一发出 `cancel` 就释放缓冲。使用异步接口时，所有权问题通常比函数返回快不快更关键。

继续追问

把阻塞函数丢进工作线程就成真正异步 I/O 了吗？

对调用者可以形成异步接口，但工作线程内部仍是阻塞 I/O。应分别说明外部接口和内部执行机制，避免把线程池成本隐藏掉。

Reactor 是同步还是异步？

通常采用非阻塞同步 I/O 加就绪通知，应用负责实际读写；从业务调用者看又可能提供异步服务。必须说明讨论接口层，不能单靠 Reactor 这个名字判断所有层的语义。

依据与核对范围

`read(2)` — 读取返回值、`EINTR`、`EAGAIN`。

`epoll(7)` — 就绪集合、`ET/LT`、`EAGAIN` 与事件生命周期。

Boost.Asio: 平台实现与事件分派 — Linux 的 `epoll` 后端与可选 `io_uring`；仅说明上游机制，具体配置以本地编译为准。

Boost.Asio: `io_context::run` — 只有执行 `run/poll` 系列接口的线程分派处理函数；多个 `run` 调用者构成线程池。

T017 原理问答

互斥量、信号量、条件变量和自旋锁怎么选？

互斥量保护共享状态，信号量记录可用数量，条件变量等待共享谓词变化，自旋锁用持续运行换取免睡眠的短等待。选型先看语义和上下文，再比较开销。

原理与成立条件

条件变量为何要搭配锁

条件变量不保存业务事实，只提供等待状态变化的机制。真实条件是队列非空、停止标志等共享谓词，需要在同一同步协议下检查和修改。`wait` 原子地释放互斥量并进入等待，返回前重新取得锁；收到通知后仍要重新检查谓词。具体无丢失唤醒推导见 T018。

自旋锁什么时候合适

自旋锁等待时持续检查锁，避免睡眠和唤醒开销，适合持锁时间极短、持有者能及时运行的场景。在单核上，如果高优先级任务一直自旋，低优先级持有者可能没有机会解锁；在不可睡眠的中断上下文，更必须保证不会等待一个被自己打断的持有者。自旋锁不是“性能更好的互斥量”。

选择先看不变量和上下文

保护多个字段的一致更新，用互斥或短临界区；计数可用信号量；等待复杂状态可用互斥量加条件变量；传数据优先考虑队列。然后检查是否在 ISR、能否阻塞、锁最长占用以及取消退出。把通知工具用于保护数据，或把保护锁当成事件队列，都会造成丢事件或竞态。

Linux 互斥量为什么不一定每次进内核

常见 `pthread` 互斥实现先在用户态用原子操作尝试取得锁，无竞争时走快速路径；竞争时通过 `futex` 等机制等待内核唤醒。`futex` 的等待会检查期望值，避免在锁状态已改变后错误睡下。解锁与唤醒不表示等待者立即运行，它还要竞争锁并接受调度；自旋、睡眠和优先级协议受实现与属性影响。

推演与反例

用生产者消费者区分工具

队列内容和关闭标志是一组共享状态，用锁保证入队、出队及关闭之间的一致性；消费者等待“非空或关闭”，适合条件变量；若用计数信号量，计数必须与可消费项目严格对应，还要额外解决容器互斥与关闭唤醒。把二值通知当计数使用，会把两次到达合成一次；把计数工具当对象寿命保护，也解决不了释放后访问。

自旋什么时候会把短等待变成死锁

在单核中，高优先级代码抢占持锁的低优先级任务后若持续自旋，持锁者得不到 CPU，锁便永远不会释放。中断若等待被自己打断的临界区，也存在类似问题。自旋适合明确不可睡眠、持锁极短且有正确抢占/中断约束的上下文；用户态长临界区通常应使用会睡眠的锁或改变所有权结构。

继续追问

一个原子变量能否替代所有锁？

仅当整个不变量能由该原子协议表达时才可能。多个字段各自原子，并不使它们的组合成为同一时刻的一致快照。

futex 为什么能避免每次加锁都系统调用？

无竞争时可在用户态以原子状态转换取得锁；发生竞争才通过内核等待。等待前比较期望状态与入队的原子衔接，可防止状态已变化却错误睡下；futex 本身仍需要用户态协议构成完整互斥量。

依据与核对范围

`pthread_cond_wait(3p)` — 谓词、原子解锁等待与虚假唤醒。

`pthread_mutex_lock(3p)` — 锁所有者、阻塞和错误语义。

FreeRTOS 官方内核书：Resource Management — 第 8 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

Linux futex(7)：用户态快速路径与内核等待 — 用户态原子操作、竞争时等待/唤醒；futex 是基础机制，并非完整互斥量接口。

T018 原理问答

条件变量为什么要配谓词，如何避免丢失唤醒？

条件变量等待的是“受锁保护的条件为真”，通知只提示重新检查。正确性来自检查谓词、原子解锁等待、重新加锁检查这一整套协议，while 循环和同一把锁缺一不可。

原理与成立条件

为什么一定循环检查

允许虚假唤醒，而且被唤醒不等于立即取得锁。假设两个消费者同时被通知，消费者 A 先拿锁并取走唯一元素，B 拿到锁时队列已空。因此写 `while(!predicate) wait`，或使用带谓词的 `wait`；用 `if` 只检查一次会把“曾有人通知”错当成“现在有数据”。

停止和超时也属于协议

退出时先设置 `stopping`，再唤醒所有可能等待的线程，让它们选择排空后退出或立即取消。若超时循环每次重新等完整的相对时长，虚假唤醒会把总等待拖长；应计算一次绝对截止时间，并选适合相对期限的单调时钟。通知放在锁内或锁外都可能正确，关键是状态修改与等待判断受一致同步保护。

推演与反例

把丢唤醒的竞态写成时间线

错误路径为 C 看见 `empty`→C 解锁→P 入队并 `notify`→C 开始睡眠，此后如果没有新通知，队列虽有数据 C 仍可一直睡。正确 `wait` 把解锁与注册等待连接起来：生产者一旦能拿到同一把锁并更新状态，它发出的通知就不能落在“已解锁但尚未成为等待者”的缝隙中。通知可在解锁后发，但谓词更新仍需遵守这个保护协议。

while 同时解决两个不同问题

虚假唤醒使线程在条件没变时返回；即使没有虚假唤醒，另一个消费者也可能先拿到锁并取走数据。两种情况下返回后谓词都可为假。退出还要使用 `closed` 这样的持久状态：消费者在 `closed` 且队列空时退出，在 `closed` 但仍有数据时继续排空。仅 `notify_all` 不修改谓词，不能让线程按预期停止。

示例：关闭后排空的消费者（C++ 示意）

```
std::unique_lock<std::mutex> lk(m);
cv.wait(lk, [&] { return stopping || !queue.empty(); });
if (queue.empty()) return; // 此时 stopping 必为真
auto item = std::move(queue.front());
queue.pop();
lk.unlock();
process(item);
```

生产者必须在同一把锁下修改 `queue/stopping`，再通知。`wait` 的原子解锁并等待消除检查与休眠之间的缺口；唤醒后重新检查谓词。实际消费者循环还应处理 `process` 失败。

继续追问

为什么修改谓词为 `atomic` 后仍可能丢通知？

原子只解决该变量访问的竞态；若“检查假值→注册等待”之间没有与生产者同步，通知仍可能落在缝隙里。状态原子性与等待协议原子性是两个问题。

超时返回时能否断定队列仍然为空？

不能，超时与生产者入队可能竞争。重新取得锁后仍需检查谓词，并按接口明确截止时间与事件同时发生时的取舍；多轮等待应复用同一绝对截止时间。

依据与核对范围

`pthread_cond_wait(3p)` — 谓词、原子解锁等待与虚假唤醒。

T019 原理问答

volatile、原子操作和内存屏障有什么区别？

volatile 保留实现规定的可观察访问，原子操作解决并发访问的不可分割与语言同步关系，内存屏障约束指定层次的排序。三者解决不同问题，不能把 **volatile** 当作多线程同步。

原理与成立条件

volatile 保证的范围

在 C/C++ 中，**volatile** 用于要求编译器按实现约定保留对该对象的可观察访问，常用于内存映射外设。它不提供多线程原子性、互斥或 **happens-before**，也不保证普通数据对另一执行者的发布顺序。**volatile int** 的自增仍包含读、计算、写，两个执行者可以互相覆盖更新。

原子操作和数据竞争

原子类型的单次 **load/store** 或 **read-modify-write** 按其语义不可被拆成对其他线程可见的中间状态。C++ 中两个线程无同步地冲突访问同一普通对象，至少一个为写，就可能形成数据竞争并导致未定义行为。“我的 CPU 读 32 位只要一条指令”不能替代语言级同步关系，也不涵盖 ++ 这样的复合操作。

三层屏障不能混用

编译器屏障约束编译重排；CPU 内存屏障约束处理器访问顺序；缓存维护处理 CPU 与非一致性 DMA 之间的数据副本。**DMB** 主要建立访问顺序，**DSB** 还等待相关操作完成，**ISB** 刷新后续指令执行环境。它们不自动实现线程锁或缓存清理。C++ 线程优先用标准原子和锁，设备通信按架构及驱动契约处理。

推演与反例

release/acquire 的证明链

生产者先写 `payload=42`，再 `ready.store(true, release)`；消费者的 `acquire load` 若读取了这次 **release** 发布的值，就有“**payload** 写入先于发布、发布与读取同步、读取先于使用”的 **happens-before** 链，随后读 **payload** 才安全。若消费者只用 **relaxed**，或根本没读到对应发布，链条不成立。这个例子还要求生产者不会在消费者读取期间再次无同步修改 **payload**；重复使用同一槽需要新的握手。

为什么多核一致缓存仍然不够

缓存一致性主要约束同一位置副本之间的关系，不自动给多个普通对象建立 C++ 的线程同步。CPU 可能重排，编译器也可以在语言规则允许范围内重排；语言数据竞争还可能直接带来未定义行为。DMA 若不参与缓存一致性，另需 cache clean/invalidate 和所有权移交；单独执行 DMB 不是把脏缓存写回内存。

示例：一次性发布（无后续写者）

```
// 初始状态: payload 已构造, ready = false
// 生产线程
payload.value = 42;
ready.store(true, std::memory_order_release);

// 消费线程
if (ready.load(std::memory_order_acquire)) {
    use(payload.value);
}
```

ready 是 `atomic<bool>`。只有 acquire 读到这次 release 发布的值时，才能据同步关系访问普通字段；发布后不再修改 payload。复用同一对象需要新的交接协议，不能循环把标志改来改去就推定安全。

继续追问

shared_ptr 的引用计数原子，就能普通读写同一个 shared_ptr 变量吗？

不能。不同 shared_ptr 实例可共享控制块；同一个实例被并发修改仍需锁或对应原子访问接口，被指向对象也要独立同步。

relaxed 什么时候足够？

只需要该原子变量自身的独立统计或标志值、且不依赖它发布其他普通数据时可能足够。必须先说明它保护的不变量，不能以“单核”或“测试没出错”替代证明。

依据与核对范围

GCC: Volatiles — volatile 的访问约束，不提供普通内存同步。

C++ 工作草案：原子内存序 — release/acquire/relaxed 与可见性。项目按 C++17，未引入新标准专属特性。

T020 原理问答

死锁、活锁、饥饿和优先级反转怎么区分？

死锁是互相等待导致无法前进，活锁是一直动作但没有业务进展，饥饿是某个参与者长期得不到资源，优先级反转是高优先级被低优先级持有的资源拖住。四者的解决办法不同。

原理与成立条件

死锁的形成条件

经典资源死锁涉及互斥使用、持有资源同时等待、不能强行剥夺、形成循环等待。两个线程各持 A/B，再等 B/A，二者都不会主动推进。工程上常用统一锁顺序打破循环等待，或一次性申请、try-lock 失败后释放并重试；超时只能提供退出机会，若不回滚已持资源，仍可能继续占用并扩大故障。

活锁与饥饿的区别

活锁中线程一直在执行和响应，却相互让步或重试而没有完成有效工作；随机退避、限制重试、引入确定仲裁可改善。饥饿是某个工作长期拿不到 CPU 或资源，其他工作仍可能正常前进；公平排队、缩短占用和设置预算可缓解。CPU 很高不排除活锁，CPU 很低也不排除死锁。

优先级反转为什么多出一个中优先级任务

低优先级 L 持锁，高优先级 H 等锁，本就有必要的资源阻塞；若中优先级 M 不断抢占 L，H 就间接受 M 延迟，出现难以控制的额外反转。优先级继承把 L 临时提升，使其更快运行到解锁点；它不能让硬件转换提前完成，也不会消除循环等待。

如何系统性排查

先收集所有线程栈和资源等待图，识别谁持有、谁等待、持有者是否还能运行。再检查嵌套锁顺序、锁内 I/O、回调重入和等待另一个线程退出的 join。只在某条卡住路径加超时可能隐藏真正错误；必须同时规定超时后的资源归还、状态回滚和调用者结果。

推演与反例

死锁条件如何用于设计

互斥、持有并等待、不可抢占、循环等待同时成立时可能死锁。规定所有线程按同一顺序拿 A、B 两把锁，可以破坏循环等待；把慢 I/O 移到锁外、先取得所需资源再进入临界区，也可减少持有并等待。超时只能让一次等待退出，如果退出后没归还已持资源或继续无休止重试，仍可能形成泄漏、活锁或严重抖动。

经典三任务优先级反转

低优先级 L 持锁，高优先级 H 等锁，中优先级 M 不断抢占 L，H 就间接被不相关的 M 延迟。优先级继承使 L 暂时获得 H 的优先级，以便尽快释放锁；它不能让设备转换立即完成，也不自动解决多锁死锁。要得到时限，仍要约束最长持锁时间和更高紧急度中断。

继续追问

“每次加锁都只等 1 秒”能保证没有死锁吗？

若超时后正确释放并回滚，可避免永久等待；若仍持其他锁循环重试，系统可能转成活锁，甚至一直无法恢复一致状态。

不用锁是否就没有死锁？

没有。两个线程可能互相 `join`，两个有界队列可能互相等待对方消费，协议双方也可能都等一个永远不会到来的消息。需要分析的是等待依赖图。

依据与核对范围

FreeRTOS: `Mutexes` — 优先级继承及任务上下文。

`pthread_mutex_lock(3p)` — 锁所有者、阻塞和错误语义。

FreeRTOS 官方内核书: `Resource Management` — 第 8 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 `main` 完全相同。

T021 原理问答

Linux 调度与 FreeRTOS 调度有什么不同？

通用 Linux 调度追求多任务响应与公平，实时调度强调优先级或期限约束；常见单核抢占式 FreeRTOS 选择最高优先级就绪任务。无论哪种调度器，“高优先级”都不等于已经证明最坏响应时间。

原理与成立条件

RTOS 调度强调什么

典型单核抢占式 FreeRTOS 选择最高优先级就绪任务；高优先级任务阻塞后才让较低优先级运行。同优先级是否轮转受时间片等配置影响。优先级是选择执行者的顺序，不是 CPU 份额；一个始终就绪的高优先级循环可以让低优先级永久得不到运行机会。

Linux 策略不能混成一类

普通调度追求公平和交互表现，`nice` 影响相对权重；其具体算法与内核版本相关，例如 `EEVDF`。

`SCHED_FIFO`、`SCHED_RR` 是固定实时优先级策略，`RR` 对同优先级加入时间片；`SCHED_DEADLINE` 使用运行预算、截止时间和周期等参数。策略名称并不证明整机满足硬实时，驱动、锁和中断也会干扰截止时间。

如何解释最坏响应时间

对有界、周期或偶发任务，在限定调度模型下，可迭代估算 $R_i = C_i + B_i + \sum \text{ceil}(R_i/T_j)C_j$ ；自身最坏执行 `C`、低优先级资源阻塞 `B`，以及更高优先级任务的干扰。实际还要纳入中断、调度开销和释放抖动；若任务自挂起或执行时间无界，不能直接照搬简化公式。

从公式反推设计

降低平均 CPU 不一定改善最坏响应；缩短最长关中断区、限制锁内耗时和为长操作设置有限超时，往往更直接。周期越短的任务常需要较高优先级，但不是盲目按周期排序；必须结合截止时间、依赖和共享资源。实时性是“在约束内完成”，不是“越快越好”。

推演与反例

固定优先级的干扰怎样计算

高优先级任务即使只执行 1 ms，若每 2 ms 就释放一次，也会消耗约一半 CPU。低优先级任务除自身执行时间外，还要承受所有高优先级任务和中断的干扰，以及资源阻塞。一个任务 `taskYIELD` 后只要自己仍是最高优先级就绪任务，就不保证低优先级运行；等待实际事件而阻塞才会从就绪竞争中退出。

调度策略与系统前提

Linux 的 `SCHED_FIFO`、`SCHED_RR` 与普通策略不同，但仍受权限、实时带宽限制、内核可抢占性和驱动长临界区影响；目标内核使用 CFS 还是 EEVDF 应按版本确认。FreeRTOS 也要核对抢占、时间片、Tick 频率与移植层配置。同名优先级在 NVIC 和任务调度中的数值方向甚至可能相反。

继续追问

CPU 只有 20% 为什么还超时？

平均利用率掩盖了突发、长临界区和资源阻塞。一次 30 ms 的不可抢占区就可能让 10 ms 截止时间失败，即使其余时间 CPU 都空闲。

CPU 平均利用率低，能证明任务不超时吗？

不能。短时间内的突发、长临界区、优先级反转和 I/O 等待都可能错过期限。实时性应看最坏响应和到达模式，而不只看一分钟平均值。

依据与核对范围

Linux Kernel: `EEVDF Scheduler` — 调度实现版本边界，不预设目标机内核。

`sched(7)` — 普通与实时调度策略。

FreeRTOS 官方内核书: `Task Management` — 第 4 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

University of York: 固定优先级响应时间分析 — Paul Emberson 博士论文，2009，§2.2.2，式 2.3–2.5；本文示例采用单核固定优先级、无释放抖动的受限模型。

T022 原理问答

中断上半部、下半部与任务化处理为什么重要？

中断前段负责必须及时处理的硬件状态，后续任务负责可以调度的解析和慢工作。拆分的目的是限制不可调度时间，并明确数据怎样从硬件上下文安全交给任务。

原理与成立条件

中断处理为什么分前后两段

中断前段直接响应硬件：读状态、清应清的标志、保存必须及时取得的数据，并安排后续工作。复杂解析、存储、协议重试放到可调度上下文，避免一个设备延迟所有同级或更低紧急度工作。分段的本质是限制不可调度时间，不是机械规定中断中只能写几行代码。

延后执行也有不同上下文

Linux 硬中断与通常的 `softirq/tasklet` 不能按普通进程上下文随意睡眠；工作队列的工作函数在内核线程上下文运行，通常可睡眠，线程化中断也有自己的约束。RTOS 常由 ISR 通知专用任务执行后半段。不能因为都叫“下半部”，就认为每种机制都能拿阻塞锁。

怎样保证交接不丢数据

ISR 与任务间需要数据容器以及唤醒协议。先保存数据，再通知消费者，避免消费者先醒来却看不到任务。缓冲容量至少覆盖消费者最长不能服务期间的输入，并明确满时策略；只发一个二值事件可能合并多次输入，所以消费时应排空数据或读取生产进度，而不是把事件数等同于字节数。

延迟预算应怎样拆

从设备产生事件，到 ISR 获得执行机会，再到任务就绪、获得 CPU、完成业务，是四段不同延迟。缩短 ISR 可能降低前两段，却增加排队或复制成本；应测最大中断屏蔽时间、缓冲高水位和任务唤醒延迟。DMA 可减少逐字节搬运，但不替你完成协议解析和流量控制。

推演与反例

延后处理为什么仍需要容量证明

ISR 只把数据放入缓冲然后唤醒任务，看似很短，但如果任务长时间拿不到 CPU，缓冲仍会满。必须计算中断到达速率、最长屏蔽时间、任务最坏调度延迟和消费速率；复制或 DMA 都只是改变搬运方式。一次 ISR 放入数据后立即退出，是把后续工作转移出去，而不是把工作消灭。

不同平台的“下半部”不能混用

Linux 的 `softirq`、`tasklet`、工作队列和线程化中断有不同上下文限制，尤其能否睡眠不能统一回答。MCU 上常用 ISR 加任务通知/队列形成类似分工，但不是运行 Linux 的 `softirq`。描述机制时应先说平台，再说谁运行、何时可抢占、共享数据如何同步以及满缓冲怎么办。

继续追问

中断中 `printf` 为什么危险？

格式化、锁、发送等待和潜在分配都可能耗时或不可重入。可在 ISR 写入固定大小事件记录，由任务格式化输出，并处理记录区满的情况。

把所有工作都放入最高优先级任务会怎样？

它可迅速接手中断，却可能长期饿死其他任务；若任务内仍做无界等待，响应链依旧无界。需要短工作段、适当优先级和可阻塞的等待点。

依据与核对范围

Linux Kernel: [Generic IRQ handling](#) — 硬中断与延后处理的上下文。

FreeRTOS: [Cortex-M interrupt priorities](#) — 优先级编码、BASEPRI 与可调用 ISR 范围。

FreeRTOS 官方内核书: [Interrupt Management](#) — 第 7 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

T023 原理问答

fork、exec、写时复制与进程间通信怎么理解？

fork 创建一个新的进程执行流，exec 用新程序替换当前进程映像，两者不是同一个操作。父子可通过继承的文件描述符通信，而普通私有内存修改不会自动同步。

原理与成立条件

fork 创建了什么

fork 创建子进程，子进程拥有独立的虚拟地址空间视图，继承许多资源引用。Linux 常用写时复制让父子初始共享物理页，只在某方需要写时分离；页表和进程管理仍有成本，所以不能说 fork 完全零拷贝或父子变量永久共享。

exec 与 fork 的区别

exec 用新程序映像替换当前进程的代码、数据和初始执行环境，成功后不返回旧调用点，进程身份并不是简单另起一个子进程。exec 本身不负责先 fork；shell 常组合二者完成新命令。文件描述符是否保留取决于 close-on-exec 等状态，泄漏 fd 会让管道和服务退出异常。

多线程 fork 的特殊风险

子进程通常只保留调用 fork 的线程，其他线程消失，但它们持有的锁状态可能留下。子进程在 exec 前若调用 malloc、日志等可能拿到无人释放的锁；应遵守 POSIX 对此阶段可调用函数的限制，必要时使用合适的 spawn 接口或严格的 atfork 协调，而非随意复制多线程运行态。

IPC 选择依据

管道适合相关进程的顺序字节流，Unix socket 适合双向与本机消息/凭证等能力，共享内存避免部分数据复制但需要独立同步和布局协议，消息队列提供消息边界但有容量限制。共享内存不是“自动线程安全”，信号也不适合搬运大量可靠业务数据。

推演与反例

从返回值理解两条执行流

`fork` 成功后，父进程得到子 `PID`，子进程得到 `0`，两边从调用之后分别继续；`exec` 成功时不返回原程序，而从新映像入口执行。现代 `Linux` 的写时复制减少了 `fork` 立即复制全部内存的成本，但页表、任务状态和后续首次写页仍有开销。把 `fork` 称为“复制一个线程”会漏掉地址空间和资源语义。

多线程 `fork` 的锁为什么危险

子进程只保留调用 `fork` 的线程，但地址空间中可能留下另一个已不存在于子进程的线程所持有的锁。子进程再调用依赖该锁的库函数可能卡住。`POSIX` 对多线程 `fork` 后、`exec` 前可调用的函数有严格限制；优先使用合适的 `spawn/exec` 流程，不把普通线程库操作任意塞进子分支。

继续追问

`fork` 后两个进程写同一个继承 `fd`，偏移是否独立？

它们常引用同一 `open file description`，文件偏移共享；进程地址空间独立不代表所有内核资源对象都独立。

共享内存是不是最快就一定最好？

它可减少数据复制，但必须自己解决通知、同步、数据布局和崩溃恢复；管道/`socket` 自带消息传递所需的一部分生命周期边界。选择应结合数据量与故障隔离要求。

依据与核对范围

`fork(2)` — 写时复制和多线程 `fork` 限制。

`exec(3)` — 进程映像替换。

`sysvipc(7)` — `IPC` 家族；与管道/`socket` 选型结合。

T024 原理问答

`fd`、`open file description`、`dup` 与 `close` 有什么区别？

`fd` 是进程中的整数索引，`open file description` 是内核维护的一次打开状态。`dup` 或 `fork` 继承的 `fd` 可以共享该状态，因此“两个 `fd`”不必然对应两份独立文件偏移或状态标志。

原理与成立条件

fd、打开文件描述与文件对象

fd 是进程描述符表里的整数索引；它指向内核的 **open file description**，其中保存文件偏移和部分状态标志，后者再关联具体文件/设备。对同一路径调用两次 **open** 通常得到两个独立打开描述；**dup** 得到新 fd，却共享原打开描述。

dup 共享与不共享的状态

共享偏移和 **O_NONBLOCK** 等文件状态标志，意味着通过一个副本读写或改变状态会影响另一个；**FD_CLOEXEC** 属于每个描述符的标志，不是共享打开描述属性。把某个 fd 复制一份不等于创建独立通信连接，也不自动建立新的读写顺序隔离。

close 只释放一个引用

只有相关引用都释放后，内核对象才最终关闭；对 **socket** 还可用 **shutdown** 表达方向结束。**epoll** 注册与打开描述的关联使 **dup/fork** 后关闭一个数值不一定立即消除所有事件，应用需明确注销。整数被复用后，旧异步操作若仅凭 fd 数值识别对象就可能误操作新连接。

close 错误不能机械重试

Linux 在某些 close 错误报告之前已释放 fd 数值，重试可能关闭其他线程刚打开并复用该数字的对象；**EINTR** 的跨平台语义也需要按系统核实。资源封装应使自身进入不拥有状态，并通过合适通道记录错误，而非无限重复 close。

推演与反例

dup 后谁的偏移会变

fd1 与 dup 得到的 fd2 通常指向同一 **open file description**，一边 **read** 推进的偏移会影响另一边；另一次独立 **open** 则通常有自己的打开状态。**O_NONBLOCK** 属于文件状态标志，相关描述符可共享；**FD_CLOEXEC** 是描述符标志。区分两层，才能解释子进程继承、关闭和 **epoll** 注册的行为。

fd 重用怎样制造误操作

连接 A 的 fd=7 被关闭后，新连接 B 可能立即拿到 7。一个迟到任务若只记着整数 7，再调用 **write** 或 **close**，可能操作 B。因此异步身份应带连接对象/代数并校验寿命；关闭一次失败后盲目再 **close** 同一数字，在 Linux 的某些失败情形下可能关闭已复用描述符，应按具体接口语义处理。

继续追问

为什么多线程程序创建 fd 时优先原子设置 CLOEXEC?

若先 **open** 再 **fcntl**，另一线程可能在中间 **fork/exec**，让 fd 泄漏进新程序。使用支持的 **O_CLOEXEC** 等原子接口缩小该竞态。

RAII 已经保证 `close`，为什么还要取消回调？

RAII 解决资源最终释放，无法自动删除外部队列里尚未执行的借用操作。回调的有效期必须早于资源释放，或能检测对象已失效。

依据与核对范围

`open(2)` — `fd` 与 `open file description`。

`dup(2)` — 共享打开状态与 `fd` 标志。

`close(2)` — 关闭、`fd` 复用和错误处理。

T025 原理问答

Cache、伪共享和内存序为什么会影​​响多核并发？

Cache 改变访问成本，缓存一致性协调同一地址的多份缓存，内存序约束并发访问的先后关系。三者相关但不等价；“硬件缓存一致”不能替代语言同步。

原理与成立条件

缓存一致性按缓存行工作

多核 CPU 缓存通常以固定大小缓存行为传输和一致性管理单位。两个线程分别更新不同变量，语言层面没有共享同一对象，却可能因变量位于同一缓存行而互相使缓存副本失效，产生不必要的一致性流量，这叫伪共享。

为什么原子变量也会很慢

原子性保证更新语义，不消除缓存行取得写权限的成本。两个核频繁写同一行中各自的原子计数器，缓存行仍可能来回迁移。把内存序从 `seq_cst` 改为 `relaxed` 只能去掉不必要排序约束，无法自动消除同一缓存行的争用。

怎样验证再优化

先定位热点及核间访问，结合 `perf c2c`、数据布局与平台支持查看高争用缓存行，再用控制实验把热写字段分开或分线程计数后聚合。单纯看到多线程就给所有字段填充，会增加内存、缓存和 TLB 压力，可能得不偿失；缓存行大小也不是所有 CPU 都固定 64 B。

与真正共享及 DMA 区分

如果多个线程本就更新同一业务计数，这是实际共享，需要改变算法或分片，不只是加 `padding`。DMA 一致性又涉及设备与 CPU 缓存副本，应按平台维护。无 D-cache 的小型单核 MCU 不存在相同的多核缓存行争用问题，不能把 Linux 服务器经验照搬过去。

推演与反例

伪共享为什么没有逻辑共享仍会变慢

两个线程分别修改不同计数器，数据没有竞争，却恰好位于同一缓存行。每次写可能使该行的所有权在核心间转移，产生额外一致性流量。把热点写字段分散到不同缓存行可改善，但具体行大小需按目标确认；盲目填充所有结构体会增加工作集和内存占用。

DMA 与缓存是另一种所有权问题

非一致性 DMA 从内存读，CPU 最新数据可能仍在脏缓存中；DMA 写完内存，CPU 又可能读到旧缓存。需要按方向在规定时机 `clean` 或 `invalidate`，并遵守缓存行对齐和共享行边界。Cortex-M3 的 STM32F103 不应直接套用带 D-cache 的 M7 操作，但移植到后者时必须重新审计这条链。

继续追问

两个只读线程读同一缓存行算伪共享吗？

一般不会触发这种写失效抖动；关键是至少有频繁写入，使其他核需要重新取得该行。共享读取常能有效利用缓存。

加 **memory fence** 能修复伪共享吗？

不能。屏障可能增加开销，它不改变两个字段处于同一缓存行的事实。伪共享是布局与访问模式问题；缺少同步则是另一个正确性问题。

依据与核对范围

Linux Kernel: [False Sharing](#) — What is false sharing 、How to detect 、Possible Mitigations；缓存行争用与测量。

C++ 工作草案：原子内存序 — `release/acquire/relaxed` 与可见性。项目按 C++17，未引入新标准专属特性。

Linux I/O 与事件驱动

T026 原理问答

select、poll、epoll 的区别，epoll 真的是 O(1) 吗？

select、poll 每次提交要检查的描述符集合；epoll 把关注集合长期保存在内核中，由事件把相关描述符加入就绪集合。它主要减少大量空闲连接下反复传递和扫描集合的工作，不代表整个服务器所有操作都是 O(1)。

原理与成立条件

select 与 poll 在做什么

它们接收本次关注的描述符集合，内核检查这些对象的状态，返回后应用再识别哪些可读写。select 使用位图及 nfd 范围，常见 glibc fd_set 有 FD_SETSIZE 限制；poll 使用结构数组，没有同样的位图上限，但仍受系统 fd 与资源限制。集合很大而活跃项很少时，反复传递和扫描有成本。

epoll 的区别在哪里

epoll 把关注关系长期注册在内核中，通过 epoll_ctl 增删改；epoll_wait 返回本次就绪项，避免应用每次重新提交所有关注对象。内核通过被监视对象的通知机制维护就绪状态。因此它对大量空闲连接很合适，但注册、注销、事件复制和回调执行仍然有成本。

为什么不能背“epoll 永远 O(1)”

一次返回 k 个事件至少要处理 k 项，连接频繁变化还会增加控制操作，全部连接持续活跃时也要执行全部业务。不同实现的数据结构复杂度不能替代端到端成本模型。连接少时，poll 的简单数组和局部性也可能足够好，不能仅凭 API 名称宣称性能提升倍数。

就绪集合并非完整业务结果

读就绪可能代表有字节、EOF 或错误；写就绪只表示可能写入部分内容。事件也会合并，不能按事件次数计算消息数。应用必须继续检查实际 I/O 返回值、保存部分状态，并处理错误、挂断和资源关闭。事件驱动的正确性来自这些状态规则，而不只是调用 epoll_wait。

推演与反例

用 N 个连接、 K 个就绪连接看成本

`select/poll` 的一次等待通常需要处理规模为 N 的集合，返回后用户也要找到哪些项就绪；`epoll_wait` 主要交付就绪项，应用仍至少处理 K 个结果。注册变更、内核回调、锁、复制和业务处理都有成本。若 N 很小或几乎所有连接都持续就绪，`epoll` 的优势未必明显；“10 万连接”也不代表 10 万个连接都能同时执行复杂业务。

描述符数量之外还有什么限制

`select` 的常见 `fd_set` 有固定表示上限；`poll` 没有同样的位图接口限制，但集合仍需传递和扫描。`epoll` 关注表同样占内核内存，并受到资源限制。连接规模还受 `socket` 缓冲、用户对象、HTTP 输入上限、`fd` 限额和每连接处理成本影响，不能只调高 `ulimit` 就认为系统具备相同容量。

继续追问

`epoll` 比 `select` 快，是否因为它完全没有用户态与内核态复制？

不是。就绪事件仍需要交付给用户态，实际读写也可能复制数据。主要区别在关注集合的维护和就绪分发方式，不能称为自动零拷贝。

`epoll` 是零拷贝接口吗？

不是。它改变就绪事件的管理方式，事件数组仍要交付用户态，实际 `read/write` 也可能复制数据。I/O 多路复用与数据零拷贝是不同问题。

依据与核对范围

`epoll(7)` — 就绪集合、ET/LT、EAGAIN 与事件生命周期。

`select(2)` — 集合、描述符表示限制。

`poll(2)` — 就绪查询与接口差别。

T027 原理问答

ET 和 LT 的区别，为什么 ET 要读到 EAGAIN？

LT 在就绪条件持续满足时仍可报告，ET 主要报告就绪状态的变化。ET 要求应用记住尚未处理完的工作，通常以非阻塞 I/O 读写到 EAGAIN 才重新等待，否则可能把已经存在的数据留在内核里。

原理与成立条件

为什么要用非阻塞并读到 **EAGAIN**

循环 **read**，正数就消费，**EINTR** 按语义重试，**EAGAIN** 表示暂时无法继续，此时才能可靠回到事件等待。如果 **fd** 是阻塞的，读完已有数据后的下一次 **read** 可能把整个事件循环挂住。**EOF**、设备错误与串口特殊 **read** 语义还需要分别处理，不能统一作为 **EAGAIN**。

写事件应该怎样管理

业务先把待发数据放入输出缓冲并尝试写；写不完时保留偏移并关注可写事件，继续直至缓冲排空或 **EAGAIN**。**LT** 下无数据仍一直监听 **EPOLLOUT**，容易不断醒来形成空转。输出缓冲也要有上限，慢对端可能让内存增长远快于内核缓冲消耗。

一直读到空会不会饿死其他连接

会。高吞吐连接可能持续补入数据，让一次回调占用过久。可设置字节或时间预算，但 **ET** 下提前退出时必须把该连接留在应用就绪队列，主动轮转继续处理；直接再次等内核新边沿会丢失继续推进的机会。公平与 **ET** 完整消费必须一起设计。

推演与反例

半读为何可能永久等不到下一次通知

管道已有 2 KB，收到一次可读事件后只读走 1 KB。剩余 1 KB 仍可立即读取，但状态没有经历新的“从不可读到可读”的转换；如果对端等你的应答而不再写，你再 **epoll_wait** 就可能卡住。读到 **EAGAIN** 表示这一轮可取得的进展已耗尽，再等待才有依据。就绪只是提示，最终仍要按 **read** 的返回值判断。

读到 **EAGAIN** 与公平性怎样兼容

持续高流量可能让一个连接长时间都读不到 **EAGAIN**，饿死其他连接。可为每轮处理设置预算，同时把该连接保存在用户态待处理集合，轮转后继续读；只有真正耗尽时才移除。不能达到预算就丢掉待处理标记、直接等下一条 **ET** 通知，否则又回到漏处理问题。写方向也应只在有待发数据时关心 **EPOLLOUT**。

继续追问

EPOLLONESHOT 能代替并发保护吗？

它可在一次报告后暂停该关注项，便于把连接交给一个工作者，处理后需重新 **MOD** 激活；对象生命周期、其他入口访问和重新激活的竞态仍要管理。

EPOLLONESHOT 能自动解决并发安全吗？

它能让一次事件交付后暂停该项通知，便于由一个工作者处理后重新激活，但对对象寿命、缓冲同步和 **rearm** 时机仍由应用负责。

T028 原理问答

串口短写、EINTR、EAGAIN 分别怎么处理？

短写表示这次调用只接受了前一部分数据，EAGAIN 表示目前无法继续，EINTR 表示被信号中断。正确发送器保存未发送后缀及偏移，只在能够继续时推进，不能把它们统一当作整帧失败后从头重发。

原理与成立条件

三类结果要分别处理

EINTR 通常表示在没有返回有效进度前被信号打断，应结合接口语义重试；EAGAIN/EWOULDBLOCK 表示非阻塞条件暂不满足，要等就绪。其他错误按连接或设备故障处理。对非零请求意外返回 0 的写操作，应防止无限零进度循环，而不是无条件重试到 CPU 满载。

本地提交、线路完成和对端接收

串口 write 成功常表示数据进入驱动/内核缓冲，硬件可能还在发送。tcdrain 等待终端输出传输完成，但仍不能证明对端正确接收和执行。应用层 ACK 才能增加对远端状态的认知，因此命名发送回调时应精确描述它确认的是哪一步。

取消发送为何需要状态

帧尚未写出任何字节时，可从输出队列直接删除；已有前缀发出后，通常需要继续发送完整帧，或使用双方约定的链路复位/重新同步机制。取消业务等待和停止线路传输是两个动作。缓存必须一直活到驱动不再访问，不能超时就释放仍被异步发送使用的内存。

推演与反例

用 100 字节演示状态推进

初始 offset=0，write 返回 40，就把 offset 改为 40；下一次从 data+40 发送剩余 60。返回 EAGAIN 时偏移不变，保留数据并等待可写；返回正数即消费那部分进展。若所有 100 字节都被内核接受，才完成本地提交。若此时业务期限到了，已写出的前 40 字节不能从线路撤回，随意丢掉后缀再写新帧可能破坏对端分帧。

错误处理为什么必须保留错误类别

真正的设备断开、无进展返回 0、暂时不可写和 EINTR 需要不同策略。回调还可能修改发送队列，所以完成当前项时应先取得回调、移除当前项，再调用外部代码，避免回调使正在引用的队首失效。fd 的发送完成只代表该接口约定的完成层级，不代表对端已经收到或执行。

示例：非阻塞发送保留偏移（伪代码）

```
while offset < frame.size:
    n = write(fd, frame.data + offset, frame.size - offset)
    if n > 0: offset += n; continue
    if n < 0 and errno == EINTR: continue
    if n < 0 and errno in {EAGAIN, EWOULDBLOCK}:
        watch_writable(); return PENDING
    return FAILED // 包括无进展的 n == 0, 避免无限空转
return LOCAL_KERNEL_ACCEPTED
```

frame 的存储和 **offset** 必须跨回调存活；同一字节流只由一个发送序列推进。最后的返回值刻意命名为本地内核接纳，不表示对端执行完成。

继续追问

为什么单写者有助于正确处理短写？

它让偏移和帧 **FIFO** 只有一个修改者，避免多个线程把两帧余下部分交叉写入同一字节流。单写者并不免除容量限制和错误处理。

一个 **write** 返回整帧长度，**RTO** 就能忽略本地发送时间了吗？

不能，驱动或 **UART** 仍可能排队。**RTO** 起点可以选择“内核接纳完成”，但预算必须包含此后的本地发送、对端处理和返回延迟。

依据与核对范围

write(2) — 短写、错误与完成边界。

termios(3) — 串口原始模式、**VMIN/VTIME** 和 **tcdrain**。

T029 原理问答

eventfd 为什么配一个队列，计数是否等于命令条数？

eventfd 是可纳入 **epoll** 的计数型唤醒通道，队列才保存命令或任务内容。通知可以合并，消费者以队列为真实状态，不能把读出的计数机械当作必须处理的命令数。

原理与成立条件

为什么要另配队列

队列保存数据与顺序，`eventfd` 告诉消费者“可能有新工作”。生产者先在同步保护下入队，再发通知；消费者醒来消费队列状态，而不是只按某一次事件次数推断队列内容。内核 `fd` 通知不能替代 `C++` 对队列的互斥或原子同步，容器的数据竞争仍要单独解决。

有限批处理的继续条件

如果一次只消费 32 项，队列可能还有剩余，而 `eventfd` 累计值已被读空。此时必须让程序主动安排下一轮处理或再次通知；否则没有新生产者时，旧工作可能永远留在队列中。空队列与写入之间的竞态也要推演，但先入队后通知让多余唤醒通常只造成一次空检查。

计数器满和关闭如何处理

非阻塞写在计数器无法继续增加时可返回 `EAGAIN`；这往往意味着已有未消费通知，但仍需明确协议。关闭时先禁止生产者继续入队/写通知，再等待它们结束，最后关闭 `fd`；否则 `fd` 数值被复用后，旧生产者可能写到无关对象。`EINTR` 和写失败也不应被当作业务已消费。

推演与反例

两个生产者如何安全唤醒一个消费者

生产者先在同步保护下入队，再写 `eventfd`。消费者读取计数以清除通知状态，然后检查并消费队列；即使多个写被合并，队列中任务仍在。若先通知后入队，消费者可能醒来发现空队列并再次睡眠，生产者随后入队却没有新通知。入队顺序与唤醒协议必须成套考虑，`eventfd` 不会自动使一个不安全的容器变安全。

限额消费后还有数据怎么办

一次只消费 32 项可以降低对其他事件的干扰，但队列还没空时必须保留“还有工作”的事实，例如自行再通知或进入本地待运行列表。若只是把 `eventfd` 读到零然后提前返回，剩余数据可能要等下一位生产者碰巧唤醒。计数满时非阻塞写 `EAGAIN` 一般说明已有唤醒待消费，但其他写错误仍应处理。

继续追问

每入队一次就 `write 1`，读取数值是否等于当前队列长度？

不保证。消费者可并行取走数据，通知可以多余，其他生产者也可正好入队；队列自身才是工作数量的权威来源。

`EFD_SEMAPHORE` 改变什么？

普通读通常取走累计值并清零；信号量模式每次读返回 1 并递减计数。两者都不保存业务载荷，也不替代任务队列的容量与退出设计。

T030 原理问答

timerfd 怎样报告超时，时钟和定时精度如何选择？

timerfd 把定时到期变成可读事件，便于与其他 fd 在同一事件循环处理。定时器决定何时到期，线程何时获得 CPU、何时运行回调是另一件事，因此超时回调可能晚于配置期限。

原理与成立条件

定时器只报告到期

timerfd 把定时器到期变成可读事件，读取的 64 位数表示自上次读取后累计的到期次数。线程晚运行时可一次读到多个到期，不表示曾及时执行过多次回调。定时器分辨率、调度延迟和业务处理时间共同决定实际反应时间。

时钟选错会产生什么问题

CLOCK_REALTIME 表示日历时间，可能被设置而跳变；CLOCK_MONOTONIC 适合通常的经过时间，不含系统挂起时间；若期限需要把系统挂起也算进去，Linux 可考虑 CLOCK_BOOTTIME。超时逻辑应在同一时钟域比较，不能拿墙上时间戳减单调时间，也不能假设定时源不受平台配置影响。

三个时间点不能混淆

业务排队起点决定总寿命，真正提交发送决定一轮响应等待起点，获得接收确认后可能进入另一结果等待期限。若把发送重试时刻不断重置成总期限起点，持续重试可能使请求永不过期；若在长队列等待时就起短 RTT 定时，又会导致未发送先重试。

扫描精度怎样估算

每 Δ 毫秒扫描截止时间，理想情况下到期发现额外延迟小于一个扫描间隔；实际还要加调度与回调阻塞。因此“超时 500 ms”通常表示判断阈值，不能承诺恰在第 500 ms 回调。可改成按最近截止时间设单次定时，但仍不能免除执行者被阻塞的延迟。

推演与反例

相对期限为什么用单调时钟

墙上时间可能被校时向前或向后调整，用它计算“等待 500 ms”会产生误判；CLOCK_MONOTONIC 避免这类跳变，但不包含系统挂起时间，若业务要求把 suspend 算入期限，需考虑 CLOCK_BOOTTIME。绝对业务时间戳和相对超时时应分别选时钟，不能用一个毫秒值隐去时钟域。

扫描周期怎样带来额外延迟

若每 200 ms 扫一次请求表，某请求在 $t=501$ ms 到期而上一轮在 500 ms 扫过，通常要到 700 ms 左右才被发现，再叠加回调排队和调度延迟。读取 `timerfd` 得到累计到期次数，可以知道错过了多个周期，但业务应按当前单调时间判断 `deadline`，不应简单把超时判定次数当真实经过时间。

继续追问

到期次数为 5，要把所有业务处理重复调用 5 遍吗？

不必然。计数告诉你错过了节拍，业务可按当前单调时间检查真实截止时间；周期控制是否补做、跳过或合并由业务决定。

把扫描周期改小一定更好吗？

它减少离散扫描误差，却增加唤醒和遍历成本。请求多时可用按截止时间排序的结构、时间轮或重设最近到期的一次性定时器；选型要结合数量和允许误差。

依据与核对范围

`timerfd_create(2)` — 时钟选择、到期计数与 `fd` 通知。

T031 原理问答

为什么用 `signalfd`，为什么要在创建线程前屏蔽信号？

`signalfd` 把选定信号转换成普通读取事件，使退出和重载逻辑在正常线程上下文执行。前提是相关线程正确屏蔽这些信号，避免信号走传统处理路径。

原理与成立条件

传统信号处理的限制

异步信号可能打断线程正在执行的库函数，因此处理器中不能随意调用非 `async-signal-safe` 的分配、日志或带锁函数。例如主线程持日志锁时被信号打断，处理器再取同一锁可能死锁。处理器应极简，或把信号转入普通执行上下文处理。

`signalfd` 怎样转换模型

先把目标信号加入信号掩码阻塞，再创建 `signalfd` 并纳入事件循环。信号产生后成为可读取的记录，程序在正常回调上下文执行停止或重载逻辑。`signalfd` 不是让信号失去语义，而是改变接收方式；哪些线程屏蔽该信号仍直接影响它会被谁处理。

为什么要在创建线程之前屏蔽

新线程继承创建者的信号掩码。主线程先屏蔽再启动工作线程，可以避免进程定向信号跑到某个未屏蔽线程触发默认处理。若线程已经存在，需要确保所有相关线程的掩码配置一致；只改主线程并不足以完成这个约束。

信号不能当可靠任务队列

普通信号待处理期间多次出现可能合并；实时信号有排队语义但也有资源限制。线程定向信号与进程定向信号还影响谁能接收。业务命令需要参数、顺序和可靠接纳时应使用专门队列或 IPC，而不是给每个业务动作随意分配一个普通信号。

管理事件循环与管理工作线程的分工

当配置加载、域名解析或连接重建可能阻塞时，可由独立管理事件循环接收 `signalfd`，再投递任务给工作线程。停止通知不应先排到这个慢任务队列尾部；事件线程可以直接发布停止状态并唤醒目标循环。不过最后 `join` 工作线程仍可能等待阻塞调用，及时接收退出请求和有限时间完成退出是两项不同保证。信号掩码配置失败时应在派生线程前中止启动，避免默认处理路径与 `signalfd` 路径混用。

推演与反例

为什么必须在线程创建前准备

线程有自己的信号掩码，新线程通常继承创建者当时的掩码。若部分工作线程已经运行且没有屏蔽进程定向信号，内核可能把信号交给它们，而不是让指定事件线程通过 `signalfd` 消费。先设置掩码再创建线程，可以让继承关系成为设计的一部分；之后再创建 `signalfd` 仍不等同于自动修改所有线程掩码。

普通信号处理函数的限制来自哪里

信号可打断任意执行位置，例如线程正持有 `malloc` 或日志内部锁。处理函数再调用非异步信号安全函数，可能重入受损状态或死锁。`signalfd` 回调则在事件循环中的普通调用路径，可安排资源重建与退出；但回调依旧不能无界阻塞所属事件循环。标准信号还可能合并，不能把它当可靠消息队列。

继续追问

收到退出信号后在 `signalfd` 回调里直接销毁所有对象可行吗？

仍要遵循生产者停止、线程 `join`、队列排空和资源释放顺序。普通上下文解除的是信号安全限制，不是生命周期约束。

`SIGKILL` 能通过 `signalfd` 优雅处理吗？

不能，`SIGKILL` 和 `SIGSTOP` 不能被捕获、忽略或这样接管。强制终止后的恢复必须依赖外部持久状态与启动恢复机制。

依据与核对范围

`signalfd(2)` — 信号掩码、线程继承与 `fd` 读取。

T032 原理问答

回调删除连接时，如何避免 **use-after-free**?

回调对象必须活到所有可能引用它的已排队操作结束。把 `fd` 从 `epoll` 注销只阻止后续注册产生事件，并不会抹掉已经返回到用户态事件数组中的指针。

原理与成立条件

为什么注销后仍可能访问对象

`epoll_wait` 已经把事件数组复制给应用，数组中的用户指针不会因稍后 `DEL` 自动被清除。处理事件 **A** 的回调可能销毁 **B**，而本批稍后仍有指向 **B** 的事件；同一个 `Channel` 的读回调也可能关闭自身，分派器随后若继续调写回调，就发生释放后使用。

把逻辑死亡和物理回收分开

先标记对象不可再处理，注销关注关系并关闭或接管 `fd`；但让对象至少活到当前事件批次不再引用它。批次分派时检查死亡标记，跳过后续回调。可用延迟销毁列表、稳定句柄或代数编号实现，关键是明确“最后一个潜在访问者”何时退出。

`fd` 复用为什么又是一层问题

操作系统可把刚关闭的 `fd` 数值交给新连接。延迟回调只比较数值，可能把旧结果应用到新连接。用连接代数或唯一 `token` 区分每一代生命周期；对象仍存活也不等于它代表的业务实例仍有效。地址复用造成的 **ABA** 同样需要身份规则。

多线程场景还需要什么

延迟一个 `Reactor` 批次只保护该线程的旧事件，无法保护其他线程正持有的裸指针。跨线程访问需要共享所有权、引用/读侧保护，或把请求送回对象所属线程处理。关闭过程还要定义晚到结果是否丢弃、如何释放其附带资源，不能只在开头判断一次 `closed`。

推演与反例

同一批事件如何触发释放后使用

`epoll_wait` 返回 **A**、**B** 两个指针，先处理 **A** 时关闭并删除 **B**；随后循环读到 **B** 的指针，它已经悬空。常见办法是把 **B** 标成失效，从活动集合移走，但暂存一份所有权，直到本轮事件批次处理完再释放。**B** 的条目被访问时先看失效标记；若读回调还能删除自身，写回调前也要再次确认。

更广义的代数与取消规则

定时器、线程池和网络库队列可能比一个 `epoll` 批次活得更久，单轮延迟回收不够。可使用 `shared_ptr/weak_ptr`、连接代数或显式取消完成确认。取消请求只是提出意图，只有保证回调不再运行或运行时能安全识别失效，才允许释放被借用对象；否则“取消了”仍可能与已取出的回调竞争。

继续追问

回调里拿到 `shared_ptr` 就绝对安全了吗？

它保护对象不析构，但对象可能已逻辑关闭，`fd` 也可能无效；仍需检查状态、身份和回调是否允许继续执行。

用 `shared_ptr` 能否彻底避免所有错误？

它可以保活对象，但回调可能在逻辑关闭后继续修改新状态，也可能形成引用环。物理寿命和逻辑有效性都需要管理。

依据与核对范围

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

`epoll(7)` — 就绪集合、ET/LT、EAGAIN 与事件生命周期。

T033 原理问答

串口 `termios` 配置为什么重要？

串口配置同时决定物理帧格式和终端驱动怎样解释字节。二进制协议通常需要原始模式，关闭行编辑、回显及软件流控，并明确波特率、数据位、校验位和停止位。

原理与成立条件

原始模式要关闭哪些加工

终端默认可能有规范行输入、回显、特殊控制字符、换行转换和软件流控。这些适合人机终端，却可能把二进制协议中的字节当作控制字符。串口原始模式应明确关闭不需要的加工，并设置数据位、校验、停止位、波特率以及本地/接收标志。

VMIN 与 VTIME 的四种情况

非规范阻塞读取中，`VMIN=0/VTIME=0` 可立即返回；`VMIN>0/VTIME=0` 等最少字节；`VMIN=0/VTIME>0` 提供从调用开始的超时；两者都非零时，通常等待首字节后使用字节间计时。`VTIME` 单位为十分之一秒。它们描述 `read` 完成条件，不是协议帧超时或字节边界。

与 O_NONBLOCK 怎样配合

O_NONBLOCK 会改变暂时没有数据时的返回行为，不能把阻塞模式 VMIN/VTIME 四种规则不加条件地套到非阻塞读取。应按目标系统手册处理 EAGAIN，必要时也接受无数据返回 0 的终端语义。socket 的 read=0 表示有序关闭，不能机械推广到所有终端配置。

刷新和排空不是同一操作

tcflush 丢弃相应队列中的未处理数据，可能破坏协议；tcdrain 等待已排队输出发送结束，可能阻塞很久。修改串口配置时要明确是立即生效、输出排空后生效，还是连输入也丢弃。热切换波特率尤其需要双方协调，避免半帧跨越两种线路配置。

推演与反例

一个合法二进制字节为什么会被吃掉

终端行规程可能把换行转换、等待整行或把 XON/XOFF 控制字节用于软件流控。协议中的 0x11/0x13 如果被驱动解释成控制字符，上层看到的字节流就变了，CRC 会失败，却并非物理链路随机出错。调试应先确认 termios 与两端格式一致，再看解析器。

VMIN、VTIME 与非阻塞不能混讲

非规范模式中的 VMIN/VTIME 控制读等待条件，不同组合可表示等最少字节、首字节后超时等语义；O_NONBLOCK 又使调用在无进展时返回。跨系统细节需查实现，不能把“VMIN=帧长”当通用分帧方法。tcdrain 关心已提交输出的发送完成，而 write 返回更接近本地接纳，两者不能替代对端 ACK。

继续追问

波特率一样为什么仍全是乱码？

继续检查时钟误差、数据位/校验/停止位、电平标准与接地、引脚映射、终端加工及方向控制。波特率只是链路约定的一部分。

改波特率之前要处理哪些旧状态？

需要考虑输出中残余帧、输入半帧、解析器状态和对端切换协议。只改寄存器或 termios 可能把一条帧的前后两段按不同速率发送。

依据与核对范围

termios(3) — 串口原始模式、VMIN/VTIME 和 tcdrain。

HTTP 如何处理半包、连续请求和慢客户端？

HTTP/1.1 在 TCP 字节流上定义消息结构，解析器要跨多次读取保存状态，并按规范确定正文长度。一次 `recv` 可能半条请求，也可能包含多条，不能用读取次数划消息边界。

原理与成立条件

HTTP 解析必须增量进行

TCP 给出字节流，头部结束标记、字段甚至正文都可能拆在任意两次读取间；一次读取也可能包含多个请求。解析器保存已消费位置与阶段，先得到完整头部，再根据消息定界规则取得正文；处理一条请求后保留剩余字节，不能清空整块输入缓冲。

正文长度为什么是安全边界

HTTP/1.1 有 `Content-Length`、分块传输及无正文等不同规则。重复长度字段、冲突长度、非法数字与 `Transfer-Encoding` 组合必须依规范处理。前后两层解析器若对请求边界理解不同，可造成请求走私；不能靠“找到一个 `Content-Length` 就读这么多”称为完整 HTTP 实现。

输入和输出都可能失控

请求头应有长度和时间限制，正文有容量限制，连接数有上限；否则慢速持续发送可占用资源。发送响应时同样可能短写，输出缓冲应保存偏移并限制积压。只限制请求体不保护慢客户端造成的输出增长，也不保护无限 `keep-alive` 空闲连接。

生命周期与顺序如何保证

流水线请求可连续到达，响应顺序和每个请求的状态不能混乱；若业务异步处理，必须维持连接身份与响应排队。EOF 时还要区分完整请求已到与半个请求被截断，错误退出应及时注销事件和释放资源。成熟 HTTP 库通常比自行补齐全部边界更经济。

第三方 HTTP 库与业务边界

HTTP 库可承担增量解析、消息定界与异步收发，但路由、参数校验、授权和数据库查询仍是应用责任。网络完成回调只交接拥有独立存储的请求，业务完成后再向连接所属执行器投递响应。响应对象必须活到异步写完成；连接已关闭时，迟到结果只能取消或丢弃，不能凭复用后的 `fd` 数值寻找新连接。

推演与反例

正文长度为什么涉及安全

请求行与头部必须有大小上限；正文按有效的 `Content-Length` 或 `Transfer-Encoding` 等规则处理。冲突长度、非法重复头和不支持的分块编码不能各层自行猜测，否则前后代理对同一字节流的边界理解不一致，可造成请求走私。实现一个受限子集时应明确拒绝不支持的形式，不能悄悄忽略后继续复用连接。

慢客户端如何耗尽资源

客户端每隔很久发一个字节，若每次收到字节就把整个超时重新计满，可能永久占用连接和缓冲。应区分头部总期限、正文期限、空闲期限与输出上限，并限制连接数量。处理完一条请求要保留剩余字节供下一条解析，出错或关闭时再一致清理连接状态。

继续追问

recv 一次返回了完整 **JSON**，是否就能直接处理请求？

还不够。先确认 **HTTP** 头部和规定的正文长度已完整到达，并处理尾随请求；**JSON** 有效不证明 **HTTP** 消息边界正确。

返回 **200** 是否意味着后台动作完成？

取决于接口契约。若只完成排队，响应应清楚表达接纳或提供结果查询；**HTTP** 状态码本身不能替底层业务建立完成保证。

依据与核对范围

RFC 9112: **HTTP/1.1** — 消息定界、正文长度和连接管理；§6–§9。

Boost.Beast: 异步 **HTTP** 服务端示例 — **listener/session**、异步 **accept/read/write** 与响应对象寿命；这是库示例，不代表应用已实现所有示例功能。

T035 原理问答

事件循环如何同时保证就绪处理、公平性与退出？

事件循环的可扩展性不仅来自多路复用，还来自每次回调的工作量可控。任何长计算、同步写盘或无界排空，都可能推迟其他连接、定时器和退出请求。

原理与成立条件

公平性从哪几个维度考虑

事件循环要限制单个连接、单类事件和单轮批处理的占用。大量小消息、长解析和持续可写都可能让其他事件饥饿。预算可以按字节、条数或执行时间，但达到预算后应保存足够状态并主动安排后续处理，不能因追求公平把工作遗忘。

队头阻塞为什么难被平均值发现

若一次同步配置重载耗时 **300 ms**，串口、超时和停止信号都可能等它结束。平时回调很快，平均 **CPU** 仍很低，**p99** 或最大响应却显著变坏。应记录每类回调的占用分布、两次循环之间最大间隔与队列等待，而非只看每秒处理数量。

正确停止是一个协议

先禁止新任务进入，再让生产者停止，叫醒阻塞消费者，按约定排空或取消队列，等待线程退出，最后销毁依赖资源。若先 `join` 一个仍在等永远不会到来的通知的线程，退出本身就死锁。若先销毁被回调捕获的对象，停止阶段会出现正常运行中没有的悬空访问。

停止也要有期限和结果

关键持久工作可能需要排空，过期遥测可能允许丢弃；必须按业务选择。有限停止期限到达后要有明确降级与记录，强制退出不能伪装成全部提交成功。控制平面停止命令应避免排在无界业务洪水后面，否则系统可能连退出都无法及时响应。

推演与反例

用处理预算建立公平性

每轮限制读字节数、处理任务数或实际执行时间，把未完成工作记在本地待处理集合，然后轮转处理。预算不能破坏 ET 的继续处理责任；后台线程也要有有界队列，否则只是把拥塞移到看不见的地方。稳定运行要求长期消费能力覆盖到达率，短时预算解决公平性，容量和背压解决过载。

停止标志为什么需要唤醒

一个线程把 `running=false`，主循环却正在无限期 `epoll_wait`；若没有新事件，它可能永远不再检查标志。正确退出通常既改变持久停止状态，也通过 `eventfd`、信号等唤醒等待点，然后停止接纳新任务、处理已有任务并退出。修改原子标志只保证可见性，不会凭空制造调度事件。

继续追问

只设置 `atomic<bool> stop=true` 为什么线程还不退？

标志只改变状态，不会自动唤醒阻塞在队列、`epoll` 或 I/O 的线程；还要触发它所等待的机制，并让循环在醒来后检查退出条件。

回调中写一条日志也可能阻塞吗？

可能。即使平时很快，文件系统、锁或日志队列满时都可能改变行为。需要了解日志接口的最坏路径和满队列策略，而不是只看函数名称含 `async`。

依据与核对范围

`epoll(7)` — 就绪集合、ET/LT、EAGAIN 与事件生命周期。

`eventfd(2)` — 计数器、非阻塞读写与合并通知。

帧协议、可靠传输与幂等

T036 原理问答

如何设计一条能抗分包和噪声的串口帧？

字节流没有天然帧边界。接收器要靠同步标志、长度、类型和完整性校验逐步识别消息，并在任意分包、连续多帧和损坏输入后恢复到可继续解析的状态。

原理与成立条件

字节流如何恢复消息边界

串口和 TCP 都可能把应用消息拆开或合并。常见定界方法有固定长度、分隔符加转义、长度前缀和同步头加长度。分隔符需要处理负载中相同字节；长度字段一旦损坏可能让解析器错误等待很久，因此常配合法范围、校验与重新同步规则。一次中断或 read 的边界不是消息边界。

增量解析器怎样工作

用状态保存当前在找帧头、收长度、收正文还是收校验；每输入一字节更新已收计数，只在足够数据到达后读字段。帧可以跨任意输入块，状态不能在每次回调结束时清空。容量上限应在接受长度时立即检查，不能先复制数组再判断长度非法。

噪声与同步头重叠

发现非法长度或 CRC 后需要重新寻找候选帧头；例如 AA AA 55 中第二个 AA 仍可能是新帧起点，简单清空后丢弃当前字节可能错过它。是否回扫已收缓存取决于协议设计和内存预算；重要的是保证持续噪声下时间、空间有界，并能最终识别后续合法帧。

校验通过后仍需业务验证

类型对应的最小/精确长度、字段取值、版本和状态都要继续检查。CRC 只过滤一类字节错误；它不能防止合法校验但语义非法的输入。测试应把每个合法帧从所有可能位置切分，再加入连帧、坏长度、坏校验和噪声前缀，验证消费和恢复行为。

推演与反例

一个增量解析器需要记住什么

设帧为同步头、长度、正文、CRC。它必须记住目前在找头、收长度、收正文还是收校验，以及已收字节数。一次输入只有半个头时不能丢掉合法前缀；一次输入多帧时每完成一帧后还要继续处理余下字节。若正文也可能包含同步头，正常收正文阶段不能见头就重启，必须由当前长度和校验决定边界。

恢复策略怎样避免越界和锁死

长度必须在写入正文缓冲前检查。校验失败后可重新搜索候选同步位置，但要考虑重叠前缀，例如 AA AA 55 中后一个 AA 仍可能是新头。错误长度可能吞掉后续帧，因此还应有最大帧长、必要的组帧超时和重同步策略。解析器接受外部字节时任何路径都必须有界，不能通过无限等待更多字节掩盖非法状态。

继续追问

为什么 IDLE 后就把当前缓冲当整帧会失败？

发送方可在一帧中停顿，也可连续发多帧而没有足够空闲；物理空闲只表示接收进度，帧界仍应由协议决定。

用 IDLE 或 read 返回短数据来划帧可以吗？

只有协议明确把静默间隔作为定界条件且满足时序约束时才行。一般自定义长度协议应把这些当传输分块，最终边界由协议自身决定。

依据与核对范围

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

T037 原理问答

CRC16 与累加和有什么区别，能保证不出错吗？

累加和把字节作加法，CRC 把消息看作二进制多项式做模二除法。CRC 对特定差错模式有可证明的检测能力，但仍有漏检模式，也不提供防恶意篡改的认证。

原理与成立条件

为什么比简单累加更适合突发错误

累加和可能漏掉相互抵消的字节变化，甚至交换字节后不变。适当生成多项式可保证检测某些长度内突发及特定比特错误模式； r 位 CRC 对长度不超过 r 位的突发有相应保证，但超过长度并非全部可检。随机残余错误率常近似 2^{-r} ，不能当作对任意输入或攻击者的保证。

反射多项式和线上端序

0x8005 与反射算法使用的 0xA001 可描述同一生成多项式的不同位处理方向；仅给出多项式仍不足以复现校验。CRC16/MODBUS 常用初值 0xFFFF、反射处理、最终不异或。结果哪一个字节先在线上传输又是独立协议约定，不由 CPU 字节序自动决定。

怎样验证实现

用公开 **check** 向量及自建边界向量分别核对参数、覆盖范围和输出顺序。例如 ASCII “123456789” 的 CRC16/MODBUS 数值为 0x4B37，低字节先发时是 37 4B。CRC 不带秘密，攻击者能修改负载并重算；需要防篡改与身份鉴别时应考虑 MAC 或认证加密。

推演与反例

为什么调换字节可能骗过累加和

两字节 01、02 与 02、01 的加法和相同，但字节位置在 CRC 的多项式中对应不同次数，因此通常得到不同余数。生成多项式决定哪些错误模式是其倍数而无法检出；合适的 r 位 CRC 可检测长度不超过 r 的突发错误，但其他性质还要看多项式和消息长度。随机错误下常说的约 2^{-r} 漏检概率依赖分布假设，不是对所有输入的绝对保证。

一套 CRC 参数必须完整给出

除了宽度和多项式，还要约定初值、输入/输出反射、最终异或、参与计算的字节及结果线上顺序。0x8005 与反射实现的 0xA001 是位方向相关的表示；看到同名 CRC16 仍不能确定双方兼容。用固定 **check** 向量确认基础算法，再用真实帧验证覆盖范围，才能把算法错误和协议拼帧错误区分开。

继续追问

帧头也必须算 CRC 吗？

由协议约定，不能凭经验猜。同步头常用于寻找边界，长度和正文通常需要保护；接收端必须与发送端对完全相同的字节序列计算。

攻击者改了数据后能重算 CRC 吗？

能。CRC 没有秘密密钥，不能证明来源或阻止重放。需要身份鉴别和防篡改时使用合适的 MAC、认证加密及新鲜度机制。

依据与核对范围

Modbus Organization: Serial Line V1.02 — 串行 CRC 附录；本项目只使用 CRC 变体，不是 Modbus 协议。

T038 原理问答

停止等待、GBN 与 SR 怎样工作，窗口上限如何推导？

停等用最少状态换取较低链路利用率；GBN 批量在途但通常从缺口起重传；SR 分别确认并缓存乱序数据。窗口大小受带宽时延积、缓存以及序号歧义共同限制，不能只追求更大。

原理与成立条件

窗口为什么影响吞吐

若发送一帧耗时 T_{tx} ，往返等待远大于 T_{tx} ，窗口 W 太小就不能持续填满链路。可按带宽时延积估计在途需求，再考虑接收 RAM 与执行能力。窗口大并不自动更快：链路已满、业务执行串行或接收缓存不足时，增大窗口只会增加积压和结果等待。

SR 半序号空间规则的由来

经典循环序号 SR 中，接收方必须区分上一轮延迟的旧帧和新一轮同序号帧。若发送/接收窗口可重叠太多，就可能把重传旧帧当新帧；常见对称窗口要求 $W \leq 2^{m-1}$ 。这是特定模型下的必要约束，仍需最大报文寿命等假设，不能凭此消除任意长延迟造成的歧义。

接收顺序与执行顺序可以分开

SR 可提前接纳后续帧而暂不交付业务，等缺口补齐再按序交付。这样改善线路利用率，但永久缺口仍会阻塞后续执行。设计必须规定重试耗尽后如何跳过、重置或中止；默默丢掉缺口继续执行，可能违反控制命令的顺序语义。

推演与反例

利用率可以怎样近似计算

设一帧发送耗时 T ，最后一位发完到确认返回还需 D ，忽略处理和 ACK 发送开销，停等利用率约为 $T/(T+D)$ 。允许 W 帧在途后，理想利用率上界可近似写成 $\min(1, W \times T/(T+D))$ 。这里 D 的定义已经排除本帧发送时间，避免把 RTT 和 T 重复相加；有丢包、半双工、有限接收处理能力时还会更低。

半序号空间限制来自“旧帧像新帧”

对称 SR 窗口为 W 、序号空间为 M ，接收方交付一整窗后，新窗口不能与发送方仍可能重传的旧窗口发生无法区分的重叠，因此典型约束是 $M \geq 2W$ 。这个结论依赖循环序号模型及报文寿命等前提；若协议在序号用完前换会话，则还需证明新会话与旧迟到报文能被区分，不能仅引用公式就结束设计。

反例：M=8，窗口 W=5 为什么有歧义

1. 发送方发送 0, 1, 2, 3, 4；接收方全部收到并交付。
2. 对这些帧的 ACK 全部丢失，发送方仍可能重发旧 0。
3. 接收方新窗口为 5, 6, 7, 0, 1，包含序号 0。
4. 若没有额外代际信息，旧 0 会与新一轮 0 混淆。

相同大小 SR 收发窗口的一般条件： $2W \leq M$ 。

这是循环序号空间的经典约束。另加不重复会话号、限制序号在会话内不回绕等规则，会改变接收合法性判定，但必须明确额外假设。

继续追问

使用 8 位序号为什么不直接开 128 帧窗口？

理论最大值不是资源建议。要结合帧大小、接收缓存、结果保留、链路 RTT 和业务时延；还要遵守具体协议的会话与寿命规则。

SR 为什么还会产生队头阻塞？

链路可先接收后续帧，但要求按序交付的业务仍要等前面的缺口。SR 减少无效重传，不自动取消业务顺序约束。

依据与核对范围

RFC 1982: [Serial Number Arithmetic](#) — 通用模序号比较背景；项目 SR 具体策略以源码为准。

MIT 6.02: [可靠传输与 ARQ 课程原稿](#) — 第 20 章 Stop-and-Wait、Sliding Window 与 ACK；项目窗口与缓存寿命由本地实现给出。

MIT 16.36: [GBN 与 Selective Repeat](#) — Lecture 18, SRP Rules: $M \geq 2W$ ；区分一般循环序号与具体应用的会话内不复用策略。

T039 原理问答

传输确认、按序交付与业务完成为什么不是同一状态？

接收确认表示对方已承担保存责任，按序交付表示数据可以进入业务，执行结果表示业务已经完成。这三个事件可能相隔很久，必须用不同状态与资源计数描述。

原理与成立条件

为什么需要两种在途计数

接收窗口保护传输顺序与接收缓冲，业务在途表保护尚无最终结果的请求。一个命令已被接收但仍在执行，可能不再需要接收重传，却仍占业务资源。只维护一个计数，会在 ACK 后过早删除请求，或把执行等待误当丢包不断重发。

确认可以改变哪些状态

接收 ACK 可以标记已接纳、停止接收阶段重传，但不能产生业务成功结果。结果消息通常隐含命令已到达，可在接收 ACK 丢失时补足接收认知；若结果先到，状态机应允许直接完成。重复结果应补确认，但不能再次通知上层执行完成。

发送许可要满足多个约束

是否能发下一条，可能同时取决于最早未确认序号限定的窗口范围、未完成业务数量、接收端信用和本地输出容量。即使后续项已确认，前面的缺口未补上，窗口也未必能前移。简单用“ACK 数量减一”替代范围判断，会在乱序确认时突破接收窗口。

结果缓存与接纳责任

节点执行前应预留足以记录命令与结果的资源，否则动作已经发生却无法保存结果，将难以响应重试。接收确认时承诺的资源边界应包含这个需求。窗口小于缓存容量，可以给已完成未确认结果留空间，但仍要分析最坏结果滞留与新命令竞争。

推演与反例

ACK 乱序时窗口为什么不能随便前移

发送 0、1、2，先收到 1 和 2 的接收确认，0 仍未确认。虽然已确认数量是 2，窗口左边界仍被 0 挡住；若简单按 ACK 数减在途数就继续无限发送，会超出接收端可接受范围。另一方面，0 已确认但执行仍很慢时，它可不再需要链路重传，却仍占业务结果表中的一个位置。

结果先到的合法顺序

接收 ACK 可能丢失，而结果消息已经返回。只要结果身份、类型和长度有效，结果本身可证明该请求至少被接收并执行，状态机应允许直接进入完成。之后迟到的接收 ACK 不能把状态倒退，重复结果可以补发结果确认，但不能再次调用完成回调。这要求把状态转移设计成单向、幂等的规则。

继续追问

接收 ACK 都齐了，为什么还不能释放全部业务对象？

结果可能还没到，调用者回调、结果期限和去重身份仍需保留。传输结束不等于请求生命周期结束。

哪一刻可以复用请求对象内存？

只有相关输出、定时器、回调和重试引用都已结束或被安全隔离后。业务完成是必要条件之一，不代表所有异步借用立即消失。

依据与核对范围

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

MIT 6.02: 可靠传输与 ARQ 课程原稿 — 第 20 章 Stop-and-Wait、Sliding Window 与 ACK；项目窗口与缓存寿命由本地实现给出。

会话号与序号为什么缺一不可？

序号回答“本轮哪一条”，会话号回答“属于哪一轮”。组合键防止上一轮迟到的请求或确认命中新一轮相同序号，但仍需明确会话生成、接受规则与重启后的保存范围。

原理与成立条件

序号解决重复，会话解决哪一代

短序号用来关联当前协议实例中的消息，会话或 `epoch` 区分重连/重建后的实例。若只使用 `seq=3`，延迟到达的旧确认可能命中新发送的 `seq=3`。把 `session` 与 `seq` 一起作为键，可以隔离旧消息，但前提是会话标识不会在相关消息寿命内再次碰撞或被错误接受。

随机与单调标识的区别

随机足够宽标识依概率降低碰撞，不天然表达新旧顺序；单调标识允许接收方拒绝旧代，但需要解决分配与重启持久化。用当前时间生成要考虑时钟回拨、分辨率和多个发送者。若没有认证，攻击者还可能伪造更大代数，所以新旧判定与身份鉴别不能混为一谈。

序号耗尽要有明确流程

可以限制最大在途及报文寿命后安全回绕，也可在回绕前排空并换会话。换会话通常先停止旧代提交、处理旧在途，再通过握手建立新代。只有本地变量加一而对端仍接受旧代，会留下过渡期歧义；迟到的旧回调也必须带会话校验。

重启是最容易漏掉的边界

接收端只在 `RAM` 保存会话高水位，掉电后就忘记过去；随机启动标识或持久计数可改善区分，但仍需对双方恢复做完整设计。业务 `ID` 的寿命通常长于一次链路 `session`，不能把链路键当作跨所有重连的永久去重键。

推演与反例

随机标识与有序标识不能直接互换

随机足够宽的标识主要降低碰撞概率，不天然支持数值越大越新；单调递增会话可让接收方维护高水位，但要处理持久化、分配冲突和重启回退。基于墙上时间生成还要考虑时钟回拨。若接收方用“大于旧值”决定切换，发送方也必须遵守同一全序契约，而不是随便生成一个随机数。

旧回调必须带哪一代身份

旧会话的写完成、定时器或网络结果可能在新会话建立后才执行。若回调只带 `seq`，它可能修改同号新请求；因此校验至少包含会话与请求身份，必要时再带发送尝试代数。会话重建不仅是清空哈希表，还要让外部队列中残留操作无法污染新状态。

继续追问

64 位 session 是否意味着永不重复执行？

不意味着。标识宽度只缓解冲突，是否重复还取决于接收策略、保存期限、掉电恢复，以及业务效果与记录是否一致提交。

64 位会话为什么仍不能保证跨掉电不重复？

如果接收方丢失已执行记录，同一个业务可能以新会话身份再次到达。链路会话区分与长期业务去重是不同层次，后者需要稳定业务 ID 和相应持久状态。

依据与核对范围

RFC 1982: Serial Number Arithmetic — 通用模序号比较背景；项目 SR 具体策略以源码为准。

T041 原理问答

怎样设计 RTO、总截止时间与有限重试？

RTO 控制一轮发送后多久重试，总截止时间限制请求整个生命期，重试预算限制额外尝试次数。三者分别解决暂时丢包、无限等待和资源占用，起点不能混用。

原理与成立条件

一轮重试和总体截止时间

单轮 RTO 判断“这次发送后等多久再尝试”，总体 **deadline** 判断“这个请求还能活多久”。每次重试可以更新本轮起点，却不能无条件延长总寿命。还要区分发送前排队、等待接收、等待执行结果，否则排队拥塞会触发无效重发，慢业务也会被误认作线路丢包。

RTO 应覆盖哪些延迟

一次往返包含线路发送、驱动缓冲、对端处理和返回调度。设置过短会产生重复与拥塞，过长则恢复慢；可依据有界模型或 RTT 估计选择。重传报文的 ACK 很难区分对应哪一次尝试，直接用于 RTT 估计会有歧义。TCP 的估计和退避机制可作背景，但自定义协议不必照搬全部机制。

重试次数为什么要明确初次是否计入

“重试 3 次”常指初次发送之外再发 3 次，总尝试 4 次。失败日志、计时和测试必须统一口径；输出队列一直未真正发送时，也不应凭空消耗真实发包轮次。对非幂等业务，还需明确哪些阶段允许重传同一请求，哪些阶段只能查询结果。

超时处理不能删除一切痕迹

晚到结果可能证明业务已发生，旧发送回调也可能仍排在事件队列里。状态机需要完成只回调一次、旧轮次不能改新状态、部分发送缓冲继续收尾等规则。超时是本地决策，不能倒推远端动作被撤销；上层需要知道失败与结局不确定的区别。

推演与反例

RTT 估计为何不能直接取最近一次测量

正常 RTT 会抖动，通常要结合平滑值和变化幅度留出裕量；TCP 的经典形式是 SRTT 加上与 RTTVAR 相关的裕量，并在超时后退避。重传后到达的 ACK 可能对应第一次发送，也可能对应重发，直接从最近发送时刻算样本会低估 RTT，因此需要处理重传歧义。自定义协议可采用固定有界 RTO，但也应解释选值覆盖哪些延迟。

排队与发送期限要分开

假设请求在本地队列等了 300 ms、线上往返正常只需 50 ms。如果从入队立即启动 100 ms 单轮重传计时，就可能尚未发出便排入多份重试。可把排队纳入总期限，而在约定的本地发送完成点启动单轮计时。该完成点如果只是内核接纳，后续驱动排队也仍要计入 RTO 预算。

继续追问

把 RTO 从 500 ms 改到 50 ms，响应就一定更快吗？

正常响应不会因此提前，反而可能在合法处理延迟内重复发送。应先测延迟组成与丢失原因，再选择能覆盖正常波动的阈值。

超时后能向上层报告“没有执行”吗？

通常不能。请求可能已执行而结果丢失。应区分发送前拒绝、明确业务失败和结果未知，重试非幂等操作前先解决这种不确定性。

依据与核对范围

`timerfd_create(2)` — 时钟选择、到期计数与 fd 通知。

RFC 6298: TCP RTO 计算 — §2 RTT/波动估计、§3 重传样本、§5 定时器管理；作为重试基础，自定义 SR 参数以项目为准。

有序协议缺口无法补齐时，怎样恢复会话？

有序协议遇到永久缺口时，必须让双方就“跳过还是重建”达成一致。发送方只删本地超时项，不会改变接收方等待的序号，也不能恢复后续业务的顺序执行。

原理与成立条件

有序交付为何会被一个缺口卡住

接收端已缓存 6、7，但执行基序号是缺失的 5。若命令有依赖，不能跳过 5 直接执行后面的动作。重传可以补临时缺口，有限重试耗尽后必须有协议规定的恢复动作，否则队列保持有数据却永远不能前进，形成协议级队头阻塞。

跳过与重建的取舍

显式跳过要求双方一致记录该序号已放弃，并说明后续命令仍合法；重建会话则丢弃旧代推进关系，让双方在新起点同步。重建更容易隔离混乱状态，但旧会话中有些操作可能已执行，只是结果未到，因此不能把所有旧命令当成“未执行”自动复制到新代。

恢复过程要避免双重控制

暂停新提交，给旧请求一个明确终态或不确定结局，处理尚未完成的线路输出，建立更高代数的会话，再开放入口。所有旧回调、结果和确认都通过代数校验隔离。若新 OPEN 还未确认就发送新命令，可能再次在双方会话认知不同的状态下工作。

业务补偿与传输恢复分工

传输恢复的目标是让链路继续接纳新请求，不负责自动撤销已发生的物理效果。业务层若需要恢复目标状态，可以读取现状后发送绝对目标，或使用持久业务 ID 重试。对于“再走一步”这类动作，不做结局核实的自动重放可能扩大故障。

推演与反例

为什么整体重建有时更简单

接收端等 5，却缓存了 6、7。若要跳过 5，需一个本身可靠、可去重的跳过协议，并说明 6、7 在缺少 5 时是否仍有意义；重建会话可以直接结束旧顺序关系，让双方从新起点开始。但旧 5 可能已经执行、仅结果丢失，因此重建只是恢复传输能力，不是把现实世界回滚到未执行。

恢复期间最容易漏掉的发送残留

本地可能已经写出半条旧帧，或者旧重试还排在发送队列。切换时要区分尚未开始的旧项与已经部分写出的帧：前者可因身份失效而取消，后者需按流完整性策略收尾或关闭流。新会话握手成功后再放行新命令，旧回调继续按代数过滤，不能仅清一个 active 标志。

继续追问

为何不把丢失序号对应请求删除就继续？

删除本地表项不会改变接收端等待的执行基序号；双方必须通过明确协议事件对跳过或新起点达成一致。

能否在新会话自动重发所有失败命令？

只有业务明确允许这种语义才行。绝对设置值可相对容易重建目标状态；增量动作需要稳定业务去重或查询实际结果，不能把链路重连当成重放授权。

依据与核对范围

RFC 1982: [Serial Number Arithmetic](#) — 通用模序号比较背景；项目 SR 具体策略以源码为准。

MIT 6.02: [可靠传输与 ARQ 课程原稿](#) — 第 20 章 Stop-and-Wait、Sliding Window 与 ACK；项目窗口与缓存寿命由本地实现给出。

T043 原理问答

去重缓存满了、过期了或结果 ACK 丢了会怎样？

去重表不是普通缓存，它记录接收方已经承担的执行与结果责任。容量、TTL 起点、重复是否续期、确认后释放以及过期后的拒绝规则共同决定可靠性范围。

原理与成立条件

结果缓存保存的是协议责任

接收方执行后保存请求键、参数身份和结果，使同一请求重传时返回原结果，而不是再执行一次。未执行的待处理项也占资源；因此缓存不能只按“已完成数量”估算。结果发出去并不等于对端收到，只有结果确认或明确到期策略才能结束保存责任。

TTL 的起点影响保证

从接收起计时可能让长业务刚执行完就过期；从完成起计时则给回传保留固定时间。重复请求是否续期也要明示：无限续期可能让恶意或故障发送者永久占槽，不续期则必须保证重试策略与有效窗口协调。TTL 应与最大网络延迟、总请求期限和容量一起分析。

满缓存时不能先承诺后丢弃

若无槽位仍回复“已接收”，发送端可能停止重传，而接收端无法保存命令或最终结果。合理选择是拒绝/不确认、实施信用流控，或提供明确忙响应。对已经接纳的非幂等请求，随意淘汰记录会重新打开重复执行窗口，不能只用普通 LRU 思路处理。

缓存过期不一定允许重新执行

若协议还记得该序号已在执行基序号左侧，可拒绝再次执行业务，但可能再也无法重放结果。于是“防重复执行”和“始终可返回原结果”有不同寿命。要向调用者解释这一区别，否则有限缓存会被误称为永久 **exactly-once** 服务。

推演与反例

满缓存时为什么不能先 **ACK**

若收到非幂等命令后先确认，再发现没有空间保存请求与结果，发送方可能停止重试，接收方却无法履约。应先预留记录，再接纳并确认；没有资源时不接纳、返回忙或用信用流控。随意 **LRU** 淘汰已接纳但未结束的项，会把重复执行风险重新引入系统。

“不再执行”与“还能回原结果”寿命不同

一个协议可能通过单调执行基序号记住 **0..20** 已经执行，而只保存最近若干条具体结果。**21** 之前的旧请求不会重做，但缓存被释放后也无法回答旧 **3** 的结果。相反，只保留短 **TTL** 哈希表而没有序号历史，过期后可能把同一请求当新请求执行。必须说明使用的是哪一套状态，而不是笼统说“做了去重”。

继续追问

RESULT_ACK 丢失会怎样？

接收方继续保存或重发结果，发送方对重复合法结果再次 **ACK**；应避免重复业务回调，缓存最终还需有限释放规则。

TTL 比发送方总期限短就一定错误吗？

需要连同完成时刻、结果重传、窗口推进和过期拒绝规则一起判断。短 **TTL** 会缩短结果可恢复期，但不必然破坏“旧请求不重执行”；保证的是哪一种性质必须分别写出。

依据与核对范围

RFC 1982: Serial Number Arithmetic — 通用模序号比较背景；项目 **SR** 具体策略以源码为准。

MIT 6.02: 可靠传输与 ARQ 课程原稿 — 第 20 章 **Stop-and-Wait**、**Sliding Window** 与 **ACK**；项目窗口与缓存寿命由本地实现给出。

T044 原理问答

幂等、去重与恰好一次有什么区别？

幂等是重复操作的效果属性，去重是识别重复请求的机制，恰好一次则涉及定义好的观察范围与失败模型。稳定请求 **ID**、结果记录和业务效果之间的原子关系决定保证能做到哪一层。

原理与成立条件

幂等是效果属性，去重是识别机制

一个操作重复执行多次与执行一次效果相同，称为幂等，例如设置目标值；去重则识别同一请求已处理，从而跳过或返回原结果。即使设置值本身幂等，重复日志、通知或继电器脉冲等附带效果也未必幂等；必须说明观察哪些业务效果。

请求键必须稳定且防错配

重试保留同一个业务 ID，新业务使用新 ID；接收方将 ID 与规范化参数或摘要关联。同 ID 不同参数应拒绝，而不是返回旧结果或覆盖记录。连接序号通常只在一次会话内稳定，跨重连由发送者重新生成的序号不能承担长期业务去重。

数据库事务能解决的部分

业务效果若完全在同一数据库，可在事务中以唯一键插入请求记录、修改业务数据并保存结果；重复命中后读取已记录结果。并发请求还需让“检查与写入”由唯一约束和事务保证，而不是先查不存在再无保护执行。外部物理动作不能自动纳入数据库原子提交。

物理动作怎样降低不确定性

优先使用设置绝对状态，结合动作 ID、设备侧持久状态和恢复查询；必要时做可补偿流程。必须承认断电落在物理效果与持久记录之间的风险，某些业务需要硬件反馈或操作规程处理。不能通过加一个布尔 **done** 就证明任意物理动作恰好一次。

推演与反例

数据库内的效果为什么较容易合并

如果业务只是更新同一数据库，可在事务中插入唯一请求键、修改业务数据并保存结果。并发重复请求由唯一约束仲裁，提交成功后可返回已保存结果；不能先在事务外查重再执行，因为两个线程可能都查到不存在。失败后要看事务是否真正提交，提交结果不确定时仍需按同一键查询，而非生成新键盲目重做。

物理动作为什么留下空隙

先写“已完成”再让电机转，断电可能留下记录成功但动作未发生；先转电机再写记录，断电后又可能无法知道已转过。普通数据库事务不能自动覆盖电机。可改成设置绝对目标、记录动作 ID、读取位置反馈、设计恢复流程，但必须给出硬件可观察性与持久化假设，不能靠一个 **done** 位宣称任意动作恰好一次。

继续追问

HTTP PUT 被称为幂等，是否每次响应都必须相同？

幂等主要约束重复请求的预期服务端效果，不要求状态码、时间戳或所有响应字节完全相同；具体接口仍需定义副作用。

设置同一个值就必然幂等吗？

只对选定业务效果可能幂等。若每次设置还触发计费、通知、脉冲或重置定时器，重复调用的总效果可能不同，应把这些副作用纳入定义。

依据与核对范围

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

RFC 9110: HTTP 幂等语义 — §9.2.2 Idempotent Methods，约束预期效果，并非要求所有响应内容相同。

T045 原理问答

如何验证协议状态机，而不只测试正常收发？

协议验证要检查状态与不变量，正常收发只是其中一条路径。应控制消息丢失、重复、乱序、迟到和时间推进，证明每次输入后资源与完成语义仍正确。

原理与成立条件

状态机测试先写不变量

先规定永远应成立的性质：不会执行窗口外请求、一次请求最多完成回调一次、缓存满不伪确认、旧代不能改新代。再枚举状态和事件，而不是只覆盖函数行数。相同输入事件序列应能复现结果，可控时钟比真实 `sleep` 更适合边界与回绕测试。

单独丢包不足以覆盖故障

分别丢命令、接收 `ACK`、结果、结果 `ACK`，行为不同；再加入重复、乱序、发送短写、延迟回调和新旧会话交错。错误往往来自组合，例如超时刚处理完收到旧结果，或重载后旧发送回调抵达。测试需要覆盖事件顺序而不只是数据值。

用模型检查迁移是否合法

可以写较简单的参考状态模型，输入生成的事件序列，对比接纳、交付顺序和终态；记录随机种子以复现。模糊测试擅长字节解析健壮性，却不自动理解协议语义，需要额外的不变量判定。越界无崩溃也不等于消息没有被错误执行。

证据层级要分别报告

纯 C 核心测试验证协议计算，伪终端可覆盖网关 `read/write` 集成，节点主机替身无法精确模拟中断和 DMA，真实板卡再观察波形、覆盖与复位。每层失败都能定位更小范围，但上层通过不能代证下层时序。

推演与反例

怎样从不变量推导测试

先写出“同一请求最多执行业务一次”“窗口不越界”“完成回调至多一次”“总期限不被重试延长”等规则，再生成破坏条件的序列：结果先于接收 ACK、旧会话 ACK 到新会话、缓存刚过期、发送半帧时取消。测试不仅看返回值，还统计执行次数、保留槽数、发送内容与最终状态。

虚拟时间为什么比等待真实超时好

把单调时钟作为依赖注入，可以直接检查 **deadline** 前一单位、等于和后一单位，不必每次睡几秒，也避免调度抖动让测试偶然通过。解析层再用任意切分点与噪声输入测试，与协议层的状态测试分开；最后用真实串口或 PTY 验证接口衔接。各层证据解决不同问题，不能彼此替代。

继续追问

最有价值的边界用例是什么？

选择最容易跨越生命周期的事件：最后一轮超时与合法结果同时到达、序号耗尽换会话、缓存到期后一份旧命令再次到达，并明确预期只有一个业务终态。

随机压测很久没错，是否比边界测试更充分？

随机测试能发现未想到的组合，但不保证命中关键边界。应保存种子与失败序列，结合针对不变量的确定性测试和真实链路验证。

依据与核对范围

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

TCP、MQTT、HTTP 与网络边界

T046 原理问答

TCP 为什么还需要应用层分帧？

TCP 可靠传输的是有序字节流，不保存应用每次 `send` 的消息边界。接收端必须根据应用协议的长度、分隔符或其他明确规则恢复消息，即使每次发送都恰好是一条完整命令。

原理与成立条件

TCP 保证的单位是字节

TCP 为连接提供有序字节流，通过序号、确认与重传恢复部分传输丢失。一次 `send` 的边界不会成为对端 `recv` 的边界：协议分段、路径 MTU、缓冲与读取大小都会改变交付块。因此所谓“粘包/拆包”通常是应用没有正确实现消息定界，而不是 TCP 把协议消息弄坏了。

应用层如何分帧

固定长度简单但浪费且不灵活；长度前缀需校验最大长度及整数溢出；分隔符需转义；文本协议也必须严格定义正文长度。读取时把新字节追加到受限缓冲，循环解析已完整的消息，保留残片，不能只处理一条就丢掉尾部，也不能为恶意长度直接分配任意大小内存。

TCP 的“可靠”不覆盖业务提交

`send` 返回通常表示本地内核接纳，TCP ACK 表示对端 TCP 接收相应字节，并不意味着对端进程已解析或数据库提交。连接断开时，本地可能不知道最后一个请求是否已执行。因此请求响应协议仍需业务 ID、结果与恢复规则；换成 TCP 不会消除分布式结局不确定。

EOF 和半关闭

`recv` 返回 0 对流 `socket` 表示对端发送方向有序结束，已缓冲数据通常先被读出；它不直接表示本端发送方向也不能再发送。`shutdown` 可关闭某个方向，`close` 释放本地描述符引用。收到 EOF 时，若解析器仍持有半条消息，应按截断错误处理，而非把残片当成功请求。

推演与反例

合并与拆分为什么都是正常现象

应用连续 `send` 两条 20 字节消息，接收方可能一次读到 40 字节，也可能先读 7 字节再读 33 字节；网卡、TCP 分段、接收缓冲和线程调度都可影响读取分块。这里通常称为“粘包/半包”的现象并不是 TCP 把数据顺序弄错，而是应用错误地把接口调用边界当成消息边界。

定界方法各有什么代价

固定长度解析简单但可能浪费空间；长度前缀能承载任意二进制数据，但必须验证上限并处理头部不完整；分隔符需要转义或保证正文不包含分隔符；关闭连接定界限制了复用和双向交互。无论哪种方法，接收缓冲都要保留未完成消息及多余后缀，不能每次 `read` 后清空。

继续追问

TCP_NODELAY 能解决粘包吗？

不能。它影响 Nagle 小包合并策略与延迟，不提供消息边界。无论开关如何，接收方都必须正确按应用协议解析字节流。

开 TCP_NODELAY 能解决粘包吗？

不能。它影响 Nagle 算法相关的小包发送行为，不改变 TCP 字节流语义。应用仍必须自行分帧。

依据与核对范围

RFC 9293: TCP — 握手、序号、字节流和关闭；§3.4–3.8。

RFC 9112: HTTP/1.1 — 消息定界、正文长度和连接管理；§6–§9。

T047 原理问答

TCP 三次握手、四次挥手和 TIME_WAIT 如何联系实际故障？

握手用于建立双方的序号与连接状态，关闭则分别结束两个方向的数据发送。TIME_WAIT 让主动关闭方在一段时间内处理迟到报文及最终 ACK 的重传问题，不是单纯的资源浪费。

原理与成立条件

三次握手交换了什么

客户端 SYN 带初始序号 `x`，服务端 SYN+ACK 确认 `x+1` 并给出 `y`，客户端 ACK 确认 `y+1`。SYN 各消耗一个序号。双方建立本次连接的序号空间并确认彼此发送接收路径；第三次确认还帮助避免服务端把历史重复 SYN 直接当成已建立的新连接。不能只背“三次是最低次数”而不解释状态与旧报文。

为什么关闭常见四次却不固定四包

TCP 双向独立，每一方发送 FIN 表示本方向不再有新数据，另一方可以先 ACK，等剩余数据发完再 FIN。ACK 与 FIN 可能合并，也有同时关闭，所以实际报文数不必固定四个。FIN 同样占一个序号；收到 FIN 后的 CLOSE_WAIT 表示本地应用仍需完成关闭。

TIME_WAIT 的作用与归属

通常发送最后 ACK、主动关闭一侧进入 TIME_WAIT，保留足够时间让对方重传 FIN 时可再次确认，并降低旧连接延迟报文污染相同四元组新连接的风险。经典等待为 2MSL，具体系统取值及复用策略以实现为准；同时关闭也可能双方进入，不能绝对说“只有客户端”。

状态异常怎样推理

大量 CLOSE_WAIT 常提示应用未及时关闭收到 FIN 的连接；大量 TIME_WAIT 可能只是短连接负载特征，应结合连接频率与资源耗尽判断。重用端口、减少等待等调优牵涉正确性和系统版本，先考虑连接复用与正常关闭，不应凭状态多就直接改内核参数。

推演与反例

三次握手中分别确认了什么

A 发送 SYN 和初始序号 x；B 返回 SYN、自己的初始序号 y，并确认 x+1；A 再确认 y+1，双方才完成相互序号认知。SYN 占用序号空间，第三次确认使 B 知道自己的同步信息已被 A 收到。还必须结合旧重复报文和具体状态机解释，不能简化为“验证网络能不能双向通”就认为两次足够。

四次挥手为什么也可能看到三段

一方 FIN 只表示它不再发送数据，对方仍可继续发送自己的剩余数据，因此 ACK 与另一个方向的 FIN 在逻辑上分开；若条件合适可合并到一个报文段。收到对端 FIN 后应用迟迟不关闭，可能停在 CLOSE_WAIT；主动关闭方完成最终确认后通常进入 TIME_WAIT。两种堆积的排查方向不同。

继续追问

第三次 ACK 丢了会怎样？

服务端可能重传 SYN+ACK，客户端再次确认；携带有效确认的后续数据也可推进状态。关键是双方状态机和重传处理，不能说连接立刻永久失败。

TIME_WAIT 能一律调短或删除吗？

不应先从参数下手。先看连接创建频率、主动关闭方、连接复用和资源是否真的耗尽；缩短保护时间可能改变迟到报文隔离与重传处理条件。

依据与核对范围

RFC 9293: TCP — 握手、序号、字节流和关闭；§3.4–3.8。

tcp(7) — Linux TCP 状态、缓冲与选项。

TCP 流量控制与拥塞控制有何不同？

流量控制防止接收端缓冲被压垮，拥塞控制防止网络路径被压垮。发送方能在途的数据同时受接收窗口 `rwnd` 和拥塞窗口 `cwnd` 约束，还要扣除已经在途的未确认字节。

原理与成立条件

流量控制与拥塞控制保护谁

流量控制用接收窗口 `rwnd` 保护接收方缓冲，拥塞控制用发送端估计的 `cwnd` 控制向网络注入的数据量。可发送的新数据通常受两者较小值减去当前在途字节限制。接收方很快不代表网络没有拥塞，链路空闲也不代表慢应用有足够接收空间。

经典慢启动与拥塞避免

Reno 类经典算法在慢启动阶段随新 `ACK` 扩大 `cwnd`，理想条件下每 `RTT` 近似翻倍；达到 `ssthresh` 后转入较慢的增长，典型拥塞避免约每 `RTT` 增一个 `MSS`。丢包或超时触发减少窗口与恢复。现代系统可采用 CUBIC 等其他算法，不能把某个经典算法的每个数值当成所有 TCP 的固定行为。

快速重传与 RTO

重复 `ACK` 可能表明后续数据到达而中间有缺口，经典三次重复 `ACK` 可触发快速重传，不必等 `RTO`。乱序也可能产生重复 `ACK`，因此推断并非完美。`RTO` 根据 `RTT` 及波动估计并退避，重传样本有歧义；不能把应用请求超时与 TCP 重传定时器混成一个参数。

接收窗口为零时如何恢复

发送方暂停正常发送新数据，并通过零窗口探测等机制获取窗口恢复信息，避免窗口更新丢失后双方永等。应用停止读取可逐步传导为 TCP 背压，但若应用先把数据搬进无限队列，内核窗口仍可能持续打开，压力只转移到用户态内存。

推演与反例

为什么接收缓冲很大仍可能很慢

即使 `rwnd` 很大，连接刚开始时 `cwnd` 可能较小，需要根据 `ACK` 逐步增长；发生丢包或拥塞信号后又可能收缩。反过来，路径空闲但接收应用读得慢，`rwnd` 可能变小甚至为零。慢启动不是“线性慢慢增大”，经典过程按 `RTT` 可近似指数增长；具体算法和实现版本会改变细节。

带宽时延积与缓冲膨胀

若路径带宽为 `B`、往返时间为 `R`，要持续利用链路通常需要量级为 `B×R` 的在途数据；缓冲过小限制吞吐，过大且缺少控制又可能把排队时延推高。吞吐、时延和丢包需要一起看，不能只把 `socket` 缓冲调到最大。串口虽然不运行 TCP，同样要面对生产消费速率与排队增长，只是控制机制不同。

继续追问

扩大 **socket** 发送缓冲一定降低延迟吗？

可能提高高带宽长 RTT 场景吞吐，却也允许更多数据排队，增加延迟和过期数据。要结合带宽时延积、业务新鲜度和实际瓶颈。

超时重传说明一定发生拥塞了吗？

不一定，可能有链路错误、接收端停顿或 ACK 丢失；TCP 仍按其算法对信号做保守处理。应用排障应结合具体路径证据，不能把协议动作当成唯一根因。

依据与核对范围

RFC 5681: TCP Congestion Control — 拥塞窗口、慢启动与拥塞避免。

RFC 9293: TCP — 握手、序号、字节流和关闭；§3.4–3.8。

T049 原理问答

MQTT QoS 0、1、2 到底保证什么？

MQTT QoS 0 不确认，QoS 1 用确认实现至少一次交付并允许重复，QoS 2 用分阶段状态避免同一次协议消息被重复交付。它们首先约束相邻 MQTT 通信端点之间的协议交付，不自动把外部业务动作纳入事务。

原理与成立条件

QoS 的保证限定在哪一段

MQTT QoS 描述发送方与接收方之间一次协议传递的交付语义，发布者到 broker、broker 到订阅者是各自的一跳，订阅获准的 QoS 也限制下行级别。QoS0 没有 MQTT 层确认；QoS1 用 PUBLISH/PUBACK，重发可能产生重复；QoS2 用四步交换与报文状态避免同次协议消息重复交付。

QoS2 四步为什么能去重

PUBLISH→PUBREC 表示接收方保存处理该报文所需状态，PUBREL→PUBCOMP 推进并结束这次交付。

Packet Identifier 与双方状态机使重复报文能被识别，而不是再次作为新消息。状态必须按会话契约保存；发布者以新标识再次提交同一业务内容，仍是新的协议消息。

三类确认不能混用

publish API 返回成功常仅表示客户端库接纳；PUBACK/PUBCOMP 表示协议阶段完成；业务响应表示订阅应用或更下游设备的执行结果。即使 QoS2 已交付一次，业务回调执行一半后进程崩溃，数据库或执行器效果仍需应用自己的事务与恢复方案。

选择 QoS 要连同资源讨论

更高 QoS 增加报文、缓存与状态维护，低带宽节点或断续网络尤其需要容量预算。周期遥测可按业务容忍损失选 QoS0，重要命令常结合 QoS1 与应用幂等。不能仅靠 QoS 数字决定端到端可靠性，持久会话、客户端本地持久状态和上层补偿同样重要。

推演与反例

QoS 2 的四步在防什么重复

PUBLISH、PUBREC、PUBREL、PUBCOMP 使双方在不同阶段保留消息标识与处理状态。某一步丢失时可以重传对应控制报文，接收方依状态避免重复向后交付同一协议消息。它依赖会话状态及标识的正确保存；发布应用自己用一个新标识重新提交相同业务，仍可能成为另一条合法消息。

Broker 确认与设备执行隔了几层

发布方收到 PUBACK，说明当前 MQTT 交互完成了规定确认，不代表某个订阅应用已经处理，更不代表订阅应用控制的电机已经动作。Broker 到订阅者是另一段交付，订阅最大 QoS 也影响实际等级。需要业务闭环时，应让最终执行者返回稳定业务 ID 对应的结果，并定义掉线、重复与重启恢复。

继续追问

DUP=0 是否证明业务从未收到过这条内容？

不能。它只涉及这次控制报文的重发标记；同一业务内容可以以新报文 ID 再发布，应用仍需自己的稳定请求标识。

QoS 1 重复时 DUP 必然为 1 吗？

不能把 DUP 当业务去重键。规范允许同一业务内容以不同 Packet Identifier 再次发送；业务重复与某次 PUBLISH 的重发是不同层次。

依据与核对范围

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

Eclipse Mosquitto: C API — publish 返回值、网络线程、回调与重连。

T050 原理问答

Clean Session、Retain、Keep Alive 和遗嘱分别是什么？

Clean Session 决定会话状态是否保留，Retain 保存主题最近一条保留消息供新订阅者获取，Keep Alive 检测连接活动，遗嘱用于异常断开后的通知。这四者不能互相替代。

原理与成立条件

Clean Session 保存的是会话状态

MQTT 3.1.1 中 Clean Session=true 请求使用新的会话状态，断开后不保留该会话；false 配合稳定 ClientId，可让 broker 保留订阅及相应未完成 QoS 状态。客户端也需要正确保存它负责的状态。它不是任意长时间的无限消息仓库，更不替应用持久化尚未交给 MQTT 的命令。

Retain 是主题最后值语义

带 RETAIN 发布让 broker 保存某主题的最后保留消息，之后匹配的新订阅者可先取得该值。它适合配置、状态快照，却不是历史队列。对“执行一次动作”使用 retain 可能导致新订阅或重连时再次收到旧动作，必须慎重区分目标状态与一次性指令。

Keep Alive 与遗嘱

Keep Alive 限制控制报文间隔，空闲时 PINGREQ/PINGRESP 帮助确认 MQTT 通信仍在推进；broker 对超时有规范规定。遗嘱是连接异常结束等条件下由 broker 代发的消息，正常 DISCONNECT 通常抑制遗嘱。它们判断 MQTT 连接状态，不验证传感器、数据库和命令业务是否健康。

ClientId 冲突与在线判定

同一 ClientId 的新连接可能导致已有连接被断开，多个设备错误共用标识会表现为反复上线掉线。在线状态还存在网络分区、延迟和遗嘱发布时差，不能当成绝对实时事实。需要业务健康时再增加状态时间戳与有效期，避免旧 retain 在线值被长期误解。

推演与反例

持久会话需要什么前提

MQTT 3.1.1 中要使用稳定 Client ID 并按 Clean Session 设置恢复会话，Broker 和客户端还要具备相应状态保存能力。订阅、未完成 QoS 交换及离线消息的处理受协议与服务端资源约束。只把 Clean Session 改成 false，并不会自动为应用生成跨重连业务 ID，更不会对未入库的本地命令在进程重启后恢复。

为什么控制命令慎用 Retain

新订阅者建立订阅时会收到主题的保留值，适合“设备当前目标配置”这类状态表达；若内容是“再开一次阀门”这样的事件，重连订阅可能触发过期动作。命令应带有效期、身份和明确的状态/事件语义。Keep Alive 表示协议连接活动，遗嘱出现也不一定表示硬件损坏，可能只是网络分区。

继续追问

Clean Session=false 加 QoS1 就能自动补齐掉电期间所有遥测吗？

不能补尚未发布或客户端已丢失的数据；broker 的会话持久性、限制和客户端恢复能力也要核实。应用持久补传需另设计。

心跳业务消息与 MQTT PING 能互相替代吗？

PING 主要验证 MQTT 连接路径，业务心跳可带传感器状态、任务进展和数据新鲜度。两者观察层次不同，应按故障定位需求设计。

依据与核对范围

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

T051 原理问答

断网、重连、消息重复时怎样保证业务可追踪？

断网恢复要解决两件事：消息有没有被持久保存，以及重复到达时能否识别为同一个业务。重连只是恢复传输通道，不能填补已经丢失的业务记录。

原理与成立条件

连接恢复与业务恢复的区别

重新建立 TCP/MQTT 连接，只恢复通信通道；断开期间哪些请求已执行、哪些结果尚未提交，仍需业务记录。瞬断可能发生在任意阶段，发送方看到失败并不证明对端未处理。恢复逻辑首先区分可直接重试、需查询结果和必须人工/补偿处理的请求。

outbox 与 inbox 分别解决什么

outbox 在本地事务中保存待发送业务记录，让进程崩溃后仍能找到未确认工作；inbox 用业务 ID 标记接收与处理结果，识别重复。发送成功后删除 outbox 与远端效果仍通常不是同一事务，所以可重复发送加接收去重是常见组合，而非声称网络跨端天然原子。

重试必须有身份与节奏

同一业务重试使用相同 ID，附带类型、参数摘要和过期时间；新动作使用新 ID。指数退避加抖动可以减少大批客户端同时恢复时的冲击，仍需最大次数/时长与可观察失败。对明显的参数错误反复重试没有意义，错误分类决定恢复策略。

重连后的顺序和有效期

离线期间积累的命令可能已经过时，恢复时应检查有效期与当前设备状态。配置可合并为最新目标，动作命令不可随意折叠或重排。去重记录保留期至少与允许业务重试的范围协调，删除记录后仍允许旧请求重发会重新打开重复执行窗口。

推演与反例

outbox 怎样关闭本地提交空隙

若先改数据库再发消息，进程可能在两步之间崩溃；若先发消息再改数据库，也可能产生相反不一致。可在同一数据库事务里写业务变化与待发送 **outbox** 记录，再由后台反复发送，确认后标记完成。发送与标记之间仍可能重复，所以接收方还需要稳定 ID 和 **inbox/幂** 等处理；这是一组配合机制，不是单个发送队列。

有限资源如何决定恢复策略

断网无限久而数据持续产生，有限磁盘不可能保证永久保存所有数据。应定义保留期、空间上限、优先级、合并或拒绝策略，并区分遥测与控制命令。恢复时也要限速，避免把全部积压瞬间灌入节点导致新一轮超时。可靠性承诺必须包含这些资源与故障持续时间条件。

继续追问

云端没有收到成功，是否应该换一个 ID 再发？

如果是在重试同一次业务，换 ID 会绕开去重，可能再次执行。应保留 ID 并查询或重发；只有明确的新业务才分配新 ID。

数据持久化成功后能立即告诉用户动作成功吗？

只能报告已可靠接纳或已记录，执行成功仍需要执行方结果。接纳、传输、执行和结果持久化是四个不同里程碑。

依据与核对范围

Eclipse Mosquitto: C API — **publish** 返回值、网络线程、回调与重连。

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

SQLite: **Transaction** — 事务边界与提交失败。

T052 原理问答

心跳、健康检查与有效进展分别能证明什么？

心跳证明某条路径在某个时刻还能产生活动，不证明所有功能健康。健康检查应把任务存活、业务进展、外设有效性、数据新鲜度和通信双向能力分开观测。

原理与成立条件

心跳只能证明观察到的层

链路心跳表明某个通信路径最近可达；进程心跳表明某段代码还能运行；业务心跳需证明关键工作有进展。一个线程持续回应 ping，其他线程可能已经死锁；因此健康定义应围绕关键服务，而不是仅看 TCP 连接存在或 CPU 未复位。

超时判定是怀疑，不是证明

网络延迟、调度暂停和拥塞都可能让健康节点晚回心跳。缩短间隔提高发现速度，也增加流量和误报；通常需要连续失败阈值、恢复去抖和单调时钟。网络分区下双方都可能活着却互不可达，不能仅凭一次超时断言远端已死。

健康信息应该包含什么

分别报告最近通信、最近成功采样、各传感器有效性、队列积压和错误计数，让上层区分节点离线、局部传感器失败及处理变慢。状态位是瞬时快照，计数能表达长期趋势，时间戳说明新鲜度；用单一布尔 **online** 会丢失很多诊断信息。

心跳与负载控制

业务忙时若心跳被排在同一拥塞队列尾部，会把过载误判为断线。可为控制消息保留预算，但不能让心跳独立畅通后掩盖业务完全停滞。设计要同时考虑探活优先级和“已处理多少有效工作”的进度指标。

推演与反例

“有心跳、命令没响应”如何成立

发送任务和采样任务仍运行，能周期性发心跳，但接收 DMA 已因错误停止；或者接收仍有数据，命令任务却卡在传感器锁上。此时连接看起来活着，控制链实际已失效。要区分这两类问题，需观察 RX 字节/错误计数、解析成功数、命令开始/结束、TX 进度，而不是仅增加心跳频率。

有效进展要和任务职责绑定

周期采样任务可上报上次采样完成时间，命令任务可上报状态机推进或有界空闲循环，写库任务可上报队列消费位置与错误状态。空闲任务不应因为没有业务而被误判故障；持续积压时又不能仅凭循环转了一圈就视为健康。阈值要覆盖正常最长工作及调度延迟。

继续追问

心跳一直正常，数据库却没有新数据，从哪里查？

按采样有效性、串口解析、业务分流、写库入队和提交逐层检查；心跳只缩小部分故障范围，不能跳过数据链验证。

节点在线时为什么仍要发送传感器无效标志？

在线只说明节点或部分任务存活。独立有效性可防止把上次成功读数当作当前新数据，也帮助区分设备离线与单个传感器故障。

依据与核对范围

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

T053 原理问答

设备接入网络需要哪些认证与输入边界？

网络安全首先明确通信对象是谁、消息有没有被改、调用者允许做什么，以及外部输入最多消耗多少资源。TLS、权限控制、输入验证与业务去重各自解决不同问题。

原理与成立条件

安全需求拆成三件事

认证回答对方是谁，授权回答其能做什么，完整性/保密性保护传输内容。TLS 经正确证书与主机名验证可建立受保护通道，但应用仍需对设备、主题和命令权限进行授权。只加密而不验证身份，可能仍连接到攻击者控制的端点。

为什么 CRC 不能防攻击

CRC 无密钥且线性，攻击者能修改数据并重算；MAC 或认证加密借助秘密密钥验证来源与内容。即便有 MAC，也要用请求 ID、计数或 nonce 及接收窗口防重放；一个过去合法的开阀命令被原样重播，校验仍可能完全正确。

输入验证怎样形成边界

在分配与复制前校验长度，解析数值时检查完整消费、溢出和范围，对设备标识与命令类型建立白名单/权限。传感器周期等参数还应符合资源预算，不能仅满足整数类型范围。限制连接数、队列量与重试速率，防止合法格式的洪水耗尽资源。

密钥与部署也影响保证

私钥不硬编码进公共仓库，配置文件权限与进程身份应最小化；证书轮换、设备撤销、可信时间及固件完整性需要生命周期方案。日志要保留足够审计信息又避免输出凭据。安全机制必须沿实际命令路径验证，不能因为网络库支持 TLS 就宣称应用已启用。

推演与反例

TLS 完成了哪些验证才有意义

加密通道如果没有正确验证证书链、主机名或预共享密钥身份，仍可能连接到错误对端。服务端认证与客户端认证也不同；Broker 确认设备身份后，还要按主题或业务权限限制发布订阅。CRC 只能发现某些传输差错，不提供密钥身份鉴别，不能替代 TLS 或消息认证。

合法连接也可能提交危险输入

已认证客户端仍可能发超长 JSON、越界周期、未知设备或无限命令。应在分配大块内存、排队和执行前验证大小、类型、范围、速率与权限，必要时给业务设置有效期和防重放标识。安全策略必须与有界队列和错误响应连接起来，否则“有认证”仍可能被合法流量拖垮。

继续追问

TLS 已防篡改，还要业务幂等吗？

需要。合法重试、进程崩溃和重复发布并不是攻击，TLS 不知道动作是否已执行，也不替应用保存业务结果。

串口在本地，是否完全不需要输入校验？

不需要默认把它当公网攻击面，但物理噪声、驱动错误、测试工具和被替换节点都可能产生异常输入。长度、状态与范围校验本身就是基本正确性要求。

依据与核对范围

RFC 8446: TLS 1.3 — 认证、完整性与传输保护的边界。

OASIS MQTT 3.1.1 — §3.1.2 会话/遗嘱/Keep Alive、§3.3 Retain、§4.3 QoS。

存储、日志与运行期生命周期

T054 原理问答

SQLite 的连接、读写并发与线程模式怎么理解？

SQLite 可以有多个连接，但同一数据库文件的写入需要协调；WAL 改善读写并发，通常仍只有一个写者。让写连接归一个线程、读连接归相应读线程，可以把事务状态与线程所有权说清楚。

原理与成立条件

单写者为何适合 SQLite

SQLite 是嵌入式数据库，连接在进程中执行 SQL；多个线程各开连接并不意味着同一库可以任意并发写。串行写者集中管理事务和顺序，减少应用层连接竞争。它不能让慢存储消失，而是把写入延迟隔离到专门队列，并使接纳、提交和失败回报边界可管理。

WAL 的读写路径

修改页先追加到 WAL，读事务记录一个可见提交边界，读取时组合数据库文件与该边界前的 WAL 页，从而获得一致快照。写者可继续追加，不必让读者看见中途变化。WAL 允许读写并行，但仍只有一个写者能在同一时刻推进写事务，并不是“所有操作无锁”。

checkpoint 与长读事务

checkpoint 把已提交 WAL 内容迁回主数据库；若仍有读者依赖较旧快照，回收或推进会受限制。长期不结束的查询可能导致 WAL 增长、I/O 和恢复压力。读连接独立很有用，但仍要限制长事务与查询范围，并观测 checkpoint 和 SQLITE_BUSY。

线程安全模式不等于业务安全

SQLite 编译和运行期线程模式决定内部互斥行为；即使 `serialized` 模式保护连接，也不能让多个线程随意交错 `BEGIN`、`INSERT`、`COMMIT` 还保持各自业务事务边界。连接归属、语句寿命和事务所有权应由应用明确规定，而不是寄希望于库内部锁自动猜测意图。

推演与反例

`serialized` 模式为什么不等于业务事务安全

SQLite 的 `serialized` 模式可在接口层对共享连接调用做串行化，但线程 A 的 `BEGIN`、线程 B 的 `INSERT`、线程 A 的 `COMMIT` 仍可能在业务上混用一个事务上下文。数据库内部互斥不理解“这些语句属于哪个请求”。因此连接使用权、预编译语句复用和事务范围仍需应用层管理，不能看到线程安全编译选项就随意共享连接。

WAL 的读快照怎么成立

写者把新页面版本追加到 **WAL**，读事务基于开始时可见的提交位置读取一致快照。读者不必因为每次追加而停止，但长读事务可能阻止检查点回收其仍需看到的历史版本，导致 **WAL** 增长。独立读连接缓解互相干扰，却不消除文件级锁、检查点和磁盘容量约束。

继续追问

WAL 开启后为什么仍会遇到忙？

写者间仍需串行，**checkpoint**、锁升级和其他操作也可能竞争。应按错误类型与事务状态设计有限等待/重试，不可无限循环 **SQLITE_BUSY**。

多开几个写线程就能线性提升 **SQLite** 吞吐吗？

通常不能，文件级写者竞争仍存在，反而增加锁等待。先减少每行提交开销、复用语句并缩短事务，再用测量判断是否需要换存储架构。

依据与核对范围

SQLite: [Write-Ahead Logging](#) — 读写并发、**checkpoint** 与 **WAL** 保留。

SQLite: [Using SQLite in multi-threaded applications](#) — 连接线程模式与应用事务归属。

T055 原理问答

批量写入为什么能提速，怎样控制额外延迟？

批量写入的收益主要来自把多条记录合进较少的事务，摊薄事务与同步开销。是否额外等待攒批，是另一个策略选择，直接影响低负载延迟。

原理与成立条件

批量为什么能加速

每条记录各开一次事务，会重复承担事务管理与同步成本。多行共用一次事务可摊薄这些固定开销，复用预编译语句还可减少解析与编译 **SQL** 的工作。收益取决于存储与事务配置，不是“插入次数除以批大小”就能得到性能倍数。

三种批量边界

按行数限制内存，按时间限制低负载等待，按字节数限制大记录；实际可组合使用。等待凑满批会增加首条记录延迟，机会式批处理则只合并已经排队的数据，不主动等新数据。两者需要明确区分，尤其不要把一个行数阈值描述为定时刷新机制。

失败时批次怎样定义

批量事务必须检查 **BEGIN**、每次绑定/step 和 **COMMIT**，失败时按事务状态回滚并回报。若 **BEGIN** 失败仍继续执行，语句可能落入不同事务语义；若中间某条失败却继续提交，可能形成部分成功。接口需要返回提交结局或明确允许逐项成功，不能仅因使用了 **BEGIN** 字符串就宣称原子批次。

排队与确认的业务边界

入队成功说明任务被内存队列接纳，数据库提交才说明到达数据库层的持久边界。进程在二者之间退出会丢队列内容。控制任务如换库或停止不能被跨越合并，否则先后关系被改变；批处理优化应服从业务顺序，不应把控制标记当普通行忽略。

推演与反例

两种批处理的延迟不同

固定每 100 ms 刷一次，即使只有一条数据也可能多等近 100 ms；机会式批处理先阻塞等第一项，再立即取走此刻已有项，到数量上限或暂时无数据就提交。前者主动攒批提高合并机会，后者利用已有积压，空闲时延迟更低。两者都必须限制单次批量，避免大事务长期占写锁或拖住资源切换。

上限单位必须一致

队列容量可能按任务数计，一个任务又包含多行记录；事务阈值可能按行数计，加入最后一个任务后实际行数可略超阈值。算内存和延迟时不能把“1024 项”直接当“1024 条传感数据”。还要考虑字符串、vector 容量与队列节点开销，不能只乘 payload 长度。

继续追问

批次越大越好吗？

更大批次占用更多内存、延长锁持有和失败恢复时间，也可能恶化查询及尾延迟。需在吞吐、最大等待和恢复成本间选择。

合批时遇到“切换数据库”任务怎么办？

它应作为顺序屏障：先把屏障前的行写入旧连接，再切换，屏障后数据使用新连接。跨过控制任务继续合批会破坏提交顺序。

依据与核对范围

SQLite: Transaction — 事务边界与提交失败。

ACID、WAL 和断电安全有什么关系？

ACID 分别描述原子性、一致性、隔离性和持久性，WAL 是实现事务与并发的一种机制，不是“绝不丢数据”的开关。持久性还依赖同步配置、文件系统和存储设备兑现写入承诺。

原理与成立条件

ACID 应分别解释

原子性要求事务效果整体成立或撤销；一致性要求事务维护已定义的约束，数据库不能自动理解全部业务规则；隔离性约束并发事务互相可见的方式；持久性描述成功提交后在规定故障条件下保留。隔离级别与错误处理仍需单独讨论，不能把四个词合成“绝不丢数据”。

WAL 为什么有助于恢复

先把可恢复的修改和提交信息写入日志，再在适当时机迁入主文件；恢复时依据日志识别完整提交，避免把未完成事务误当成功。数据库文件和 WAL 共同构成当前状态，运行时随意只复制主文件可能丢掉最新数据，备份应使用一致的数据库备份方式。

返回值是事务证明的一部分

提交可能失败，I/O 错误、空间不足和锁竞争都要检查。某些错误会改变事务是否仍活跃，不能盲目重试同一语句或报告成功。业务 API 若完全不向上游暴露提交结局，上层最多知道已尝试写入，无法构建严格的端到端提交确认。

推演与反例

COMMIT 与 checkpoint 各做什么

WAL 模式提交记录让这一批变化成为已提交事务，checkpoint 再把合适的 WAL 页面合并到主数据库。提交不要求每次都把全部变化写回主文件，所以能减少随机写与等待；恢复时依据 WAL 中的提交边界判断哪些变化有效。长读事务可能限制检查点推进，因此需要监测 WAL 大小与读事务时间。

NORMAL 与 FULL 的具体差别

WAL 配合 synchronous=NORMAL 在正常条件下保持事务原子性与一致性，但系统掉电或操作系统崩溃可能让已经返回成功的最近事务回退；FULL 在每次提交增加必要同步以提高该层持久保证。进程自身崩溃与整机断电不同，页缓存仍在时恢复条件更好。无论何种设置，硬件若谎报同步完成，软件也无法单独证明物理介质已保存。

继续追问

fsync 成功就能保证任何硬件故障都不丢吗？

它建立操作系统和设备承诺下的持久边界，不涵盖失效介质、虚假缓存刷新实现或物理毁坏。面试中需说明故障模型，而非给无限保证。

一个事务里单条 **INSERT** 失败，会自动整批回滚吗？

取决于错误类别、冲突策略和应用如何处理。应用若记录错误后继续执行并提交，可能形成部分业务成功；必须明确检查返回值并给出批次失败策略。

依据与核对范围

SQLite: [Transaction](#) — 事务边界与提交失败。

SQLite: [PRAGMA synchronous](#) — WAL/NORMAL 与断电持久性。

T057 原理问答

索引、预编译语句和查询上限为什么重要？

索引减少定位和排序的工作，预编译语句减少重复解析并支持安全参数绑定，查询上限限制结果生成与传输成本。它们解决的瓶颈不同，应按真实查询模式组合。

原理与成立条件

B-tree 索引怎样减少扫描

索引按键有序组织，先通过树定位范围，再扫描匹配项，避免逐行检查整表。组合索引按多个字段的字典序排序，查询条件对前导字段的约束决定能缩小多少范围；不能把“最左前缀”简化为后面的列绝对毫无价值，优化器还会考虑排序、覆盖、统计和其他访问策略。

用实际查询推导索引

例如按 `device_id` 等值、`ts` 范围查历史并按 `ts` 排序，可考虑 `(device_id, ts)`。若只按 `ts` 跨设备查询，该索引未必最合适。索引包含查询所需全部列时可能覆盖查询、减少回表；代价是额外空间和每次写入的维护成本，不能给每列都建索引后再谈写入性能。

预编译与绑定分别解决什么

`prepare` 把 SQL 编译成可执行语句，重复执行时 `bind` 参数、`step`、`reset` 并按需清理绑定，减少重复编译。绑定值还避免把用户输入拼进 SQL 语法，但表名/列名等标识符通常不能当普通值参数绑定，需要白名单构造。绑定内存的复制与寿命策略也要遵守接口契约。

怎样判断优化是否生效

看 `EXPLAIN QUERY PLAN` 与代表性数据上的延迟，区分全表扫描、索引查找、排序临时结构和结果返回成本。`LIMIT` 不能消除大 `OFFSET` 的跳过成本，大量历史翻页可考虑按稳定键进行游标翻页。测量要带数据量、分布、缓存冷热和并发负载。

推演与反例

联合索引为什么看字段顺序

查询 `WHERE device_id=? ORDER BY ts DESC LIMIT n` 时，以 `device_id`、`ts` 排序的索引可先定位某设备范围，再按时间方向扫描少量记录。若只建相反顺序，可能需要遍历更多候选项。等值过滤、范围过滤、排序方向和覆盖列都会影响计划，最终用 `EXPLAIN QUERY PLAN` 核对，不能只凭“建了索引”判断有效。

绑定值不等于拼 SQL 文本

预编译后用 `bind` 传递参数，使用户输入作为值解释，避免把引号或运算符拼进 `SQL` 结构。表名、列名等结构通常不能靠普通值参数代替，需要白名单。语句复用还要正确 `reset`、处理绑定数据寿命和错误码；数据库行指针一般只能在接口规定期间使用，返回给业务前应取得必要副本。

继续追问

查询只有十行，为什么仍慢？

可能先扫描或排序了大量候选数据，最后才 `LIMIT`；返回行数少不代表访问行数少，要查执行计划和条件的可用索引。

索引越多越好吗？

每次写入还要维护索引，占用磁盘和缓存。应围绕高频查询建立必要索引，并衡量写放大与实际扫描量。

依据与核对范围

SQLite: [Query Planning](#) — 组合索引、过滤与排序。

SQLite: [Binding values to prepared statements](#) — 绑定数据与语句执行。

T058 原理问答

为什么异步日志要在锁外写盘，队列满怎么办？

异步日志把磁盘 `I/O` 从调用线程移到消费者线程，但生产者仍承担格式化、同步和排队开销。锁应只保护缓冲交接，慢写盘放在锁外；队列满时的策略必须显式定义。

原理与成立条件

异步日志改变了哪段成本

业务线程把记录写入内存缓冲或队列，后台线程集中写文件，将磁盘等待从业务路径移开。格式化、复制、锁与入队仍发生在前台，严重过载时还有满队列问题。所谓异步只描述交接模型，不代表日志开销为零或日志已经落盘。

双缓冲的交接规则

生产者写当前缓冲，满时把它交给后台并换到空缓冲；后台独占待写缓冲直到 I/O 完成。交换应在短锁内完成，实际文件写出尽量在锁外。没有空缓冲时要选择阻塞、丢弃或降级，不能同时让两方写同一块内存，也不能后台未完成就复用。

顺序与持久化边界

同一队列可保留入队顺序，但多线程日志的打印时间不等于事件真实发生顺序；时间戳应尽量在事件发生处记录。**write** 成功、用户缓冲刷新和 **fsync** 也属于不同持久边界。发生崩溃时异步日志可能丢尾部，关键审计记录需要更强策略和成本预算。

日志系统也要能失败

磁盘满、路径权限、写入 **EINTR**/短写、零进度与后台线程退出都要处理；错误报告不可递归调用同一个失败日志通道。要限制记录长度、速率和队列高水位，避免故障时每次重试都打印大量日志，把原始故障放大为 I/O 与内存雪崩。

推演与反例

双缓冲怎么减少临界区时间

生产者向当前缓冲追加，缓冲满或需要刷新时，在短锁内把整块缓冲转交待写队列并换上空缓冲；消费者取得待写块后释放锁，再循环处理短写与错误。若消费者一直拿着同一把锁写文件，其他线程仍会被磁盘延迟阻塞，名义上的后台线程就失去主要价值。

有限缓冲必然需要取舍

磁盘暂停期间到达日志总量超过队列容量，有限内存无法同时做到永不阻塞、永不丢失和无限接纳。可按级别丢弃、采样、阻塞关键日志或降级到另一通道，并累计丢失数。日志若用于业务审计，通常需要比调试日志更强的持久化与失败契约，不能只共用一个“尽力写”的接口。

继续追问

把 **flush** 调得越频繁是否越安全？

先确认 **flush** 指向用户缓冲、内核写入还是设备同步；频繁同步会增加时延，只能在明确故障模型下讨论更强持久性。

write 成功后日志就能抗掉电吗？

通常只能说明数据被相应内核路径接纳；还要看 `fsync/fdatasync`、文件系统和介质。线程排空、内核接纳、介质持久化是不同完成层级。

依据与核对范围

`write(2)` — 短写、错误与完成边界。

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

T059 原理问答

配置快照、资源切换与事务原子性是什么关系？

原子替换配置指针可以让读者看到完整的新快照，却不能让数据库、串口和网络客户端一起原子切换。内存发布与外部资源变更属于不同事务边界。

原理与成立条件

不可变快照怎样发布配置

读取方一次取得完整配置对象，此后只读；更新方先解析并校验新对象，再原子替换共享指针。读者可以继续持旧版本，不必在每个字段访问时加锁，也不会读到新旧字段混合。发布顺序和对象保活分别由原子同步与共享所有权承担。

指针替换不是资源事务

配置中的串口名、数据库路径或网络参数会触发外部资源变更。新配置对象合法，不代表新串口能打开、新数据库能写或 `broker` 可连接。如果先公布配置再逐一重建失败，就可能出现“显示新配置，部分资源还按旧配置运行”的混合状态。

两阶段更新的思路

先解析、做字段间一致性校验，并尽可能准备候选资源；条件满足后在拥有资源的线程提交切换，最后公布生效版本。无法提前准备或无法回滚的外设，需要显式部分失败状态与恢复动作。整个过程的失败语义应设计出来，不能仅靠 `try/catch` 恢复全部外部效果。

并发更新与版本观测

多次重载可能交错，候选异步完成时需要版本或代数检查，避免旧准备结果覆盖新配置。向用户区分期望版本与实际生效版本，日志记录失败资源及保留策略。配置对象每次都相等也不代表上次资源重建成功，重试逻辑不能仅比较字段后跳过修复。

推演与反例

一次重载为什么会出现混合版本

先发布新配置，再依次重建数据库和串口，如果数据库成功、串口失败，读取配置的线程看到新串口路径，实际 I/O 仍可能走旧端口。若允许这种降级，应暴露期望版本与实际生效状态；若要求全有或全无，就要预构造候选资源、验证后在受控点提交，并处理无法回滚的外部动作。

不可变快照怎样减少读侧锁

加载新配置时先构造完整对象并校验；发布后不再原地修改，读者一次取得一份共享所有权，整次操作都用这份快照。这样避免读一半旧字段、一半新字段，旧读者也不会因替换而悬空。但同一操作若反复重新加载指针，仍可能跨越版本，应让快照范围与业务操作范围一致。

继续追问

atomic shared_ptr 就是无锁配置系统吗？

不保证原子共享指针实现 lock-free，资源重建也可能阻塞；它主要保证一致发布和生命周期，不是对所有路径的性能承诺。

为什么配置语法正确仍不能保证重载成功？

路径可能打不开、端口可能被占用、Broker 可能拒绝连接。语法校验与资源可用性验证是两步，生效结果必须按实际执行反馈。

依据与核对范围

C++ 工作草案：原子内存序 — release/acquire/relaxed 与可见性。项目按 C++17，未引入新标准专属特性。

T060 原理问答

如何有序切换后台资源而不破坏读写一致性？

切换一个被后台消费的资源，最好让“切换”与普通数据经过同一条有序通道。这样顺序本身确定哪批数据属于旧资源、哪批属于新资源，避免读写侧各自切换而短暂错配。

原理与成立条件

热切换要先定义分界点

“换数据库”必须说明旧数据写到哪、新数据从何时写到新库。按墙上时间切分可能受线程排队影响；在单写者 FIFO 中插入切换控制项，可以令其之前任务先处理、之后任务使用新连接，建立可观察的顺序边界。

读者为什么需要保留旧连接

切换时已有请求可能正在查询旧库。立即 `close` 旧连接会让它们悬空或失败；读请求取得共享连接快照，切换只改变后来请求取得的指针，最后一个旧请求结束再释放旧连接。这个机制解决寿命，不会把跨库查询变成一致事务。

候选连接如何失败

可先打开和校验候选库，再排入切换控制项；若准备失败，旧库继续提供服务。若切换完成通知晚到，又发生新一轮切换，需要防止旧完成覆盖当前读连接。候选资源的拥有者也必须覆盖队列等待和回调寿命，失败时明确释放。

切换不是数据迁移

新库可为空，旧历史仍留在旧文件；要合并历史需单独迁移与查询策略。批处理不能把切换前后记录合成一个跨边界批次，也不能在旧任务没执行完时提前关闭写连接。停止与切换同时出现时同样需要统一控制项顺序。

推演与反例

把控制任务放进数据队列的效果

队列为数据 A、数据 B、Swap、新数据 C。消费者先完成 A、B 对旧连接的写入，再替换连接，之后 C 使用新连接。切换完成时发布读侧新连接，才可保证读侧不会先看到一个还未被写侧接管的新库。Swap 入队成功只表示将来会执行，不等于切换已经完成。

旧资源何时可以销毁

正在处理查询的读线程可能还持有旧连接，切换后新查询用新连接，旧查询允许完成并释放自己的引用。写连接的替换也必须等此前批次结束。共享所有权解决“何时最后一个使用者离开”，有序控制任务解决“什么时候逻辑切换”，两种机制各有职责。

继续追问

指针已换到新库，为什么页面还看到旧记录？

已有请求可能持旧快照，这是允许的请求级一致性；如果要求全系统同一瞬间切换，就需要暂停入口、等待旧请求排空等更强协调。

切换任务进不了满队列怎么办？

应返回明确失败或在有限期限内等待，保留旧资源继续运行；不能先把配置标成已生效，再默默丢掉切换任务。是否允许这种暂时不一致应由接口定义。

依据与核对范围

C++ 工作草案：原子内存序 — `release/acquire/relaxed` 与可见性。项目按 C++17，未引入新标准专属特性。

SQLite: Write-Ahead Logging — 读写并发、checkpoint 与 WAL 保留。

优雅退出为什么要先停止生产者，再排空消费者？

优雅退出按照依赖顺序结束工作：先停止产生新任务，再关闭队列入口、唤醒消费者，按契约排空或取消，等待线程退出，最后释放被借用资源。这个顺序与构造顺序相反但并不总能只靠默认析构自动完成。

原理与成立条件

停机首先停止谁

先停止新的生产和接纳，才可能让存量工作稳定减少；之后通知消费者按约定排空或取消，等待线程结束，再关闭被使用资源。若消费者先退出而生产者继续入队，会留下永远无人处理的任务；若先释放回调依赖，停止过程中可能出现释放后使用。

排空必须与最终提交区分

队列为空只说明任务已被取走，后台可能仍在写数据库。需要消费者处理完当前任务并报告退出，才能确认内存处理阶段结束；最终是否提交成功还取决于错误回报。`join` 保证线程终止，不自动证明其所有业务成功。

退出等待也需要唤醒路径

`stop` 标志不能叫醒条件变量、`epoll` 或阻塞设备调用。为每个等待点设计 `close/notify`、`eventfd` 或有限超时；等待者醒来后用状态谓词判断退出。线程不应持有另一个退出线程所需的锁再 `join`，否则关闭流程本身形成死锁。

有限退出和故障策略

系统服务管理器通常有停止期限，到期可能强制终止。应用应让关键数据优先提交、次要日志有界排空，并记录未完成数量。无限 I/O 会使“优雅退出”没有完成上界；一开始就选择有限等待与可取消接口，通常比析构时补救更可靠。

推演与反例

为什么先销毁队列会产生悬空访问

网络回调可能还在运行并准备入队，如果主线程已经释放队列或 `eventfd`，回调会访问无效资源。先停网络生产者并等待其线程结束，再关闭队列，消费者才能在“不会再有新项”的前提下安全排空。对相互依赖线程还需避免一边持锁一边 `join`，防止被等待线程正需要同一把锁退出。

排空也必须有业务边界

写数据库能否一直等、远端离线时是否继续重试、日志失败是否阻止进程退出，都需要明确。有限退出期限可能意味着最终取消部分任务，此时应给请求终态并保存必要恢复记录。队列空只说明这一层消费完，后面若还有内核缓冲、网络发送或持久化步骤，不能据此宣称业务全部完成。

继续追问

进程正常退出时 **OS** 会回收资源，为何还要自己关闭？

OS 不能替业务完成队列提交、发送结果或维护设备状态；正常资源回收也有助于发现泄漏和保证嵌入其他程序时的正确性。

为什么 **detached** 线程不适合随意使用？

对象析构后无法自然 **join** 它，后台线程可能继续借用已经释放的成员。除非生命周期独立且清楚管理完成协议，否则显式所有者和可等待退出更易证明正确。

依据与核对范围

[pthread\(7\)](#) — 进程内共享与线程独立状态。

[C++ Core Guidelines](#) — [R.1/资源管理](#)、[CP/并发与接口生命周期](#)。

Cortex-M3 与 STM32 系统基础

T062 原理问答

STM32 上电后如何走到 main，再启动 FreeRTOS？

Cortex-M 复位后先从向量表取得初始主栈地址和复位入口，再由启动代码建立 C/C++ 运行环境，才进入 main。RTOS 调度器又在此之后建立任务运行环境，二者是不同阶段。

原理与成立条件

复位后 CPU 从哪里开始

Cortex-M 的启动向量表首项给初始主栈指针 MSP，第二项给复位处理程序入口；处理器依据启动映射取这些值。STM32 的启动引脚/选项与芯片系列决定映射主 Flash、系统存储器或其他区域。地址 0 的启动别名与实际 Flash 基址是两个概念，不能一概说程序永远从 main 地址开始执行。

C 运行环境由谁准备

复位处理程序通常调用早期系统初始化，把 .data 的初值从非易失存储复制到 RAM，清零 .bss，执行必要运行库和全局构造初始化，再进入 main。具体先后看启动文件。此阶段还不能任意假定全局变量和 C++ 对象都已就绪，早期 SystemInit 的代码必须遵守初始化时序。

main 到调度器之间

通常先初始化 HAL、系统时钟和外设，再创建队列、互斥量、任务等对象。使能中断和启动 DMA 前，应保证 ISR 将访问的句柄与缓冲已有效。调度器启动时选择首个就绪任务并恢复其上下文；任务函数通常无限循环，若要结束，应通过内核接口删除自己而不是普通 return。

为什么启动时序是正确性问题

外设先来一个中断而队列还没创建，会在“看似正常初始化”过程中访问空句柄；时钟修改后未更新延时参数，会让微秒时序错误。调度器前不能直接使用依赖任务阻塞的延时接口。排查未到 main 或未进任务时，要按向量、栈、段初始化、时钟、对象创建和调度器逐层定位。

推演与反例

为什么 **main** 前必须搬运和清零

有初值的可写全局变量，初始内容通常存在 **Flash**，启动时要复制到其 **SRAM** 运行地址；未显式初始化的静态存储对象所处 **BSS** 要清零。否则全局变量不满足语言规定的初始值。随后还可能调用 **C++** 静态对象构造，再调用 **main**。**SystemInit** 与运行库初始化的具体次序以启动文件为准，早期代码不能假定后续才初始化的数据已经可用。

任务怎样从一块栈开始运行

创建任务时内核为它构造初始上下文，栈中预先放入入口、参数、状态等内容；启动调度器后移植层通过异常返回机制开始第一个任务。任务函数不能像普通 **main** 子函数一样自然返回，结束时应显式删除或进入受控路径。调度器启动失败往往涉及内核任务创建、堆容量或中断配置，不能只看任务入口是否有断点。

继续追问

下载成功却进 **HardFault**，先看什么？

检查向量表初始 **MSP** 是否位于有效 **RAM**、**Reset_Handler** 地址及 **Thumb** 状态、链接地址与下载地址是否匹配，再看故障现场；下载校验成功只证明写入字节一致。

Bootloader 跳应用为什么不能只调用一个函数指针？

还要处理向量表、**MSP**、残留中断、**SysTick**、外设和特权/栈选择状态，并校验入口地址。否则应用遇到第一次中断时可能仍跳到旧表或使用错误栈。

依据与核对范围

ST PM0056: **Cortex-M3** 编程手册 — 异常、寄存器、**NVIC**、**SysTick** 和故障状态；Rev 7。

Arm CMSIS: **NVIC** 接口与优先级 — **Priority Grouping**、**NVIC_SetPriority**、**__NVIC_PRIO_BITS**；CMSIS 逻辑值与寄存器编码。

T063 原理问答

Flash、SRAM、DATA、BSS、堆和任务栈如何分布？

链接脚本描述段在 **Flash/SRAM** 中的放置，启动代码把这种布局变成运行时状态。**text**、**rodata**、**data**、**bss**、**C** 库堆、**RTOS** 堆和各类栈有不同来源，不能只看某个数组大小估算 **RAM**。

原理与成立条件

.text、**.rodata**、**.data**、**.bss**

代码通常进 **.text**，常量可能进只读段；带非零初值的静态对象通常在 **.data**，零初始化静态对象通常在 **.bss**。段名和放置是工具链约定与链接策略，不能把 **const**、**static** 这些语言属性直接等同于某个绝对物理地址。优化还可能完全消除未使用对象。

装载地址与运行地址

.data 需要掉电后仍保留初值，所以镜像可在 **Flash** 保存一份，运行时复制到 **SRAM**。它因此同时消耗镜像和 **RAM**；**.bss** 只需记录大小并清零，通常不在镜像保存同样长度的零字节。看 **size/map** 时需区分文件占用与运行期占用。

任务栈和主栈

每个任务需要保存局部变量、调用现场和部分异常上下文，**RTOS** 常从堆分配任务栈；异常处理使用 **MSP**，任务一般使用 **PSP**。任务栈总预算之外，还要给 **ISR** 嵌套、启动代码和主栈留空间。链接脚本里的栈预留不会自动限制所有运行期任务栈。

const 与 **Flash** 写入的误区

const 限定通过相应类型访问的修改规则，不保证对象必在 **Flash**；局部 **const** 可能放栈或寄存器。**Flash** 擦写又有页/半字等硬件粒度、执行时延和寿命限制，不能当普通 **RAM** 随意写。大查找表、格式化库和全局数组分别要从 **map** 中核对，而不是只凭源码估算。

推演与反例

LMA 与 **VMA** 为什么需要两个地址

带初值变量运行时可写，放在 **SRAM** 的 **VMA**；复位后 **SRAM** 内容不可靠，初值又要从 **Flash** 的 **LMA** 装载。链接器提供边界符号，启动代码按它们复制。**BSS** 通常不需要在 **Flash** 为每个零存一份镜像，而由启动循环清零。**noinit** 区若要跨复位保留，必须明确启动流程不覆盖它，并用校验/版本判断内容是否可信。

一份 **RAM** 预算要把哪些项相加

静态变量占 **data/bss**，动态创建的 **FreeRTOS** 任务栈、**TCB**、队列通常来自内核堆，主栈/**MSP** 则还要承受异常处理与嵌套。**C** 库 **malloc** 可能又使用另一块堆。把内核堆数组算在 **BSS** 后再把其中任务栈重复加入总 **RAM** 会双计；反过来只统计应用任务又会漏掉 **Idle** 和 **timer** 任务。**map** 文件与运行期高水位应对应解释。

继续追问

固件 **40 KiB**、**SRAM 20 KiB**，为什么仍可能内存不够？

Flash 文件大小不表示 **RAM** 余量，动态堆、多个任务栈和中断主栈有独立开销；**.data** 还同时占两边资源。

Flash 还剩很多空间，为什么创建任务失败？

创建任务主要需要 RAM 中的栈和控制块，Flash 富余不能直接替代可写 RAM。还要分清总剩余堆与最大连续可分配块。

依据与核对范围

GNU ld: [Linker Scripts](#) — 段布局、装载地址与运行地址。

ST STM32F103x8/xB 数据手册 — 存储规格、时钟与电气参数范围。

T064 原理问答

STM32 时钟树、APB 定时器倍频与 PWM 怎样计算？

外设频率应沿时钟源、PLL、AHB、APB 和外设分频逐级推导。STM32F1 的定时器输入时钟在 APB 分频不为 1 时通常为对应 PCLK 的两倍，不能直接拿 CPU 主频代入所有公式。

原理与成立条件

时钟树为何不止一个频率

内部/外部振荡源经 PLL 和分频形成系统时钟，再分给 AHB、APB 与各外设。CPU 主频、总线频率、定时器输入时钟和外设采样时钟可能不同；修改 PLL 后若仅更新一个全局数字，不代表全部外设分频都仍合法，还需满足电压、Flash 等待周期和总线最高频率限制。

STM32F1 的定时器倍频规则

该系列在 APB 预分频为 1 时，相应定时器时钟通常等于 PCLK；预分频不为 1 时，相应定时器内核时钟为 $2 \times \text{PCLK}$ 。这个规则属于具体时钟树，不能推广成所有 STM32、所有外设都自动倍频。USART、I2C 等也不能照套定时器规则。

时钟误差如何传到系统

UART 位宽、PWM、I2C 速率和 DWT 延时都依赖实际输入时钟，振荡器误差与温漂会影响时序。切换时钟应等待稳定与切换状态，按芯片要求配置 Flash 延迟和分频；故障路径需要有限等待。测量可用 MCO、定时器输出或已知 GPIO 脉宽校验。

PWM 频率与占空比

对向上计数、边沿对齐的常见 PWM 模式，周期仍由 PSC/ARR 决定，比较寄存器 CCR 决定在周期中切换的计数点。例如有效高且 $\text{CNT} < \text{CCR}$ 时输出有效，占空比约 $\text{CCR}/(\text{ARR}+1)$ 。极性、PWM 模式和中心对齐会改变解释；更新预装载可让新参数在更新事件生效，避免周期中途改值产生毛刺。

推演与反例

一个定时器周期怎样算

假设系统 72 MHz，APB1 分频为 2，则 PCLK1=36 MHz，相关定时器时钟为 72 MHz。向上计数模式下，更新频率通常为 $f_{TIM}/((PSC+1)(ARR+1))$ ；PSC=71、ARR=999 时得到 1 kHz。公式中的加一来自计数包含零。高级定时器重复计数器、中心对齐模式等会改变事件频率，应注明模式再计算。

PWM 频率和分辨率如何权衡

边沿对齐 PWM 中，周期由 ARR 决定，占空比由 CCR 与计数周期关系决定；ARR 越大，可分的占空比步数越多，但在给定计数时钟下频率越低。更新 ARR/CCR 还要考虑预装载，避免在周期中途改变导致毛刺。系统提高主频时同时要满足电压、Flash 等待周期及 APB 最高频率等限制。

继续追问

APB1 36 MHz，TIM4 为什么还能按 72 MHz 算？

因为当前 APB1 分频不是 1，F1 时钟树给该定时器内核使用 $2 \times PCLK1$ ；先确认分频配置才能使用这个结论。

改系统时钟后哪些软件常数会失效？

UART 波特率、I2C 时序、TIM 周期、SysTick 重装值以及按 CPU 周期计算的 DWT 延时都可能需要重新配置；不能只更新 SystemCoreClock 变量。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

T065 原理问答

NVIC 优先级为什么数值越小越高？抢占与子优先级怎样区分？

NVIC 中断优先级通常是数值越小紧急度越高，FreeRTOS 任务优先级则通常数值越大越高。抢占优先级决定能否打断当前异常，子优先级只在相应候选异常之间影响选择次序。

原理与成立条件

数字优先级与紧急度相反

Cortex-M 可配置异常优先级数值越小越紧急；FreeRTOS 任务通常数值越大越优先，两套排序不可混用。NVIC 寄存器字段可能有 8 位，但芯片只实现其中高若干位，STM32F103 实现 4 位，因此库常用逻辑值 0..15，再移位写入寄存器。

抢占优先级与子优先级

分组把有效位划分为抢占和子优先级。抢占部分决定能否打断正在执行的异常；同一抢占等级的子优先级只在多个待处理异常间决定先后，不能造成同级嵌套。没有更紧急者时，异常在满足屏蔽条件后进入；`pending`、`active` 与 `enable` 是不同状态。

为什么 FreeRTOS 限制 ISR 调用范围

内核更新就绪链表等共享结构时，用 `BASEPRI` 屏蔽一部分中断。如果更紧急且不被屏蔽的 `ISR` 也修改这些结构，就可能在临界区中打断内核，造成破坏。因此只有落在允许范围的 `ISR` 才能调用 `FromISR`；更紧急 `ISR` 应仅处理独立硬件或通过安全延后路径交接。

怎样核对配置

先确认 `__NVIC_PRIO_BITS`，再确认分组，区分库逻辑优先级与左对齐硬件值。例如逻辑阈值 5、有效位 4，对应 `BASEPRI 0x50`；逻辑 5..15 的可配置 `ISR` 才在该阈值保护范围。0 是最高紧急度且不能用非零 `BASEPRI` 阈值屏蔽，不能让调用 `RTOS` 的 `ISR` 留在默认 0。

推演与反例

逻辑优先级与寄存器编码怎样换算

若实现 4 个优先级位，逻辑值 0..15 存在 8 位优先级字段的高 4 位，逻辑 5 对应寄存器值 `0x50`。`CMSIS/HAL` 接口可能接收未移位逻辑值，底层 `BASEPRI` 配置则可能需要已移位编码。把 5 当成 `0x05` 直接写寄存器会丢失有效位，造成与预期完全不同的屏蔽范围。

分组为什么会影响 RTOS 合法性

`PRIGROUP` 划分已实现的位为抢占位和子优先级位。依赖 `BASEPRI` 的内核希望中断屏蔽与调用 `API` 的范围可按其端口规则判断；某些不合适分组会使开发者给出的数字无法表达预期紧急度。调用 `FromISR` 的 `IRQ` 必须位于可被内核临界区屏蔽的范围，更紧急的 `IRQ` 可用于短硬实时处理，但不能直接进入内核对象操作。

继续追问

把串口中断从 6 改成 3，响应更快就一定更好吗？

若它调用 `FromISR` 而阈值仍为 5，会越过内核保护范围；先满足正确性，再讨论延迟。中断越紧急也会增加其他工作延迟。

两个 IRQ 数值一样会相互嵌套吗？

同一抢占优先级通常不能彼此抢占；挂起后的执行次序还受子优先级和异常号等规则影响。不能仅凭“先到的中断”解释所有情况。

依据与核对范围

ST PM0056: `Cortex-M3` 编程手册 — 异常、寄存器、`NVIC`、`SysTick` 和故障状态; Rev 7。

FreeRTOS: Cortex-M interrupt priorities — 优先级编码、BASEPRI 与可调用 ISR 范围。

Arm CMSIS: NVIC 接口与优先级 — Priority Grouping、NVIC_SetPriority、__NVIC_PRIO_BITS; CMSIS 逻辑值与寄存器编码。

FreeRTOS 官方内核书: Interrupt Management — 第 7 章; 用于通用机制。具体节点实现另对照随包内核源码, 不推定与上游 main 完全相同。

T066 原理问答

中断入栈保存哪些寄存器, MSP 和 PSP 各做什么?

Cortex-M3 异常入口由硬件保存基础返回现场, 软件或 RTOS 移植层保存其余任务现场。线程可使用 MSP 或 PSP, Handler 模式使用 MSP; 进入异常前的那份基础帧可能保存在被打断线程的 PSP 上。

原理与成立条件

MSP、PSP 和 EXC_RETURN

MSP 是主栈, PSP 常供任务使用; Thread 模式可按 CONTROL 选择, Handler 模式用 MSP。异常期间 LR 中保存特殊 EXC_RETURN 编码, 异常返回时据此决定返回模式、使用哪个栈及相关帧类型。它不是普通 C 函数返回地址, 调试时要把异常 LR 与堆栈中原 LR 分开。

RTOS 为什么还要软件保存

硬件基本帧没有自动保存所有通用寄存器。任务切换时移植层再保存 R4-R11 等需要跨上下文维持的状态, 将栈顶记入 TCB; 选择新任务后反向恢复。嵌套中断未必发生任务切换, 内核常把切换延后到最低紧急度 PendSV, 避免在高紧急度处理器中穿插复杂调度。

尾链与延迟到达有什么意义

处理一个异常结束时, 若另一个合适异常已等待, 处理器可通过尾链减少完整退栈再入栈; 更高紧急度异常在入口阶段到达也可改变先服务对象。这些优化减少固定开销, 但总响应仍受屏蔽、总线和更高紧急度工作影响, 不能把手册理想周期数当整机最坏延迟。

推演与反例

一次任务被中断的栈变化

任务在 PSP 上运行, 异常入口把 R0-R3、R12、LR、PC、xPSR 等基础帧压入该任务栈, 然后处理函数在 MSP 上运行; 若 Handler 再被嵌套, 相关现场压入 MSP。基础帧常为 8 个字, 必要时还有对齐填充。不能因为“中断函数用 MSP”就漏算任务栈上的异常入栈空间。

EXC_RETURN 为什么不是普通返回地址

异常处理期间 LR 中的特定 EXC_RETURN 值描述返回到 Thread/Handler、使用哪一个栈等信息。异常返回据此恢复栈中现场。FreeRTOS 的 PendSV 还保存 R4-R11、更新 TCB 栈顶，并加载下一任务栈，让异常返回继续另一个任务。把该值当普通函数地址跳转会破坏返回过程。

Cortex-M3 基本异常帧（按栈内偏移）

```
SP + 0   R0
SP + 4   R1
SP + 8   R2
SP + 12  R3
SP + 16  R12
SP + 20  被打断代码的 LR
SP + 24  返回 PC
SP + 28  xPSR
```

另可能有对齐填充；M3 不含硬件浮点扩展帧。

这 8 个字由异常进入机制保存；PendSV 的 R4-R11 软件保存区不是这张基本帧的一部分。异常处理器当前 LR 中的 EXC_RETURN 和帧里的原 LR 是两回事。

继续追问

任务 PSP 足够大，为什么中断仍可能栈溢出？

ISR 嵌套和 ISR 内调用主要消耗 MSP，需独立预算；任务栈与异常主栈是两套容量，不能只测任务高水位。

切换上下文为什么不是硬件全包了？

硬件只保存定义好的基础帧，不知道哪个任务应运行，也不维护 TCB、延时队列和所有软件寄存器现场。调度选择与剩余保存由内核移植层完成。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态；Rev 7。

Arm CMSIS: 内核寄存器 — __get_MSP / __get_PSP / __set_BASEPRI / __set_PRIMASK；具体 M3 异常帧对照 PM0056。

T067 原理问答

PRIMASK、BASEPRI 与临界区有什么区别？

PRIMASK 广泛屏蔽可配置优先级异常，BASEPRI 按阈值屏蔽一段紧急度范围；它们都不是把芯片所有硬件停止。临界区要说明防的是谁、屏蔽多久以及退出时如何恢复原状态。

原理与成立条件

临界区必须保护同一组访问者

如果共享变量还被更紧急、未屏蔽的 **ISR** 修改，使用 **BASEPRI** 的任务临界区就不能独占它；需要重新设计交接或使用适当更强的短保护。单核关中断也不能阻止 **DMA** 写 **RAM**，更不能推广为多核系统的全局互斥。保护机制的范围必须覆盖实际写者。

嵌套与恢复旧状态

临界区退出要恢复进入前的屏蔽状态，而非无条件开中断；否则内部函数可能提前打开外层要求关闭的中断。**RTOS** 提供嵌套计数和 **ISR** 专用保存/恢复接口，不宜混用手写寄存器修改破坏它们。**ISR** 使用哪个宏也受移植层契约限制。

代价在最坏延迟而非平均占用

长临界区推迟 **SysTick**、串口回调和任务切换，可能造成 **DMA** 缓冲被覆盖、多个 **Tick** 只留下一个待处理状态。区内不应等待队列、互斥量或依赖被屏蔽中断更新的超时。保护几十条指令合理，并不意味着可以把几毫秒传感器事务整体包起来而无需分析。

推演与反例

为什么内核保留更紧急中断

以 **BASEPRI** 建立临界区时，可调用内核 **API** 的中断被屏蔽，因而任务与这些 **ISR** 不会同时修改内核链表；更紧急中断仍能响应硬件，但必须不访问受保护内核结构。这是一份双方契约：若高紧急度 **ISR** 违规调用 **FromISR**，它就可能在内核数据结构更新到一半时进入，问题不一定立刻暴露。

嵌套临界区为何要保存状态

外层已屏蔽中断，内层若退出时无条件全开，会提前破坏外层保护。任务临界区通常维护嵌套计数，**ISR** 专用屏蔽接口可能返回旧掩码供恢复；具体用法由端口规定。普通用户代码不要混用不配对的 **enable/disable** 与 **RTOS** 临界区，尤其不能在临界区里等待依赖被屏蔽中断的完成事件。

继续追问

进入临界区后 **DMA** 还收不收？

若 **DMA** 和外设时钟仍工作，硬件传输可继续；延迟的是 **CPU** 处理相关中断。缓冲因此可能被继续写满甚至覆盖，临界区不是 **DMA** 暂停按钮。

DMA 在关中断时会停吗？

通常不会。**DMA** 控制器仍可搬数据，只是 **CPU** 处理其完成或半满通知被延后；因此长临界区仍可能导致循环缓冲覆盖，甚至使多次硬件事件合并。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态; Rev 7。

FreeRTOS: Cortex-M interrupt priorities — 优先级编码、BASEPRI 与可调用 ISR 范围。

Arm CMSIS: 内核寄存器 — __get_MSP / __get_PSP / __set_BASEPRI / __set_PRIMASK; 具体 M3 异常帧对照 PM0056。

FreeRTOS 官方内核书: Resource Management — 第 8 章; 用于通用机制。具体节点实现另对照随包内核源码, 不推定与上游 main 完全相同。

T068 原理问答

HardFault 应当怎样定位, 为什么不能只在里面死循环?

HardFault 首先是现场采集问题, 再是根因分析问题。应保存异常栈帧和故障状态寄存器, 区分非法地址、指令状态、除零、栈破坏或被升级的其他异常, 而不是只停在死循环里猜测。

原理与成立条件

HardFault 是入口, 不一定是根因

Cortex-M3 的非法访问、执行状态等故障可由 MemManage、BusFault、UsageFault 描述; 未使能相应处理或处理失败时可能升级为 HardFault。HFSR 的 FORCED 等信息指示升级, 再结合 CFSR 子状态判断; 只看“停在 HardFault_Handler”无法定位哪条业务语句有错。

首先保住现场

在极简异常入口根据 EXC_RETURN 选择 MSP/PSP, 保存硬件栈帧中的 PC、LR、xPSR 及必要状态寄存器。读取 BFAR/MMFAR 前检查有效标志。故障可能发生在压栈阶段或栈已损坏, 此时栈帧地址与内容也需要校验, 不能盲目再次深度调用格式化库。

从状态到故障代码

精确总线故障时堆栈 PC 常能帮助定位访问指令; 缓冲写引起的不精确故障可能不能直接指向真正起点。结合反汇编看地址来源、对齐和访问宽度, 再追溯越界、空指针、栈破坏和错误函数指针。使用与故障固件完全对应的 ELF/map, 地址才能映射正确。

复位后还要能读取证据

可把最小故障记录写到适当保留区域或在重启时输出, 配上固件版本、复位原因和校验; 不应在故障态进行无限 I/O。栈水位、哨兵与可选 MPU 可提前发现破坏, watchdog 仅负责最终复位, 不会自动解释为什么出错。

推演与反例

现场如何指向出错指令

根据异常返回信息确认故障时使用 **MSP** 还是 **PSP**，定位堆栈中的 **PC**、**LR** 与参数寄存器；结合 **CFSR**、**HFSR** 及地址有效标志判断 **MMFAR/BFAR** 是否可解释。用对应 **ELF** 反汇编和源码映射检查指令，例如一次访问指针加偏移后的地址。若错误是非精确总线故障，堆栈 **PC** 未必就是最早出错指令，不能过度解读。

为什么故障处理程序本身要克制

栈可能已经损坏，日志堆锁和外设状态也可能不可靠。在 **Handler** 中执行复杂 **printf**、动态分配或无限等待串口，可能再次故障或掩盖原现场。更稳妥的是用最小代码保存有限寄存器到预留区，附带校验和复位原因，下次启动再输出；同时确认看门狗是否允许保留足够取证时间。

继续追问

PC 指向一个正常函数，是否能排除内存越界？

不能，栈或函数指针可能更早被破坏，真正错误与最终异常有时间距离；需结合寄存器、调用链和周边内存找首次破坏点。

开栈溢出 Hook 就能抓住所有栈越界吗？

不能，它只在特定检测点与条件下发现部分问题；越界可能已经破坏调度结构，或发生在 **MSP**。还需静态栈预算、高水位、保护区与故障现场结合。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态；Rev 7。

T069 原理问答

寄存器读改写、BSRR 和位操作有什么并发问题？

寄存器不是普通内存变量，读写可能有硬件副作用。读改写会经过多个操作，既可能与中断竞争，也可能误处理写一清零、读清除等特殊位；应按寄存器契约使用专用置位/复位接口。

原理与成立条件

内存映射寄存器如何访问

外设寄存器映射在处理器地址空间，读写相应地址通过总线触发硬件操作。**volatile** 让编译器按实现约定保留访问，但寄存器可能具有读清零、写 1 清除、只写、保留位等特殊语义，不能把所有寄存器都当普通变量做任意 **|=** 或 **&=**。

读改写的竞态

`REG |= mask` 通常先读整个寄存器、改本地副本再写回；若两次访问间 `ISR` 或硬件改变其他位，旧副本可能覆盖新状态。例如两个执行者分别更新同一输出寄存器不同引脚，仍可能丢更新。解决方法包括专用置位/清零寄存器或覆盖所有写者的适当临界区。

BSRR 为什么适合 GPIO

STM32F1 的 GPIO BSRR 用不同位段表达置位与复位，一次写入让硬件修改指定引脚，避免软件先读 ODR 再写回的窗口。它保护的是这次位更新，不自动保护更复杂的“判断引脚后执行多步”事务；多个引脚有严格同步要求时也要查看单次写及电气时序。

位带与原子指令不是万能替代

位带属于特定地址范围的架构映射能力，不是所有 MCU/寄存器都支持；LDREX/STREX 的适用内存类型和成功条件也有限。设备寄存器最好遵循手册提供的访问方式。更一般地，单次总线原子更新不等于多个对象间的 C++ happens-before。

GPIO 模式与速度容易混淆

推挽能主动驱高和驱低，开漏只主动拉低、释放时需上拉；浮空输入无确定空闲电平，上拉/下拉输入给出默认偏置，模拟模式通常关闭数字输入路径以减少干扰和功耗。GPIO speed 描述输出驱动/边沿能力，不是软件翻转频率；过高边沿速度会增加振铃和 EMI，应按负载与时序选择。

推演与反例

GPIO 的丢更新怎样发生

任务读 ODR 得到旧值，准备置位 bit0；中断先置位 bit1，任务随后把旧值加 bit0 写回，就可能把 bit1 清掉。使用 BSRR 可在单次写中表达某些位的置位或复位，避免为这些操作读取整个 ODR。它不自动使“先判断输入再切换输出”的多步业务原子化，复杂状态仍需协调。

GPIO 模式背后的电气行为

推挽可以主动驱动高低，开漏只主动拉低，释放后的高电平靠上拉；输入浮空可能被噪声影响，模拟模式可关闭部分数字输入路径以适应模拟信号。复用功能把引脚控制权交给外设，但仍需配置正确的电气模式、速度和上下拉。F1 的 MODE/CNF 与后续系列寄存器结构不同，不能跨系列照抄位域。

继续追问

为什么清标志不能直接 `status &= ~flag`?

若寄存器是写 1 清除，读改写可能反而清掉其他已置位标志。先查该位清除语义，再生成正确写值或规定的读序列。

对 W1C 状态寄存器做 reg

= flag 有什么风险？| 读回的其他已置位状态也可能被写成 1，从而一起清除。应按手册要求只写需要清除的位，避免套用普通变量的读改写习惯。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

GCC: Volatiles — volatile 的访问约束，不提供普通内存同步。

T070 原理问答

HAL、LL 和裸寄存器怎么取舍？

HAL 提供状态机、超时和较统一接口，LL 更贴近单次外设操作，裸寄存器让控制最直接。选择应依据时序约束、代码体积、可维护性与对硬件的掌握，而不是预先把某层判为一定慢或一定可靠。

原理与成立条件

三个层次分别封装什么

直接寄存器代码最接近硬件，开发者负责所有状态与时序；LL 提供较薄的寄存器操作封装；HAL 进一步封装句柄、状态机、超时与回调。抽象程度越高通常越容易初始化和移植应用，但可能带额外检查及不显眼的阻塞点，必须读到真正影响时序的路径。

选择看约束而非阵营

普通初始化和非关键路径用成熟 HAL 可减少配置错误；严格微秒时序可用定时器/DMA、LL 或局部寄存器实现。不能仅以源代码行数判断性能，也不能认定 HAL 天然线程安全。句柄状态、锁和回调上下文需要对照当前驱动版本核实。

混用的风险来自状态不一致

应用直接改外设寄存器后，HAL 句柄仍可能以为在原模式运行；直接停止 DMA、改中断或清标志，也可能破坏 HAL 的完成状态机。可混用，但应划清某个外设何时归哪层管理，切换时同步状态，不能让两套驱动同时独立拥有硬件。

超时接口也要读实现

某些 HAL timeout 靠 HAL_GetTick，若当前上下文屏蔽其 Tick，就可能永远不到期；HAL_MAX_DELAY 又可能绕过超时。DMA 启动返回 HAL_OK 只表明启动成功，最终结局在回调或错误路径。把 API 名字和返回码直接当完成证明，是驱动集成常见问题。

推演与反例

高层驱动隐藏了哪些状态

一次 HAL 接收启动不仅设置地址和长度，还可能检查句柄状态、锁、DMA 回调和错误标志。如果应用直接改寄存器而不更新句柄状态，硬件与软件认知会分离，下一次 HAL 调用可能返回 BUSY 或执行错误清理。混用不同层时要明确哪个模块拥有外设，以及如何保持两层状态同步。

优化要从具体路径开始

在微秒采样循环中，函数封装和 GPIO 读取路径可能影响判位；在等待 180 ms 传感器转换时，省几条配置指令却几乎没有收益。可把严格时序段写成经过测量的短操作，其余初始化和错误恢复保留可维护接口，并用逻辑分析仪验证真实波形。整体替换 HAL 前必须考虑版本、勘误和恢复路径。

继续追问

面试官要求不用 HAL 重写驱动，应先讲什么？

按时钟/引脚、寄存器配置、启动条件、数据传输、完成/错误及恢复顺序解释，先展示硬件状态理解，再讨论函数封装。

HAL 返回 OK 是否代表业务正确？

只代表该接口定义的操作成功，不代表传感器数值合理、数据新鲜或上层命令已完成。驱动结果还要经过格式、范围与业务检查。

依据与核对范围

ST 官方 stm32f1xx HAL UART 源码 — HAL_UART_IRQHandler / ReceiveToIdle 与驱动状态；项目结论以离线本地版本为准。

T071 原理问答

如果增加 Bootloader/OTA，怎样设计启动与回滚？

Bootloader/OTA 的核心是任何允许的掉电点之后仍有可运行、可验证的启动路径。下载完成、完整性检查、真实性验证、试运行确认和回滚应形成明确状态机。

原理与成立条件

Bootloader 与应用的职责

Bootloader 在启动时选择并验证可运行镜像，应用负责正常业务或下载候选镜像；芯片出厂系统 Bootloader 又是另一套固化能力。自定义升级需规划启动区、应用区、下载区与元数据，并限制擦写对当前执行、向量和中断的影响，不能仅用一个 memcpy 到 Flash 代替完整升级流程。

掉电安全依赖状态提交顺序

先接收候选镜像并校验完整性与真实性，写入可恢复元数据，再标记为待试运行；试运行通过自检才确认生效，否则回退旧镜像。A/B、外部暂存或交换方案各有 Flash 容量与擦写代价。升级任何一步掉电，重启后都应能确定仍有一个完整可启动版本。

CRC、签名和版本分别解决什么

CRC/哈希帮助检测损坏，签名结合可信公钥验证发布者及内容，版本/单调计数用于限制回滚到已撤销旧固件。只验证哈希而哈希值也来自同一不可信下载源，不能证明镜像可信。密钥、元数据原子更新与防回滚存储同样属于设计。

跳转应用前要清理环境

核对初始栈和入口地址，停止不应继承的 DMA、SysTick 和中断，按要求调整向量表与栈，再进入应用复位入口。应用不能意外继承下载期间的外设状态。具体跳转序列受内核、启动模型和编译约束影响；很多系统采用复位后由 Bootloader 选择镜像更易界定状态。

推演与反例

双镜像为何仍需提交标记

新固件写入备用区后，先校验长度、哈希及签名，再把它标为可试运行；首次启动若不能在限定条件下确认，Bootloader 回到旧镜像。状态标记本身也可能写到一半，需要原子写粒度、冗余或日志式更新。只保留两份二进制而没有可恢复元数据，仍可能不知道哪份完整、哪份已确认。

安全与可用性是两组验证

CRC/哈希发现损坏，签名验证发布者身份；防回滚还需要受保护的版本策略。应用即便签名正确，也可能启动后卡死，因此需试运行与恢复入口。Flash 页擦除、写入对齐、同 Bank 执行限制、向量表和外设状态都要纳入平台设计；空间不足时应明确外部暂存或独立恢复协议，不能承诺凭空双备份。

继续追问

只有一份应用存储空间，怎样保证升级掉电能恢复？

必须保留独立可运行的恢复入口，并有外部暂存、分块恢复协议或重新下载能力；若擦掉唯一应用又没有恢复通道，就无法仅靠软件标志承诺回滚。

为什么升级不能沿用普通 RAM 队列的可靠性说法？

升级跨复位和掉电，RAM 中的进度会丢失，必须依赖可恢复的持久状态及受保护启动代码；传输成功只是其中一步。

依据与核对范围

MCUboot 官方设计文档 — Image validation、Image trailers、Test images / image confirmation；用于解释掉电恢复和试运行，节点未实现 OTA。

ST AN2606: [System memory boot mode](#) — 启动模式背景；自定义 OTA 回滚是本文设计扩展，并非引用本手册宣称已实现。

外设、DMA 与传感器

T072 原理问答

UART 为什么无需时钟线，115200 能传多少有效数据？

UART 通过起始位让接收端对齐一帧，再按约定波特率采样数据与停止位。它没有随线传输的时钟，因此两端时钟误差、采样方式和电气质量都会影响可靠性。

原理与成立条件

UART 一字节在线上占多少位

异步 UART 用起始位建立采样相位，再传数据位、可选校验位和停止位，没有独立时钟线。8N1 是 1 起始+8 数据+1 停止，共 10 位；115200 baud 在这种二值编码下理论约 11520 字节/秒。应用有效负载还需扣帧头、校验、确认和空闲，不能把波特率直接当字节率。

波特率误差为何积累

接收方检测起始边沿后按本地时钟采样，各位没有独立同步。收发时钟偏差会让采样点逐渐偏离位中心，叠加抖动、噪声和边沿缓慢可能误码。允许误差与帧长、过采样实现和时序余量有关，不宜对所有 UART 一律背同一个百分比。

TXE 与 TC 的区别

TXE 表示数据寄存器可以接收下一个数据，移位寄存器可能还在发送；TC 才反映发送器按手册定义完成最后一帧位传输。DMA TC 又只是 DMA 搬运结束，最后字节可能刚进 UART。切 RS-485 方向或关闭外设必须选对线路完成事件。

电平与流控

MCU UART 通常是逻辑电平，不能直接与 RS-232 的电气电平等同；RS-485 需要收发器与方向控制。

RTS/CTS 可在兼容双方驱动下对接收能力施加硬件背压，软件协议也可用信用控制。仅有更大的环形缓冲不保证无限连续数据不会丢。

推演与反例

从波形推导有效吞吐

8N1 每个字节包含 1 个起始位、8 个数据位和 1 个停止位，共 10 bit。115200 bit/s 对应理想 11520 B/s，一个字节约 86.8 μ s，69 字节约 5.99 ms；协议头、CRC、确认、空闲间隔再降低业务吞吐。若增加校验位或停止位，分母也要改变，不能始终用波特率除以 8。

为什么不能背一个通用误差百分比

接收端检测起始沿后按自己的时钟取样，频差会在一帧内累积；允许误差取决于数据位数、过采样与判决方式、发送/接收时钟精度及边沿质量。具体器件的数据手册给出条件，不能把某个经验值外推到全部 UART。TTL UART、RS-232 和 RS-485 的电平与驱动也不同，名称带串口不代表能直接接线。

继续追问

69 字节在 115200、8N1 下最少多久？

忽略间隔和软件延迟， $69 \times 10 / 115200 \approx 5.99 \text{ ms}$ ；这是串行发送时间下界，不是完整命令往返时延。

UART 全双工是否表示应用可以随意并发写同一个发送口？

硬件收发可以独立，但多个软件发送者的字节仍可能交错。帧发送应由同一个所有者串行化，或用明确互斥和整帧队列保证顺序。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

T073 原理问答

DMA 解决什么问题，为什么还需要中断和缓冲？

DMA 由硬件在外设与内存之间搬运数据，减少 CPU 逐字节干预；CPU 仍负责配置、缓冲所有权、完成事件和错误恢复。DMA 提高搬运效率，不自动提供协议边界或无限容量。

原理与成立条件

普通、循环与双缓冲的差别

普通模式到计数结束停止；循环模式到末尾自动重装并从头覆盖；双缓冲/乒乓思想让硬件写一块、软件处理另一块，但是否硬件原生支持取决于 DMA 型号。循环不是自动安全队列，消费者必须赶在下一轮覆盖之前完成复制或处理。

所有权比传指针更重要

DMA 正在使用的 TX 缓冲必须保持不变且有效，RX 缓冲可能随时被硬件覆盖。真正零拷贝需要可证明的借用期限、可暂停的生产端或可归还块池；只把环形缓冲地址发给任务，任务稍晚处理就可能读到下一轮数据。取消、超时与重启时也要确定 DMA 已停止访问。

缓存一致性不能泛化

无数据缓存的 Cortex-M3 不存在某些高端 MCU 的 D-cache 维护问题；带非一致性缓存平台上，DMA 与 CPU 可能读到不同副本，需要按方向 clean/invalidate、对齐缓存行，并避免误丢同一行其他脏数据。内存屏障和缓存维护功能不同，不能用 volatile 替代。

推演与反例

一次 DMA 传输有哪些参数

请求源决定何时搬，源/目的地址与是否递增决定搬到哪里，数据宽度和计数决定每次及总量，普通/循环模式决定结束后是否重装。地址宽度不匹配可能破坏相邻内存，外设请求映射也因系列而异。DMA 会占用总线，因此即使 CPU 指令数减少，也要考虑仲裁对访问延迟的影响。

TX 的两个完成点必须区分

DMA 传输完成通常表示最后一个数据已交给 UART 数据寄存器，不表示最后一个停止位已经离开引脚。UART TC 才对应更后面的线路发送完成条件。若要切换半双工方向或关闭外设，必须等正确事件；缓冲释放最早能在哪个事件发生，也应按驱动是否还会访问它来证明，不能仅靠变量名 complete。

继续追问

DMA 完成中断里立即释放 UART TX 缓冲是否一定正确？

需看 DMA 是否已不再访问以及上层是否还需该帧重发；若要切总线方向，还必须等 UART TC，不能把两种完成事件混淆。

启用 DMA 后还能忽略中断吗？

只有明确采用轮询且轮询延迟有界时才可能；通常仍需要半传输、完成、IDLE 或错误通知来安排消费。搬运与消费是两条独立进度。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

ST 官方 stm32f1xx HAL UART 源码 — HAL_UART_IRQHandler / ReceiveToIdle 与驱动状态；项目结论以离线本地版本为准。

ST AN2548: DMA 控制器原理 — DMA 请求、总线争用与传输模式；具体通道和完成语义对照 RM0008 及本地 HAL。

循环 DMA 加 IDLE 怎样计算新数据，IDLE 是帧结束吗？

循环 DMA 把内存当环持续写，IDLE、半传输和完成事件只报告接收进展。软件根据上次处理位置与本次位置取新增区间，再把字节交给协议解析器；IDLE 不是通用帧结束标记。

原理与成立条件

由位置求增量

设缓冲大小 N ，上次处理位置 old ，本次位置 pos 。未回绕时处理 $[old, pos)$ ，回绕时处理 $[old, N)$ 与 $[0, pos)$ 。HAL 回调 $Size$ 常是本次有效数据终点，不是自上次回调新增字节数；具体含义要看 HAL 版本。把每次 $Size$ 都从数组 0 复制会重复处理旧数据。

相同位置存在歧义

$pos == old$ 可能是重复通知，也可能已经完整绕了一圈甚至多圈。只保留模位置无法还原丢失的圈数，必须保证服务足够及时或另外跟踪单调生产进度。 N 与 0 代表同一物理位置却可能表达不同边界，过早归一化可能在 TC/IDLE 相邻时重复搬运。

从 DMA 到任务的两级容量

ISR 把增量复制进软件流缓冲后，DMA 区可继续覆盖，但软件缓冲仍可能满。必须检查实际接受字节数并记录丢失；发生丢失时解析器需要重新同步，不能继续声称收到完整数据。若多个 ISR 可重入同一写者状态，还需串行化 old 的更新。

推演与反例

回绕区间怎样算才不会漏一段

缓冲长度 N ，上次位置 $old=50$ ，本次 $pos=10$ ，新增区间是 $[50, N)$ 加 $[0, 10)$ ；若 $pos \geq old$ ，则是 $[old, pos)$ 。但 $old == pos$ 有歧义：既可能没有新字节，也可能硬件已经绕过整圈。仅保留模 N 的位置无法知道发生过几圈，必须通过及时处理中断、累计代数或其他设计排除覆盖。

HAL 回调参数必须看接口契约

一些 Receive-to-idle HAL 回调的 $Size$ 表示当前数据位置，可能等于 N ，不是“本次新增长度”。把它直接当长度追加会重复旧数据；把 N 简单变成 0 又要防止相邻 TC/IDLE 通知对同一末尾产生重复处理。具体事件组合与回调次序需核对所用 HAL 版本，并测试半满、全满、回绕与紧邻 IDLE。

继续追问

把 DMA 缓冲从 64 改成 128，问题就结束了吗？

只是扩大覆盖前的时间预算；仍要检查回调位置、软件短写、最长服务间隔和持续输入，不能用容量替代时序证明。

能用“缓冲足够放一帧”证明安全接收吗？

不能。生产者可能连续发多帧，消费者也可能长时间得不到运行；容量证明要基于最坏消费停顿与到达速率，帧最大长度只是其中一个边界。

依据与核对范围

ST 官方 `stm32f1xx HAL UART 源码` — `HAL_UART_IRQHandler / ReceiveToIdle` 与驱动状态；项目结论以离线本地版本为准。

T075 原理问答

怎样计算 DMA 与软件缓冲容量，证明不会被覆盖？

缓冲能否不溢出，取决于到达数据减去已消费数据的最大积压。必须分别分析硬件 DMA 环和软件队列：前者可能被覆盖，后者可能拒绝或部分接收，扩大其中一个不自动修复另一个。

原理与成立条件

为什么半缓冲时间也重要

若软件依赖半满/全满事件搬运，回调发出时已有一半数据待处理。等到它被延迟到下一次覆盖时，那块数据已经不安全。需要分析最早未处理字节距离硬件下一次写回还有多久，不能总拿整个 N/r 当每次 ISR 的可用延迟。

CPU 关中断与硬件接收不同步

临界区延迟 DMA/USART 回调，DMA 仍可能持续写 RAM。等 CPU 恢复后，位置看似合法，但旧圈数据已经消失；CRC 只能发现部分破坏，不能恢复丢掉的命令。消费者任务同样需要及时排空软件缓冲，两个容量不能简单相加成一个保证窗口。

从分析到方案

测最大不可服务区及实际输入突发，再选择缩短临界区、定时器捕获传感时序、增大缓冲或加字节级流控。帧级窗口限制只约束命令数量，不必然限制瞬间字节突发。零拷贝如果延长消费者持有时间，反而可能使覆盖约束更难满足。

推演与反例

把中断屏蔽时间换算成字节

115200、8N1 时约每 86.8 μs 来 1 字节。64 字节环一圈约 5.56 ms，半环 32 字节约 2.78 ms。若处理每半环一次，回调被延迟几毫秒就可能接近下一次覆盖；准确余量还取决于进入临界区时 DMA 已写到哪里。以上是理想满速输入的预算示例，不是测得的中断上限。

软件缓冲还要覆盖慢消费者

即使 ISR 及时把数据复制进软件缓冲，命令任务在等待传感器转换期间仍可能不消费。容量需要覆盖这段等待中的最大输入，或者由窗口/信用限制发送方。若发送 API 只接收部分字节，必须记录实际返回长度并恢复分帧，不能更新读书签后假定全部交接成功。单纯增大缓冲只能吸收有限突发，长期输入快于消费仍会满。

容量推导：先把 bit/s 转成 byte/s

```
115200 bit/s, 8N1: 每字节 1+8+1 = 10 bit
连续字节速率 r = 115200 / 10 = 11520 byte/s
64 byte 空环填满时间 = 64 / r  $\approx$  5.56 ms
32 byte 半环间隔 = 32 / r  $\approx$  2.78 ms
```

所需可用容量 $\geq$ 已有积压 + 停顿期间最大到达量

这是连续流数量级。最坏证明还需考虑 ISR 延迟、软件队列剩余容量、突发、环绕次数与恢复消费速率；DMA 环和软件流缓冲不能无条件把容量简单相加。

继续追问

69 B 帧超过 64 B DMA 为什么有时正常？

ISR 和任务可在整帧到完前处理前半段，所以单帧不必全部同时驻留；出现长临界区或突发时，时序条件不满足才暴露丢失。

指针队列是不是自然的零拷贝优化？

不是。DMA 下一圈会重写原地址，指针仍有效但内容已变。需要可转移所有权的块池/双缓冲、回收协议与流控，先证明数据寿命，再比较复制收益。

依据与核对范围

FreeRTOS: Stream / message buffers — 单写者单读者与字节流语义。

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

ST AN2548: DMA 控制器原理 — DMA 请求、总线争用与传输模式；具体通道和完成语义对照 RM0008 及本地 HAL。

UART 的 ORE、FE、NE 出错后，为什么需要恢复 DMA？

ORE 表示接收数据未及时取走导致覆盖风险，FE 与帧格式/停止位有关，NE 与噪声检测有关。恢复不仅要清错误位，还要恢复 UART、DMA、驱动句柄和软件书签的一致状态。

原理与成立条件

UART 错误要区分原因

FE 常指停止位采样不符合格式，NE 反映噪声检测，ORE 表示前一数据未及时转移而新数据到达。波特率、电平、接线与服务不及时都可能参与。软件队列溢出和 DMA 环覆盖可能没有对应 UART 硬件错误标志，因此只统计 ORE 不能涵盖全部丢字节。

清标志与恢复接收是两步

按芯片手册的序列清错误后，还要确认 DMA 请求、相关中断和 HAL 状态是否仍启用。某些 HAL 版本把 DMA 接收错误视为需要停止当前接收，回调后若不重挂，任务仍运行但数据再也不进来。错误处理必须对照实际版本状态机，不能从别的系列复制几行清标志代码。

恢复要同步软件位置

DMA 从头重启时，软件 `old_pos`、剩余长度和解析残帧必须有一致的处理规则。保留旧位置会重复或跳字节；损坏半帧应让解析器重新同步。重挂 API 可能返回 `BUSY/ERROR`，应有有限重试或任务级恢复，不能在 ISR 无限循环。

把诊断分层保存

记录硬件错误类型、DMA 重启次数及失败、软件短写、CRC 错误与命令超时，再关联时间。一个总错误计数无法区分物理链路差与任务调度慢。恢复成功也不能抹掉证据，否则故障表现为偶发变慢却没有根因信息。

推演与反例

为什么“清了错误标志”仍收不到

驱动错误路径可能已关闭 DMA 请求、IDLE 中断并把接收状态退回 `READY`。应用只清一个寄存器位而不重新启动接收，链路就会保持静默。反过来，直接重启 DMA 但保留旧位置，也可能把重装后的新数据与旧半帧拼接。F1 的 `SR/DR` 清除顺序及 HAL 错误处理细节要按对应版本确认。

有限恢复要保留可诊断结果

可计数错误、清理不可信的接收残段并尝试重启，记录重启是否成功；连续失败时交由任务以退避和期限处理。ISR 中无限恢复循环会阻塞整个系统。错误期间已经丢失的字节不能被重新开 DMA 找回来，只能靠协议重同步和上层重传弥补。

继续追问

命令任务还在签到，为什么 **watchdog** 未发现接收坏了？

任务活性不等于外设活性；需要检查最近 **RX** 进度、恢复结果及业务反馈，不能只看循环是否继续执行。

看到 **ORE** 就能断定 **CPU** 太慢吗？

不能。还可能是长关中断、**DMA** 未启动或映射错误、优先级配置、重启窗口和驱动状态异常。应结合波形、**DMA** 计数器、错误发生时刻与消费者进度定位。

依据与核对范围

ST 官方 **stm32f1xx HAL UART** 源码 — **HAL_UART_IRQHandler / ReceiveToIdle** 与驱动状态；项目结论以离线本地版本为准。

ST RM0008: **STM32F1 参考手册** — **RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG** 等章节；配置同时核对本地 **HAL**。

T077 原理问答

I2C 为什么必须开漏，上拉电阻怎么影响时序？

常见 **I2C** 使用开漏/开集输出和上拉，实现多个设备只能拉低、释放后回高的线与行为。这支持 **ACK**、仲裁与时钟拉伸，也使上升沿由电阻和总线电容决定。

原理与成立条件

开漏如何形成共享总线

I2C 的 **SDA/SCL** 通常由开漏输出配上拉构成，参与者可拉低或释放；任意一方拉低时总线为低。这样允许 **ACK**、时钟拉伸和多主仲裁，不会像两个推挽输出分别驱高/低那样直接对抗。释放不等于主动驱高，实际高电平取决于上拉、电容及是否还有设备拉低。

拉伸与仲裁

支持拉伸的从机可把 **SCL** 继续拉低，主机应观察实际电平而非只写“释放 **SCL**”；多主仲裁通过发送同时读取 **SDA** 实现，发送高却读低者在相应条件下失去仲裁。并非所有模式、控制器和设备都支持全部能力，需遵从双方速率及接口约束。

推演与反例

上拉电阻为什么既不能太大也不能太小

电阻太大，RC 充电使上升沿变慢，采样时可能还未达到高电平门限；太小则低电平灌电流过大。常用估算 $R_{\max} \approx t_r / (0.8473 \times C_b)$ ， $R_{\min} \approx (V_{DD} - V_{OL\max}) / I_{OL}$ ，要使用器件允许的电流、总线模式上升时间和实际电容。布线、连接器及探头都会增加电容，不能只照抄一个 4.7 kΩ。

数据什么时候允许变化

普通数据位在 SCL 高电平期间保持稳定，SCL 低时改变；SCL 高时 SDA 的下降/上升分别形成 START/STOP。每 8 位后还有第 9 个 ACK 时钟，由接收方决定应答。某些器件可把 SCL 拉低延长等待，因此主机释放 SCL 后还需按系统能力确认它真的变高，输出寄存器写高不等于线上已经高。

继续追问

GPIO 配了内部上拉，就一定可以不放外部电阻吗？

不一定。内部上拉常较弱且离散，可能不满足总线电容与速度要求；要按数据手册和实测上升时间判断。

I2C 可以用推挽主动输出高吗？

普通共享 I2C 的这些信号阶段不应这样做，否则另一器件拉低时形成电气争用，破坏仲裁与拉伸。特殊模式须按专门规范设计，不能套进普通设备连接。

依据与核对范围

NXP UM10204: I2C-bus specification — §3.1 信号、ACK/时钟拉伸/总线清理，§7 上拉计算。

T078 原理问答

I2C 七位地址如何编码，传感器转换与读取怎样配合？

I2C 七位地址是设备身份，线上第一个地址字节通常由地址左移一位再附加读写位形成。驱动 API 可能要求七位值，也可能要求已移位值；正确性取决于接口契约，不取决于哪一种写法更像地址。

原理与成立条件

7 位地址与地址字节

I2C 常用 7 位从地址，在线上传输时左移一位并加 R/W 位，形成首个地址字节。驱动 API 接受未移位地址还是已移位值由接口规定；把 0x23、0x46、0x47 混称同一个地址却不说明层次，是常见 NACK 原因。分析仪往往显示 7 位地址，应核对显示选项。

转换与读出是不同阶段

数字传感器接到测量命令后，需要积分/转换时间，再提供寄存器结果。I2C 传输很快不代表新的物理测量已完成；过早读取可能取得旧数据。连续模式与单次模式的启动、更新和功耗语义不同，重复下发模式也会影响结果新鲜度，应按芯片手册处理。

BH1750 数值换算的条件

默认测量时间设置下，常规高分辨率模式读取高字节在前的 16 位原始码，典型换算 $\text{lux}=\text{raw}/1.2$ 。高分辨率 mode2 或修改 MTreg 会改变比例与时间，不能始终套这个除数。数据手册的分辨率、精度、典型时间和最大时间含义不同，整数输出也不意味着误差只有 1 lx。

驱动应输出数据与有效性

读取成功、量程溢出、转换未完成和总线错误应分别表达；不要把错误原始码直接换算后当有效光照。把浮点结果压到 u16 时需定义舍入和饱和，不能依赖越界转换。上层应知道采样时间和有效状态，才能正确展示与报警。

推演与反例

用 0x23 检查线上字节

七位地址 0x23 的写地址字节为 0x46，读地址字节为 0x47。若 HAL 要求传入左移后的地址而应用传 0x23，或库已经左移而应用又左移一次，就会访问错误地址。抓波形时还要确认分析仪显示的是七位地址还是完整地址字节，避免把正常 0x46 误判为地址配置变了。

数值读取前为何要等转换

命令发送成功只说明器件收到了配置，不表示新测量已完成。以默认测量时间寄存器的 BH1750 高分辨率模式为例，典型/最大转换时间和 raw 到 lux 系数都由手册规定；默认常用 $\text{raw}/1.2$ ，修改测量时间后换算也应调整。等待可以让任务阻塞释放 CPU，但若同时持有传感器锁，其他查询仍会被该转换时长挡住。

继续追问

读到 0 lx 是否说明传感器坏了？

可能是真实黑暗，也可能未转换、配置错误或错误被当成零。需要结合事务返回、模式、转换等待和有效标志判断。

连续测量模式下每次重发模式命令一定必要吗？

未必。要按器件行为判断是否重启测量或改变新鲜度；可设计缓存与定期更新，但不能把收到两个字节就等同于获得刚发生的测量。

依据与核对范围

ROHM BH1750FVI 原厂手册（DFRobot 镜像） — 原厂 PDF，电气特性表及 I2C Bus Access / measurement adjustment; 120 ms 典型、180 ms 最大值与默认系数。镜像为较早修订，实物版本应核对。

T079 原理问答

SDA 卡低时为什么发九个时钟，哪些故障救不了？

SDA 卡低时发送最多九个 SCL 脉冲，是试图让停在一个字节中间的从机完成剩余位并释放总线。它是有前提的总线清理，不是能修复所有 I2C 故障的通用复位。

原理与成立条件

SDA 卡低为何可能恢复

从机可能停在发送某一位或等待时钟，主机复位后丢失事务状态。主机释放 SDA，提供最多 9 个适当时钟，可从从机推进到字节/ACK 边界并释放线路，再尝试生成 STOP。这针对协议状态卡住，不会修复短路、供电异常或损坏设备。

SCL 卡低不能靠写高解决

开漏主机写高只是释放，若从机持续拉低 SCL，时钟并未真正产生。应读取实际 SCL 并设置有限等待；确认硬件不支持恢复或超过期限时，可按设备方案复位/断电恢复。不能因为循环执行了 9 次 GPIO 翻转就宣称总线已解锁。

恢复时必须独占总线

停止当前 I2C 事务与相关 DMA/中断，防止其他任务同时访问，再将引脚切到合适 GPIO 模式执行恢复。STOP 是在 SCL 确实高时释放 SDA；恢复后重建外设配置，并重新初始化可能丢状态的传感器。应校验总线空闲与每个初始化返回值。

控制重试避免恢复风暴

一次普通读失败后可进行有限恢复和重试，连续失败向上报告并退避。每次调用都无期限尝试，会让一个坏设备长期霸占总线 and 互斥量。保留首次错误与恢复结局，便于区分时序问题、设备复位和物理故障。

推演与反例

为什么是九个而不是无限个

一个常见传输单元是 8 数据位加 1 ACK 位，主机复位可能让从机仍停留在其中某一位。释放 SDA、产生时钟并观察线路，可以推进残留状态，再构造 STOP 结束事务。前提是 SCL 能被拉高、从机仍按时钟工作；若短路、供电异常或器件永久拉低，继续打时钟并不能解决。

硬件恢复与软件恢复要一起做

暂时停用外设、把引脚切到合适开漏模式、进行有界清理、恢复复用与外设配置，然后让应用知道本次事务失败并决定是否重试。不能保留旧接收数据并标成有效，也不能把恢复时的 **GPIO** 操作与另一个任务的正常 **I2C** 事务并发执行。涉及多个控制器时还要避免把合法传输误判为卡死。

继续追问

为什么不在任何 **NACK** 后都立刻发送 9 个时钟？

NACK 也可能是地址错、设备忙或正常的结束语义。应先识别错误类型和总线状态，避免打断本可正确结束的事务。

SCL 本身一直低时怎么办？

先区分合法拉伸与超过期限的异常；若无法释放，应按器件能力使用硬件复位、断电恢复或报告故障。规范中的九时钟方案主要针对 **SDA** 卡低且 **SCL** 可控制的情形。

依据与核对范围

NXP UM10204: *I2C-bus specification* — §3.1 信号、ACK/时钟拉伸/总线清理，§7 上拉计算。

ST RM0008: *STM32F1 参考手册* — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

T080 原理问答

DHT11 如何区分 0 和 1，为什么不能用普通毫秒延时？

DHT11 用高电平持续时间区分 0 和 1，必须测量或采样微秒级波形。毫秒任务延时无法定位判位窗口；同时每段等边沿都要有界，防止断线把 CPU 永久困住。

原理与成立条件

DHT11 怎样编码数据

主机先按规格拉低启动，再释放线路等待从机响应，随后接收 40 位数据。每位低电平作为分隔，高电平持续长度区分 0/1，最后一字节用于校验前面数据。不能像 **UART** 一样只给出固定波特率，正确解码依赖微秒级边沿与脉宽判断，以及匹配实际传感器版本的数据格式。

为什么采用超时边沿等待

如果传感器未接、线被拉住或时序错，等待某个电平的 **while** 可能永久不退出。每个阶段应有合理上限，失败后释放总线并返回状态；校验正确后还要检查小数位与范围。总超时由多阶段上限累计，不能只看某一个等待的 100 μs 。

忙等与关中断的取舍

DWT 可按 CPU 周期做短延时，但中断会拉长软件采样时刻；关中断能减少一类抖动，却伤害全系统响应，且更高紧急度异常未必被屏蔽。更稳健的高负载方案可用定时器输入捕获记录边沿或 DMA 搬捕获值，代价是额外资源和解码复杂度。

传感器限制不能被软件绕开

V1.3 要求读取间隔大于 2 s，上电后先等稳定；读到的还是上一轮测量结果，频繁读不能无限提高新鲜度。该版湿度小数注明为 0，温度可有小数，温度低字节最高位另作负号。其他实物/修订应分别核对，不能把协议小数字段的存在等同传感器真实精度，或统一宣称 DHT11 只能报整数。

推演与反例

从两个时间区间推导采样点

所引奥松 DHT11 V1.3 手册中，0 的高电平为 23–27 μs 、1 为 68–74 μs ，选择上升沿后约 40 μs 采样，可在理想条件下区分二者。真正余量还要扣除边沿检测延迟、函数执行、中断抢占和时钟误差。若采样点被推到 70 μs ，长脉冲也可能已回低，算法便失效。应以实物版本波形和手册确认，而不是只背“0 是 26、1 是 70”。

校验成功仍不等于数据新鲜且可表示

五字节数据含温湿度字段与累加校验；V1.3 对读取间隔、上电稳定和上一轮测量值有说明，温度低字节还带负号约定。解析应检查版本、符号、小数格式及目标数值范围。一个无符号温度协议不能靠强转发送负温；应修改格式或显式报告不支持，不能悄悄变成很大的正数。

继续追问

为何不能把 18 ms 启动低电平和位采样全部关中断？

启动阶段通常不需要微秒级 CPU 独占，把毫秒等待一起屏蔽会显著扩大串口与 Tick 延迟。应分别按时序严格程度选择等待方式。

关中断采样能否彻底消除时序问题？

只能屏蔽指定范围的抢占，还受更紧急异常、代码执行和电气波形影响，并把延迟转移给 UART、Tick 等路径。更高负载时可考虑定时器输入捕获等硬件计时方案。

依据与核对范围

奥松 DHT11 V1.3 产品手册 — V1.3_20170331：表 4/图 3–6 时序；S6 读取间隔与前次测量值；S7 温度小数及负号、湿度小数位为 0。实物版本应该核对。

CAN 的仲裁、确认、错误状态与位时序如何工作？

经典 CAN 是按标识符竞争访问的广播总线。控制器处理帧与仲裁，收发器负责差分物理信号；显性位覆盖隐性位，使优先级较低的发送者能退出而不破坏获胜帧。

原理与成立条件

CAN 控制器与收发器的分工

经典 CAN 是多主广播总线，控制器处理位时序、帧、仲裁、校验与错误状态，收发器把逻辑电平转换为 CANH/CANL 差分信号。标识符描述消息并参与优先级，不天然等同节点地址。标准帧 11 位 ID、扩展帧 29 位 ID，经典 CAN 数据字段最多 8 字节；CAN FD 的能力不能套到传统 bxCAN。

非破坏仲裁为什么成立

显性 0 能覆盖隐性 1，节点发送时同时监视总线；在仲裁字段发送隐性却读到显性的节点退出发送，获胜者继续，报文不必被所有竞争者同时破坏。相同帧格式下较小 ID 通常更优先；标准/扩展竞争还涉及 SRR/IDE 等位，不能只把两个不同长度 ID 当整数比较。

ACK 与故障限制

正确接收的节点可在 ACK 槽给出显性确认，它不表示目标应用已经消费或执行。CRC、位监视、填充等机制检测错误，错误计数驱动 error-active、error-passive 和 bus-off 等状态。恢复策略需要诊断物理层与波特率，不能无限自动重连而掩盖持续故障。

位时序与拓扑为何重要

波特率由时钟、预分频和时间段组成，采样点与重同步跳宽影响对传播延迟和时钟误差的容忍。高速 CAN 常在总线两端各接匹配终端电阻，支线过长、星形拓扑和接地问题可造成反射/共模困难。只让两个节点“设置同样波特率”不足以保证所有长度和负载下稳定。

推演与反例

按位仲裁怎样保证非破坏

两个节点同时发，都会读回总线；发送隐性 1 却读到显性 0 的节点，在仲裁字段退出发送，剩余节点继续。相同帧格式下较小 ID 通常更优先，但标准帧与扩展帧混合时还涉及 IDE/SRR 等位，不能直接把两个不同长度的 ID 当普通整数比较。仲裁优先级也不保证低优先级帧不会长期等待，高负载下仍需总线调度预算。

ACK、重传与 bus-off 的关系

正常接收并验证帧的节点可在 ACK 槽给出显性确认，这不表示目标应用已消费。物理错误、位错误和缺少 ACK 等会影响错误计数，控制器可从 error-active 转到 error-passive，严重时进入 bus-off。恢复应检查终端电阻、波特率、采样点、时钟与接地；自动重传不停并不等于业务仍在有效运行。

继续追问

一个 **CAN** 节点单独发送却不断重发，可能为什么？

正常发送模式下没有其他正常接收节点提供 **ACK** 会产生 **ACK** 错误；应检查是否需要环回测试、另一节点、终端、位时序及控制器错误状态，不能只怀疑发送函数。

波特率一样为什么还可能通信失败？

位时间还包含不同时间段、采样点和重同步宽度，线长与传播延迟也参与容差。仅比较标称 **bit/s**，不足以证明两端时序兼容。

依据与核对范围

TI SLOA101B: **CAN** 总线原理 — §4.1 仲裁、§4.4 故障限制、§5 物理总线；用于经典 **CAN**，不套用 **CAN FD** 的数据长度。

CiA: **CAN** 数据链路层 — 经典 **CAN** 的消息、仲裁和错误处理；**CAN** 控制器与应用层完成边界分开。

T082 原理问答

SPI 四种模式怎么判断，为什么错一个边沿会读错？

SPI 的 **CPOL** 定义时钟空闲电平，**CPHA** 定义在第一个还是第二个有效边沿采样。要从器件时序图确认数据建立/保持与片选范围，不能只记四个模式编号。

原理与成立条件

SPI 的四根线与传输方式

常见 **SPI** 由主机提供 **SCLK**，通过 **MOSI/MISO** 同步移位，片选区分从机与事务。全双工模式下每移出一位也移入一位，因此只想读寄存器时，主机往往仍需发送命令、地址和 **dummy** 字节来产生时钟。**SPI** 没有统一规定所有器件的地址、**ACK** 和帧格式，必须读从机协议。

CPOL 与 CPHA 怎样判断

CPOL 决定时钟空闲高低；**CPHA=0** 在相对空闲状态的第一个边沿采样，**CPHA=1** 在第二个边沿采样，另一个边沿通常用于移出/改变数据。先用“第一/第二边沿”分析，再按 **CPOL** 转成上升/下降，避免死背四种模式时配错采样点。

片选与频率不只是配置位

从机可能要求片选在命令、地址和全部数据期间持续有效，或要求事务间最小高电平时间。时钟上限受建立/保持时间、主从输出延迟、走线和电平转换器限制，**GPIO speed** 配得最高不能替代时序校核。多从机 **MISO** 应按要求在未选中时释放。

DMA 发送完成后的收尾

DMA TC 表示数据搬运结束，移位器可能仍忙；撤片选前应按具体 SPI 外设等待真正完成并处理接收数据/溢出。全双工只发不读可能积累 RX 状态错误。不同 STM32 系列的 FIFO、BSY 和 EOT 语义不同，不能把某一型号收尾顺序当通用代码。

推演与反例

用一对边沿解释模式

CPHA=0 时，接收方在第一个边沿采样，发送方必须在该边沿之前准备首位，之后在另一个边沿改变数据；CPHA=1 时通常在第一个边沿移出数据、第二个边沿采样。CPOL 决定这两个边沿分别是上升还是下降。双方错配时可能读到相邻位或边沿上的不稳定电平，低速偶尔可用也不是正确配置的证明。

全双工移位为什么也要发 dummy

SPI 每个时钟同时推进发送与接收，即使业务只想读取，也常需发送占位字节产生时钟。命令、地址、dummy cycle 和结果之间的片选保持由器件协议规定。DMA 搬完不一定等价于 SPI 最后一位结束，拉高 CS 前还要按外设完成条件处理，尤其最后一次读写和 FIFO 排空。

继续追问

SPI 第一个字节总是错，其余正常，先查什么？

检查 CPHA、片选到首个时钟的建立时间、首字节是否及时写入、dummy 数量及从机命令格式；用波形对照采样边沿，比盲目降低频率更容易定位。

SPI 有 ACK 和地址吗？

基础 SPI 没有统一的 I2C 式地址与 ACK，通常由片选和设备自身命令协议定义。错误检测、超时与多设备管理要由上层补充。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

T083 原理问答

ADC 分辨率、采样时间、DMA 与滤波怎么一起分析？

ADC 位数描述量化等级，精度还受参考电压、输入驱动、采样时间、噪声和校准影响。DMA 只搬运结果，不能让一个尚未稳定的采样电压变准确。

原理与成立条件

分辨率不等于精度

N 位 ADC 有 2^N 个量化等级，理想步长约 $V_{ref}/2^N$ ；实际还受偏置、增益、非线性、参考噪声和输入噪声影响。用 $V_{ref}/(2^N-1)$ 做端点线性标定是另一种工程换算口径，不能据此混淆理想量化步长。12 位输出不意味着每次都有 12 位有效精度。

采样时间为何受源阻抗影响

采样阶段通过开关给内部采样电容充电，信号源与开关等效电阻形成 RC。阻抗高或采样时间短，电容不能充分接近输入电压，转换结果产生误差。可延长采样、降低源阻抗或加合适缓冲，但滤波、电容与运放稳定性都需一起设计。

采样率、混叠与噪声

信号频率超过采样系统可辨识范围会折叠到低频，采样前的模拟抗混叠滤波不能由采样后的平均完全替代。多次平均可降低某些不相关随机噪声，却不能修复偏置、参考误差或确定性干扰。过采样增加有效位需要噪声/抖动及带宽等前提，不是多读几次就必然增加精度。

定时器触发加 DMA 的边界

硬件触发让采样时刻更稳定，DMA 降低搬运开销；通道扫描仍常按序采样，不代表多通道同时采样。配置时计算采样+转换周期、序列长度与触发频率，防止来不及完成。缓冲半满/全满通知后还要保证消费者及时处理，并校准参考与输入范围。

推演与反例

为什么高源阻抗需要更长采样

采样开关闭合后，内部采样电容通过信号源阻抗和开关电阻充电，电压以 RC 规律接近真实输入。采样结束太早，剩余误差直接进入转换结果；高分辨率要求更小的剩余误差，通常更需要充分建立时间。可延长采样、降低源阻抗或加合适缓冲，但要同时考虑运放稳定性、输入范围与滤波器。

用误差预算区分滤波与校准

N 位 ADC 理想量化步长约为 $V_{ref}/2^N$ ，按端点做工程换算可能用 2^N-1 ，这不改变量化器本身的等级。多次平均可降低某些不相关噪声，不能修复偏置、增益误差、参考漂移或已混叠信号。模拟抗混叠必须发生在采样前；定时器触发减少采样时刻抖动，但多通道扫描仍可能是依次采样。

继续追问

测量值随 CPU 负载变化，应该只改软件平均次数吗？

先查电源/参考耦合、模拟地、采样时序、源阻抗和 DMA/缓存处理，再判断平均能处理哪类随机噪声；稳定偏差需要校准或硬件整改。

DMA 开了后采样率还受什么限制？

采样加转换的总周期、序列通道数、触发频率、ADC 时钟范围及消费者处理能力仍有限制。必须证明上一轮结束且缓冲未被覆盖，再谈吞吐提升。

依据与核对范围

ST AN2834: ADC 精度 — Rev 10，采样电容、源阻抗、参考电压与噪声；适用参数按具体芯片数据手册。

ST STM32F103x8/xB 数据手册 — 存储规格、时钟与电气参数范围。

FreeRTOS 内核与任务通信

T084 原理问答

FreeRTOS 的任务状态和调度链路如何理解？

调度器只在就绪任务中选运行者，阻塞任务要等事件或时间条件满足后才回到 **Ready**。任务状态、优先级、等待对象和栈现场共同决定下一次何时能继续执行。

原理与成立条件

内核如何维护状态

TCB 保存栈顶、优先级和链表节点等信息。不同优先级有就绪结构，延时任务按唤醒时间组织，等待队列事件的任务还参与相应事件等待结构。事件到来或 **Tick** 到期使任务转回 **Ready**，再决定是否需要抢占。这里多个链表反映不同索引，不意味着任务同时在运行与阻塞。

阻塞为什么比轮询合适

等待队列或通知时任务退出就绪竞争，让低优先级任务及 **Idle** 获得运行。高优先级忙循环即使不断调用 **taskYIELD**，也通常只让同级就绪任务轮换，不保证低优先级得到 **CPU**。想等待数据应进入阻塞，而不是期待 **yield** 自动实现公平。

删除和挂起的陷阱

任务不应从入口函数普通返回；删除时要先归还应用资源、解除回调和锁依赖。自删除任务的内存回收还可能依赖 **Idle**，**Idle** 长期饥饿会影响回收。**suspend/resume** 也不是可靠事件计数器：先 **resume** 后 **suspend** 的竞争可能让事件丢失，通知或队列更适合事件同步。

推演与反例

从队列等待看内核链表

任务读空队列并允许等待时，需要记录它正在等哪个对象，以及最长等到什么时候。因此任务可同时参与事件等待结构与延时索引，但逻辑状态仍是 **Blocked**。数据先到时从相关等待结构移出并就绪；期限先到时解除等待并返回超时。两个来源竞争时，内核保护必须保证任务不会被重复插入或留在失效链表中。

Idle 不只是“什么都不做”

当没有更高优先级就绪任务时 **Idle** 获得 **CPU**，可处理自删除任务的资源回收并参与 **Tickless**。若高优先级任务持续 **Ready**，哪怕反复 **yield**，也可能让 **Idle** 饥饿，引起回收滞后或无法休眠。阻塞与删除都会改变调度状态，但释放应用锁、关闭驱动和取消回调并非由任务删除自动完成。

继续追问

高优先级任务延时 1 Tick 能保证低优先级充分运行吗？

只释放很短且相位相关的时间，低优先级是否满足期限仍需预算；更好的设计通常是按实际事件阻塞，并给 CPU 工作设明确负载上限。

用 **suspend/resume** 做一次事件通知为什么可能丢？

resume 发生时目标尚未 **suspend**，通常不会为将来保存一次唤醒；目标之后再 **suspend** 就可能一直等待。应使用有状态的任务通知、信号量或队列。

依据与核对范围

FreeRTOS: **Tasks** — 任务状态与调度。

FreeRTOS 官方内核书: **Task Management** — 第 4 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 **main** 完全相同。

T085 原理问答

SysTick、PendSV、SVC 为什么分工？

SysTick 推进内核时间，PendSV 执行延后的任务上下文切换，SVC 在典型 Cortex-M3 移植中启动首任务。这种分工把短异常处理与真正切换放到合适的异常优先级上。

原理与成立条件

PendSV 为什么放最低紧急度

PendSV 是可挂起的系统异常，用来把切换延后到更紧急异常完成后。这样 ISR 先完成硬件处理，再统一切换任务，避免在任意中断深处执行完整上下文切换。最低紧急度是异常优先级配置，不是任务优先级 0 的含义。

一次切换的具体顺序

硬件异常入口保存基本帧；PendSV 读取当前 PSP，将 R4-R11 等软件现场压入任务栈，保存新栈顶到当前 TCB。在适当内核保护下选择下一任务，从其 TCB 取栈顶，恢复软件现场、PSP，最后按异常返回机制恢复基本帧并继续新任务。必须使用匹配该架构的移植层。

SVC 怎样启动第一任务

调度器为新任务预先构造类似异常返回的初始栈帧，含入口、参数和状态。该 ARM_CM3 移植通过 SVC 恢复首任务上下文，之后才进入普通调度生命周期。SVC 不是只在所有 RTOS 中用于同一用途，其他移植或带 MPU 的系统可能还用它完成特权服务。

推演与反例

一次 ISR 唤醒高优先级任务的完整链

外设 ISR 把数据放入内核对象，使等待任务进入 Ready；如果新任务优先级高于被中断任务，FromISR 接口返回需要切换的标志，退出前请求 PendSV。更紧急 ISR 处理结束后，PendSV 保存原任务剩余寄存器与栈顶，选择并恢复新任务，异常返回后新任务继续。无需等下一个 SysTick 才有机会运行。

为什么把 PendSV 放低紧急度

若在嵌套中断深处随意切换，难以把线程现场和当前 Handler 的调用现场正确分离；最低紧急度使它通常在其他异常处理完毕后统一切换。硬件尾链等机制可减少异常切换开销，但具体周期还受存储等待与实现影响，不应背一个固定纳秒数作为所有 MCU 的结果。

继续追问

ISR 把高优先级任务置就绪后，还需要等下一次 SysTick 吗？

正确调用 portYIELD_FROM_ISR 请求切换时，通常可在异常退出路径经 PendSV 切换，不必多等一整 Tick；未请求时则可能增加响应延迟。

每次 Tick 都发生一次任务切换吗？

不一定。可能没有更合适的就绪任务，同优先级时间片也受配置影响；反过来，任务阻塞、让出或 ISR 唤醒都可在 Tick 之外触发切换。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态；Rev 7。

FreeRTOS: Tasks — 任务状态与调度。

FreeRTOS 官方 ARM_CM3 移植源码 — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep；上游参考，当前行为以本地快照为准。

T086 原理问答

FromISR API 与普通 API 有什么本质差异？

FromISR API 为中断上下文提供不进入任务阻塞等待的内核操作，并把是否应切换任务交给调用者处理。普通 API 的等待参数写 0，不会自动改变其合法调用上下文。

原理与成立条件

woken 参数表达什么

调用前把 `xHigherPriorityTaskWoken` 初始化为 `pdFALSE`，API 若唤醒更高优先级任务可将其置真；同一 ISR 多次调用应累计这个结果，不要中途重置。最后通过 `portYIELD_FROM_ISR` 告知移植层需要尽快调度，真正切换通常由 `PendSV` 完成，避免在数据结构尚未处理完时随意换上下文。

FromISR 也有优先级限制

在采用 `BASEPRI` 保护的 Cortex-M3 移植中，调用者必须处于 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 允许的紧急度范围。更紧急的中断不受内核临界区屏蔽，不能进入内核数据结构。函数名包含 `FromISR` 并不等于可从 `NMI`、`HardFault` 或任意高紧急度中断调用。

入队失败仍需应用处理

`FromISR` 不阻塞，缓冲满可能直接失败或只接收部分字节，具体看 API。保存返回值并计数，必要时通知任务执行恢复；不能在 ISR 无限重试等待消费者，因为被唤醒的任务尚未运行。中断前段只保存足够数据并交接，复杂重试应放任务上下文。

推演与反例

woken 为什么要累计

进入 ISR 时把标志初始化为 `false`，多次发送/通知中的任一次唤醒更高优先级任务，都应保留这个结果，最后统一请求切换。若每次调用前又清零，后一次没有唤醒新任务就可能覆盖前一次需要切换的事实。API 是否成功交付数据与是否需要立即调度是两种返回信息，都要按各自语义处理。

名称包含 ISR 也不是任意优先级可用

`BASEPRI` 保护内核临界区时，只屏蔽配置范围内的中断。更紧急中断若调用 `FromISR`，就可能进入尚在更新的队列或链表，因此被禁止。逻辑优先级 `5..15` 可调用而 `0..4` 不可调用，是“4 位优先级、阈值为 5、分组符合要求”这一示例下的结果，不是所有芯片的固定表。

四个优先级位时，逻辑值与寄存器编码

CMSIS 逻辑优先级: 0, 1, ..., 15

写入优先级字节: 逻辑值 $\ll (8 - 4)$

逻辑 5 $\rightarrow$ 0x50; 逻辑 6 $\rightarrow$ 0x60

`BASEPRI = 0x50` 时:

逻辑 0..4 仍可抢占这类内核临界区

逻辑 5..15 的相应异常被屏蔽

能越过内核保护的 ISR 不得调用内核 `FromISR` API。

假定使用与 FreeRTOS Cortex-M3 移植兼容的优先级分组。任务优先级的数值越大越优先，与这里的 NVIC 方向不同。

继续追问

只调用 **SendFromISR**，不调用 **YIELD** 会不会完全无法工作？

数据可能仍成功进入队列，任务可在后续调度点运行，但立即响应更高优先级任务的机会会丢失；应遵循移植推荐模式，而不是以“偶尔能跑”判断正确。

软件缓冲满时 **ISR** 能不能循环等任务来取？

不能。被唤醒的任务一般要等中断返回才运行，在 **ISR** 中等待它可能形成自我阻塞。应记录失败、丢失或延期恢复，由任务和流控协议处理容量。

依据与核对范围

FreeRTOS: [Cortex-M interrupt priorities](#) — 优先级编码、BASEPRI 与可调用 **ISR** 范围。

FreeRTOS 官方内核书: [Interrupt Management](#) — 第 7 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

T087 原理问答

队列为何复制数据，传指针又有什么代价？

FreeRTOS Queue 按创建时指定的元素大小复制字节，因而适合明确的数据交接。若队列元素是指针，则只复制地址，底层对象的拥有、复用与释放责任必须另行设计。

原理与成立条件

FreeRTOS 队列默认按值复制

创建时确定元素个数和单项字节数，发送时把这些字节复制进队列，接收再复制到调用者缓冲。因此发送局部结构体通常可以在入队成功后离开作用域；队列保存的是副本。但若元素本身是指针，只复制地址，被指数据不会自动被深拷贝或保活。

指针队列必须配所有权协议

生产者从池中取得块，填好数据并成功入队后交出所有权；消费者处理完成再归还。入队失败、消费者取消、任务退出和超时都要确定由谁释放。栈数组地址入队后函数返回、静态缓冲立即复用、DMA 下一轮覆盖，都属于指针仍非空却内容已无效的情形。

阻塞与唤醒如何发生

队列满时任务发送可在给定期限内阻塞，队列空时接收可阻塞；有进展时按内核规则唤醒等待者。**ISR** 只能用非阻塞专用 **API**。检查队列当前空闲再发送不是原子预留，多生产者下状态可能立刻变化，最终仍以发送返回值为准。

复制和实时预算

固定短帧按值传递便于证明寿命，代价是发送与接收复制及短临界区开销；大对象可用块池减少搬运，但增加管理复杂度。队列深度还带来等待时延，8 项不表示第 8 项一定及时处理。队列满的策略应在接口中体现，不能忽略失败却让上游以为已接纳。

推演与反例

值队列为何便于短帧发送

生产者在栈上组好长度和固定数组，发送成功后队列保有副本，生产者可以继续或返回；消费者取出后又拥有自己的副本。若改为栈数组指针，发送函数返回或再次组帧就可能让内容变化。对于大块数据，可从池取块并在成功入队时转移所有权，消费者完成后归还；失败时则由仍拥有它的生产者释放。

队列深度与时延要一起算

若消费者每项最长需要 20 ms，前面已排 7 项，第 8 项可能仅排队就等约 140 ms，还未计算抢占。增大深度吸收更多突发，同时增加内存及可能等待。先查剩余空间再发送不构成原子预留，多生产者下仍以实际发送结果为准。固定元素复制也不能直接用于依赖 C++ 构造析构的非平凡对象。

继续追问

队列里放 `shared_ptr` 是否适合直接用 C 队列 `memcpy`?

需要尊重 C++ 对象构造/析构及复制语义，不能把非平凡对象当任意字节复制后当作合法独立对象。应使用适当包装或传可管理句柄。

队列满时丢新项是否总比丢旧项安全?

看业务。遥测可能允许合并成最新值，命令和结果可能承担顺序与确认责任，不能随便淘汰。策略必须与上层接纳语义相符。

依据与核对范围

FreeRTOS 官方 `queue.h` 接口契约 — 队列元素复制、发送等待与 ISR 接口。

FreeRTOS 官方内核书: `Queue Management` — 第 5 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

T088 原理问答

StreamBuffer 与 Queue、MessageBuffer 有什么区别?

Queue 保留固定元素边界，StreamBuffer 是无消息边界的字节流，MessageBuffer 保存变长整条消息。后两者常规设计针对单写者单读者，不能把它们直接当任意多生产者队列。

原理与成立条件

三种容器保留什么边界

Queue 保存固定大小元素；**StreamBuffer** 只保存连续字节，不保留每次 **send** 的消息边界；**MessageBuffer** 在内部为变长消息保存长度，使接收按整条消息进行。选择应由数据模型决定：串口天然为字节流，固定事件适合队列，变长完整消息可用 **MessageBuffer**。

单生产者单消费者为何特殊

FreeRTOS StreamBuffer/MessageBuffer 的常规设计假设一个写者和一个读者，利用各自推进索引减少同步成本。多个写者或多个读者需按文档串行化并遵守阻塞限制，不能当作通用多生产者队列。两个 **ISR** 是否算一个串行写者，还要检查其能否嵌套及其他访问入口。

trigger level 不代表帧长

流缓冲的触发水位主要影响等待中的接收任务何时被唤醒，不负责协议定界，也不保证每次 **receive** 都返回同样数量。超时或已有数据等条件会影响返回数量；接收方必须按实际返回字节数处理。发送方也可能只写入部分字节，尤其在 **ISR** 无等待场景。

消息缓冲的容量陷阱

MessageBuffer 需要为每条消息存长度开销，接收缓冲不足时不能像字节流那样默认读半条后继续；应按 **API** 处理消息未取出。流缓冲适合一边输入一边解析，但仍需端到端容量与覆盖证明，省掉元素结构并不让溢出消失。

推演与反例

SPSC 约束为什么能简化实现

写者推进写索引，读者推进读索引，不需要每次都在多个写者之间抢占同一个预留位置。多个任务或可互相嵌套的 **ISR** 同时写时，这个前提就被破坏；按文档加外部串行保护还必须遵守阻塞参数限制，不能简单围住一个可能睡眠的发送。一个 **UART** 的两个 **IRQ** 是否构成串行写者，要看它们的抢占优先级与所有调用入口。

trigger level 到底唤醒谁

它主要决定等待接收任务在积累多少字节时应被唤醒，不是每次返回的固定长度，也不识别业务帧。已有数据、等待到期等路径会影响实际返回数量。**MessageBuffer** 另有长度字段开销，并要求接收方能容纳完整消息；缓冲太小时不能假定像流缓冲一样自动取半条。

继续追问

为什么不直接把 DMA 每次回调当一个 MessageBuffer 消息？

回调块由硬件进度决定，不是业务帧，可能半帧或多帧。人为保留这个边界往往没有业务价值，还增加长度开销。

同时用任务通知与 **StreamBuffer** 为什么要小心？

StreamBuffer 的阻塞实现可能使用任务通知槽。若应用复用同一个通知槽并采用不兼容语义，会干扰内部等待；应核对版本和可配置通知索引。

依据与核对范围

FreeRTOS: **Stream / message buffers** — 单写者单读者与字节流语义。

FreeRTOS 官方 **StreamBuffer** 接口契约 — 单写者/单读者、**trigger level** 与 **Send/Receive** 返回值；以版本适用性为前提。

T089 原理问答

任务通知的计数、事件与数值语义如何选择？

任务通知直接把状态和值保存在目标任务 **TCB** 中，适合单一接收者。增量计数、置位事件、覆盖数值与不覆盖发送分别保留不同信息，选择前要说明是否允许合并或丢历史值。

原理与成立条件

任务通知为何轻量

通知值及待处理状态放在接收任务的 **TCB** 中，发送者直接指定目标任务，不必另建通用队列对象。支持的用法包括置位、计数、覆盖或不覆盖数值；较新内核支持通知数组，具体数量受配置控制。接收者是一个指定任务，不是可被任意消费者取走的公共队列。

通知不是数据存储容器

一个 32 位值不适合携带任意消息，也不负责外部缓冲生命周期。常见设计是 **DMA/ISR** 先把数据放在已约定的缓冲中，通知任务“有进度”，任务再消费。多块完成时需计数、生产索引或队列支持，不能每次只写同一个位还期待知道完成了几块。

什么时候选择其他机制

多个消费者竞争固定元素用 **Queue**；需要多任务等待组合条件用事件组；多个任意对象间资源计数可用信号量。通知还可能与库内部使用的通知槽冲突，例如流/消息缓冲相关等待，应核实版本及索引配置。轻量的前提是拓扑和语义恰好匹配，不是所有同步都替换为通知。

推演与反例

二值、计数和位图的区别

连续发生三次事件，二值通知可能只保留“至少一次”，计数方式可保留 3，按位置位只保留哪些种类曾发生。**ulTaskNotifyTake** 的退出清零与只减一又对应不同消费方式。把计数当业务数据本身时还要考虑溢出及处理失败；若需要每次事件的参数和顺序，队列通常更合适。

通知为什么轻量又不通用

它省去独立队列对象并直接知道唤醒目标，通常开销小；但同一通知索引只有一个接收任务，不能直接替代多消费者队列或完整消息缓冲。多个独立用途应明确分配通知索引，避免库内部通知与应用状态互相覆盖。中断发送仍需 **FromISR**、正确优先级和切换请求。

继续追问

DMA 超时后来了上一次完成通知，怎么避免误认下一次传输完成？

为传输建立轮次或状态，启动前清理/识别旧事件，取消时确认硬件不再继续回调；单独把通知值清零不能关闭晚到事件窗口。

DMA 完成用通知后，发送缓冲何时可复用？

在确认通知代表对应那一笔传输的正确完成点，并排除旧完成/超时恢复竞态后。通知只是传递事件，缓冲寿命仍由传输状态机管理。

依据与核对范围

FreeRTOS 官方内核书：**Task Notifications** — 第 10 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

T090 原理问答

优先级继承具体怎样工作，为什么仍不能保证快速响应？

优先级继承让低优先级持锁者临时提高优先级，减少中间优先级任务对高优先级等待者的间接阻塞。它缩短的是可抢占干扰，不能消除临界区自身的工作时间。

原理与成立条件

继承怎样改变调度

H 等锁时，内核把持有者 L 的有效优先级提升到足以降低这类干扰的水平，L 运行并解锁后再按实现规则恢复。它不改变硬件转换时间、不自动缩短临界业务、不修复死锁。若持锁者等一个永远不会到的外部事件，提升到最高优先级也不能建立有限等待上界。

为什么二值信号量通常不继承

信号量可由不同执行者发送与等待，没有必然的“拥有者必须归还”关系，内核无法简单确定该提升谁。互斥量把所有者纳入语义，适合保护共享资源；**ISR** 不应取得任务互斥量，因为它不是可阻塞、可提升任务优先级的普通持锁者。

推演与反例

把反转时间拆开看

H 等待 L 的锁，L 还需执行 2 ms 才能释放；没有继承时，中优先级 M 可不断运行，把这 2 ms 拉得很长。继承后 L 可优先于 M 完成，但 H 仍至少等 L 剩余工作以及更高紧急度中断。如果 L 持锁等待外设 180 ms，继承并不能加快传感器，设计上应审视是否能缩短持锁范围或改用缓存/服务任务。

FreeRTOS 的继承为什么称为简化机制

其互斥量实现为了 RAM 与执行开销做取舍，持有多个互斥量时，优先级回落可能要等全部相关持锁释放，不能把完整的嵌套继承协议任意套入。二值信号量没有同样的所有者继承语义。讨论具体超时和多锁行为时，应核对所用内核的 queue/tasks 实现及文档，而不是只背“自动提高到最高”。

继续追问

把 180 ms 传感器等待放进互斥区，有继承就没问题了吗？

仍会阻塞需要同一设备的高优先级任务。可以考虑启动转换后释放不需保护的状态、由设备服务任务管理事务，或读取缓存快照，但必须保持总线和测量状态一致。

优先级天花板与继承一样吗？

不是。天花板协议基于资源的预先优先级约束主动控制访问，继承通常在高优先级任务实际阻塞时生效；可证明性质与实现条件不同。

依据与核对范围

FreeRTOS: [Mutexes](#) — 优先级继承及任务上下文。

FreeRTOS 官方内核书: [Resource Management](#) — 第 8 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

T091 原理问答

临界区、挂起调度器和互斥量分别适合什么？

互斥量适合任务之间保护可等待资源，临界区适合非常短的不可分割状态更新，挂起调度器只暂停任务切换而通常不停止中断。三者不能按“锁得更狠”排成替换关系。

原理与成立条件

临界区限制可打断者

在本类 Cortex-M3 移植中，taskENTER_CRITICAL 通过 BASEPRI 和嵌套计数保护内核允许范围，阻止相关 ISR 和任务切换干扰短更新。更高紧急度 ISR 仍可能运行，DMA 也继续工作；它不是对所有并发来源的万能锁。

互斥量允许任务等待

互斥量竞争时任务可以阻塞，其他任务得以运行；适合较长但有界、允许调度的资源操作。它不能在 ISR 中作为普通任务锁使用，并可能带来反转及死锁。选择机制时先问“谁会访问、允许等多久、等待期间谁能推进”，再问开销大小。

三者错误混用的后果

临界区中等待依赖 Tick 的超时可能永不到期；挂起调度器后等队列会妨碍使条件成立的任务运行；拿互斥量保护 ISR 直接访问的数据没有阻挡 ISR。对必须短时间跨任务和 ISR 共享的计数，可用合适原子/临界交接；对慢设备更适合请求队列和专属拥有者。

推演与反例

挂起调度器为什么保护不了 ISR 共享数据

任务暂停调度后仍可能被中断打断，ISR 可以修改共享索引；任务随后用旧值写回仍可能丢更新。只有相关访问者都被协议排除，才构成有效互斥。调度挂起期间内核还可能暂存待处理 Tick/就绪变化，在恢复时统一处理，不能在这段时间调用需要立即阻塞的操作。

临界区为什么不能调用任意延时

进入临界区后若等待一个由被屏蔽 ISR 完成的标志，完成者根本没有机会运行。即使是循环执行固定次数，只要耗时很长也会增加所有相关中断延迟、缓冲覆盖风险和时钟失真。把持锁等待替换成关中断并没有解决等待本身，而是影响范围更大。

继续追问

SuspendAll 后 Tick 没更新，与关中断漏 Tick 一样吗？

不同。ISR 若仍执行，内核可记录推迟的 Tick 并恢复处理；中断被长期屏蔽时硬件可能只保留一个待处理状态，未必知道过去了几次周期。

一个 32 位变量在 32 位 MCU 上是否不用保护？

对齐单次访问可能具有硬件原子性，但复合更新、多个字段不变量、语言可见性以及 ISR/任务协议仍需分析。不能从访问宽度直接推导所有并发操作安全。

依据与核对范围

FreeRTOS 官方 `task.h` 接口契约 — 临界区、调度器挂起与阻塞限制。

FreeRTOS 官方内核书: **Resource Management** — 第 8 章; 用于通用机制。具体节点实现另对照随包内核源码, 不推定与上游 main 完全相同。

T092 原理问答

vTaskDelay 与 vTaskDelayUntil 的周期误差有什么区别?

`vTaskDelay` 从调用时刻相对等待, 周期会叠加本轮执行时间; `vTaskDelayUntil` 按保存的基准推进目标唤醒点, 更适合保持周期相位。二者都只能让任务到时就绪, 实际运行仍取决于调度。

原理与成立条件

相对延时为什么漂移

`vTaskDelay(T)` 从调用时刻起阻塞若干 Tick。若循环先执行 C, 再延时 T, 启动间隔约 C+T, 还受调度影响; 每轮执行时间变化会累积到相位。它适合“至少稍后再试”的相对等待, 不适合单靠这种写法维持固定采样栅格。

DelayUntil 怎样保持相位

`vTaskDelayUntil/xTaskDelayUntil` 按保存的上次计划唤醒时刻加周期得到下一目标, 执行耗时只占用周期内部预算。调用者正确初始化并保存 `previousWakeTime`; 若目标尚未来到则阻塞, 已错过则不等待或按接口返回信息报告, 后续相位处理由设计决定。

Tick 量化和调度误差

延时单位是 Tick, 应使用适当转换并检查转换为零或溢出; 请求 1 Tick 的真实等待与调用相位有关, 并不保证完整 1 ms。到期进入 Ready 后还需得到 CPU, 所以 RTOS 延时不是高精度脉冲发生器, 严格边沿用硬件定时器更合适。

超期后的追赶策略

若任务每轮耗时超过周期, `DelayUntil` 可能连续不阻塞以追赶旧目标, 形成过载。业务需选择跳过过期周期、合并工作或更新基准, 并记录超期; 不能让无限补做把高优先级任务变成永久就绪。周期计时与工作完成计数也不能混用。

推演与反例

用 10 ms 周期比较漂移

一轮计算 3 ms 后调用 `Delay(10)`，下一轮约在 13 ms 后开始，执行时间变化会不断进入周期。`DelayUntil` 则把目标设置为原基准加 10 ms，正常只再等待约 7 ms，长期保持 0、10、20...的目标网格。若一次工作超过周期，目标可能已经在过去，函数可能不阻塞；应用需要决定补做、跳过还是报告错过期限。

一个 Tick 延时不等于至少完整一个 Tick 时长

调用相对延时的时刻可能紧邻下一次 Tick，延时 1 Tick 的实际阻塞时间可能接近零到一个节拍，再加调度等待。若设备要求严格不小于某个物理等待时间，需考虑取整和相位，或用硬件时间判断。周期函数减少相位漂移，不会修复 Tick 丢失或中断长期屏蔽造成的时基错误。

继续追问

调整周期变量后能否立刻改变已经开始的延时？

通常不会自动重排既有阻塞；若要求即时响应配置，需事件唤醒或重新计算目标，并规定从旧基准还是当前时刻开始新周期。

为什么周期计算不宜每轮只做 `elapsed+=周期`？

超期、补做和 Tick 失真时，累计逻辑次数未必等于真实经过时间。需要实际墙钟/单调时间语义的业务，应记录时间戳并计算差值。

依据与核对范围

FreeRTOS: `vTaskDelayUntil` — 相对与绝对周期延时。

FreeRTOS 官方内核书: [Task Management](#) — 第 4 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

T093 原理问答

FreeRTOS heap_1 到 heap_5 的分配策略与适用条件是什么？

FreeRTOS 的多种 heap 实现是在释放能力、碎片处理、多个内存区与可分析性之间取舍。选型关键是分配模式和最坏延迟，不是编号越大越先进。

原理与成立条件

heap_4 为什么仍可能失败

它维护空闲块并查找足够大的块，释放时尝试与地址相邻空闲块合并。被活对象隔开的空闲区不能合并，故总空闲量大于请求仍可能分配失败；分配耗时也与空闲链结构有关，不能宣称严格固定常数时间或永无碎片。

静态分配和内存池

静态创建接口由应用提供 TCB、栈、队列存储，便于预算并减少运行期分配失败；固定块池为特定对象提供简单的分配上界，代价是容量与块内浪费。选择时还要看对象是否会频繁创建删除、需要哪些尺寸、允许的最坏延迟，不是 heap 数字越大越先进。

监测与失败策略

总剩余堆、历史最低剩余和最大连续块回答不同问题。任务/队列创建失败应立即进入可诊断的安全路径；malloc 失败钩子只能帮助发现，不会凭空释放足够内存。ISR 中一般不应走普通堆分配，应使用预分配资源并设满时行为。

推演与反例

五种常见策略怎样区别

heap_1 只分配不释放，适合启动后不再释放的模式；heap_2 可释放但不合并相邻块，容易留下外部碎片；heap_3 包装 C 库分配器；heap_4 在释放时合并相邻空闲块；heap_5 可管理多个非连续内存区域并采用相关合并策略。各实现的可用配置与版本要核对，不能同时任意链接多个提供同名 pvPortMalloc 的实现。

合并空闲块为什么仍不等于无碎片

如果两个空闲块之间夹着一个长期存活对象，就不能合并；总剩余堆大于一次申请仍可能失败。heap_4 的搜索与合并路径也随块分布变化，不能宣称固定 $O(1)$ 或绝对实时。固定块池可减少尺寸搜索和外部碎片，但会有块内浪费及容量耗尽，需要明确失败策略。

继续追问

同时用 malloc 和 pvPortMalloc 可以互相 free 吗？

除非实现明确约定同一个分配器且接口兼容，否则不能。heap_4 通常管理自己的数组，与 C 库堆是不同来源；必须成对归还。

heap_4 的空闲字节数够创建任务，为何还会失败？

还可能缺少足够大连续块、需要额外 TCB/对齐开销，或在同时创建 Idle/timer 任务时超出预算。应检查具体申请项与最大块，而不是只看一个总量。

依据与核对范围

FreeRTOS: Memory management — heap_1..5 与静态分配。

FreeRTOS 官方内核书: Heap Memory Management — 第 3 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

如何估算任务栈，为什么中断栈要单独看？

栈预算要沿最深调用链估算，再结合运行期高水位验证。任务栈、主栈/中断栈和堆是不同资源，FreeRTOS 栈深度接口的单位还可能是字而不是字节。

原理与成立条件

栈深度单位先确认

FreeRTOS 原生任务创建常以 `StackType_t` 元素数表示深度，不是字节；在 32 位 ARM_CM3 上 384 个元素是 1536 B。CMSIS 等包装层接口可能使用字节，不能跨 API 套单位。每个任务都要保存自己的调用现场，任务越多并非只增加一个 TCB。

最坏调用链如何预算

沿最深同步调用路径累计局部变量、保存寄存器、库函数及递归空间，再加任务相关异常入栈与余量。格式化输出、大数组和错误恢复路经常比正常路径深；中断 Handler 使用的 MSP 也需独立考虑最大嵌套。平均函数栈耗没有证明意义。

高水位能证明什么

栈创建时填充已知模式，查询连续未被覆盖区域得到历史最小剩余。它只反映实际运行覆盖，未执行路径不会被测到；编译器保留但未写入的大区域也可能影响解释。测量要覆盖最大格式化、传感器失败、重试及嵌套场景，并与 map/编译器栈使用报告结合。

检测与预防

开启合适栈溢出检查、断言和失败钩子能更早暴露问题，但不是保护所有越界的硬件隔离。溢出可能先破坏邻近 TCB/链表，再在远处崩溃。避免无界递归和大局部数组，必要时使用静态/池化缓冲并明确所有权，同时保留主栈安全余量。

推演与反例

栈空间怎样一步步累计

任务普通调用保存返回地址、寄存器和局部变量；最深路径还可能调用格式化、错误恢复与库函数。Cortex-M 任务被中断时，其基础异常帧可能压在任务栈上，Handler 的后续调用与嵌套再使用 MSP。必须把任务异常帧余量与 MSP 最大嵌套分别计算，不能只给三个应用任务分完 RAM 就认为没有其他栈消耗。

高水位不是最坏情况证明

填充模式被覆盖到多深只能反映已经执行且实际写栈的路径；从未出现的错误、最大报文格式化或编译器变化可能用得更多。某些分配但未写入的栈区域也会影响解释。结合编译器栈使用报告、map、覆盖测试及保护机制，保留合理余量后再调整；不能把测得的最后 100 字节全部裁掉。

继续追问

高水位还剩 100 字节，能再缩 100 吗？

不应机械缩到零。观测未涵盖全部路径，未来编译器与库变化也会改变占用，应结合最坏分析和余量再调整。

大缓冲放 **static** 就一定更好吗？

可减少每任务栈压力，但变成共享长期对象，需要同步和寿命设计，也固定占 **RAM**。应看是否可重入、是否会被 DMA 或其他任务同时使用。

依据与核对范围

FreeRTOS: **Stack high-water mark** — 水位单位与观测范围。

FreeRTOS 官方 **task.h** 接口契约 — 临界区、调度器挂起与阻塞限制。

T095 原理问答

软件定时器和事件组有哪些容易忽略的上下文问题？

软件定时器回调通常由公共定时器服务任务串行执行，事件组表达一组状态位而不保留每次事件次数。理解它们首先要回答“在哪个上下文执行、是否排队、是否允许阻塞”。

原理与成立条件

软件定时器在哪执行

FreeRTOS 软件定时器回调由同一个定时器服务任务串行调用，不是每个定时器一个硬中断。到期精度受 Tick、服务任务优先级、命令队列和前面回调影响；一个回调很慢会拖延其他定时器。因此回调应短小、通常不阻塞，把慢工作交给专门任务。

事件组表示状态组合

每一位表示一个条件，任务可等任意位或全部位，并选择满足时清除。重复置同一位会合并，不保留事件次数；多个等待任务可能因同一置位同时解除阻塞。有效用户位数受 Tick 宽度和内核定义影响，不能固定认为整个 **EventBits_t** 都可供应用使用。

ISR 置事件的特殊路径

置位可能唤醒数量不定的等待任务，FreeRTOS 的 **xEventGroupSetBitsFromISR** 通常把操作延后到定时器服务任务，以保持 **ISR** 工作有界；因此依赖该命令队列空间，也不是硬中断里立刻完成所有唤醒。需要单接收者低开销通知时，可比较任务通知，但语义并不完全相同。

推演与反例

定时器命令成功与回调执行之间隔了什么

start/stop 等 API 可把命令放入服务任务队列，服务任务取得 CPU 后才处理；回调若做长 I/O，会推迟其他所有软件定时器。回调向自己的命令队列无限等待，在队列满时可能等不到消费者，因为消费者就是自己。回调适合做短状态更新或通知工作任务，长业务放其他上下文。

事件组的清除与广播行为

置位可能同时满足多个等待任务，重复置同一位会合并；等待任意位、所有位与退出清位组合要按实际需求选择。**ISR** 置事件位可能通过定时器服务任务延后处理，以避免在 **ISR** 唤醒数量不定的任务，因此还依赖该命令队列容量。用户可用位数受 Tick 宽度等配置影响，不能固定假定全部位都可用。

继续追问

事件组全部位曾经置一，是否代表各任务同一时刻健康？

只表示在该观察窗口内分别发生过签到；窗口、清除原子性与签到所代表的进展仍需设计，不能把位图当一致时间戳。

先 **GetBits** 再 **ClearBits** 能保证只清除本轮已观察到的签到吗？

两个独立操作之间可能出现新签到，随后 **ClearBits** 把它一起清掉。若要求严格区分观察窗口，应采用能原子完成所需语义的等待/清除协议，或使用进度计数/时间戳。

依据与核对范围

FreeRTOS: **Software timers** — 定时器服务任务与回调约束。

FreeRTOS: **Event groups** — 事件位与等待任意/全部。

FreeRTOS 官方内核书: **Software Timer Management** — 第 6 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

FreeRTOS 官方内核书: **Event Groups** — 第 9 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

时间、低功耗与故障恢复

T096 原理问答

HAL Tick 与 RTOS Tick 各管什么，合并时基有什么前提？

HAL Tick 服务驱动的毫秒等待与超时，RTOS Tick 服务任务延时和调度。可以使用不同硬件时基，也可在满足启动、中断和低功耗约束后共享，关键是两套时间语义始终成立。

原理与成立条件

统一时基要解决什么

调度器启动前仍可能需要 HAL 延时；中断上下文不能随意调用只适合任务的 API；RTOS Tick 频率可能不是 1 kHz，需要转换并考虑舍入；Tickless 跳过周期中断后还需让 HAL 经过时间语义一致。统一方案必须覆盖所有阶段，不能只让正常任务中读数相同。

两个时基的成本与好处

独立 HAL 定时器保持驱动对毫秒超时的依赖，减少侵入 RTOS 移植；代价是占用硬件定时器和周期唤醒，且两计数可能因屏蔽或睡眠发生差异。若长临界区同时屏蔽二者，双时基也不会自动互相校正，可靠性仍取决于边界设计。

超时为何会在中断中卡死

若 HAL 函数等待由低紧急度 Tick 推进的期限，而当前高紧急度 ISR 不退出，该 Tick 进不来，有限 timeout 也可能不再推进。原则上把此类阻塞操作移到任务；确需 ISR 轮询时应使用适合该上下文的有界硬件计时机制，而非一味提高 Tick 紧急度。

推演与反例

调度器启动前为什么容易出问题

HAL 初始化和某些驱动已经需要超时，此时 RTOS 尚未开始，直接把 HAL_GetTick 改为任务 Tick 可能得到不推进的时间。反过来，调度启动后 SysTick 的处理权、频率与优先级要符合内核要求；如果 HAL 中断中的等待依赖一个无法抢占当前 ISR 的 Tick，超时可能永远不发生。时基统一不是简单改两个函数名。

低功耗期间两套钟怎样保持一致

RTOS Tickless 可停止周期 Tick，并在唤醒后补偿内核时间；如果另一个 HAL 定时器被停掉，其 uwTick 不会自动获得相同补偿。于是两个“毫秒”可能逐渐不同。应决定 HAL 时间是否必须包含睡眠，对正在等待的超时有什么含义，再设计补偿或约束调用范围，不能只恢复中断就称为时钟恢复。

继续追问

合并两个 **Tick** 一定更省电吗？

可能减少周期中断，但若 **Tickless** 补偿、其他唤醒和低功耗时钟未配套，收益有限且可能出错。先量化唤醒来源，再决定是否值得改变时基契约。

保留两个 **1 kHz** 中断是否一定合理？

不一定，额外唤醒会影响功耗；但合并的收益要与启动和补偿复杂度比较。先明确哪一个周期中断可在 **Idle** 暂停，再证明醒来后的时间契约。

依据与核对范围

ST 官方 **stm32f1xx HAL UART 源码** — **HAL_UART_IRQHandler / ReceiveToIdle** 与驱动状态；项目结论以离线本地版本为准。

FreeRTOS: **Low power support** — **Tickless** 进入、唤醒与 **Tick** 补偿。

FreeRTOS 官方 **ARM_CM3 移植源码** — **SVC/PendSV**、**xPortSysTickHandler**、**vPortSuppressTicksAndSleep**；上游参考，当前行为以本地快照为准。

T097 原理问答

Tickless Idle 的完整进入、唤醒和补偿过程是什么？

Tickless Idle 在没有可运行业务任务时，用一次较远的唤醒替代多个周期 **Tick**，并在醒来后补回实际经过的内核时间。它需要睡前重检、提前唤醒处理和计数边界，不能简化成关 **SysTick** 后执行 **WFI**。

原理与成立条件

Tickless 的目标

周期 **Tick** 在所有任务都阻塞时仍会唤醒 **CPU**；**Tickless** 估计下一项调度工作前的空闲时间，把多个节拍合成一次较长计时，减少无意义唤醒。它优化空闲管理，不自动选择芯片最深睡眠，也不表示调度器完全没有时间概念。

入睡前的竞态怎样避免

Idle 估算空闲时间后，可能刚好有中断使任务就绪。移植层需在适当屏蔽与调度状态下再次确认能睡，再设置计时与执行等待指令；否则可能在工作已到达后错误长睡。临界范围与唤醒机制必须按架构实现，不能简单“队列空就 **WFI**”。

醒来如何补时间

可能由计划定时器到期，也可能被 UART 等提前唤醒。根据剩余计数、重装值与硬件状态计算真正经过的完整 Tick，调用内核步进/恢复逻辑，再保留不足一 Tick 的相位。补偿不能越过应当解除阻塞的时刻而不处理任务，因此预估上限与内核接口契约很重要。

长睡眠还取决于时钟

如果深睡眠停止了所用 SysTick 时钟，就需可在睡眠中运行的 RTC/LPTIM 等重新设计计时与补偿；计数位宽也限制单次时长。低功耗收益还受进入退出耗时、外设、调试器与板上电流影响，只有配置选项为 1 不能证明实际省电幅度。

推演与反例

睡前为什么要再次确认

计算可睡 100 ms 后，可能有 ISR 把任务置为 Ready；若仍按旧估计入睡，会推迟本应立即运行的任务。端口在受控中断状态下重检是否允许睡眠，确定下次到期点，重装可用计时器，再进入等待。这个序列还要防止“检查后事件到达但睡下时错过唤醒”的竞态，因此应遵循架构与移植代码。

提前唤醒怎样补偿

如果 UART 在计划的 100 ms 前、实际 30 ms 时唤醒，不能直接把 Tick 加 100。要根据保留下来的计数器值区分完整 Tick 和不足一个 Tick 的余量，恢复下一节拍相位，再处理到期任务。vTaskStepTick 不是允许随意跨越所有延时任务的万能加法接口，补偿必须与内核最早唤醒点及版本实现配合。

继续追问

UART 提前唤醒时能补上预计空闲的全部 Tick 吗？

不能，应按实际已睡时间补偿；补预计全部时长会把系统时间跳到未来，让延时任务与超时过早触发。

Tickless 开启后为何仍每毫秒醒？

可能还有 HAL 定时器或其他周期中断持续唤醒，也可能任务以短超时轮询导致 Idle 窗口很短。需要同时检查硬件唤醒源与任务阻塞期限。

依据与核对范围

FreeRTOS: Low power support — Tickless 进入、唤醒与 Tick 补偿。

FreeRTOS 官方 ARM_CM3 移植源码 — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep；上游参考，当前行为以本地快照为准。

硬件计数器位宽怎样限制一次休眠时长？

一次最长休眠由计时器范围、计时频率、最近任务期限和其他唤醒源共同决定。软件预计可空闲很久，不代表所用硬件计数器能一次计那么久。

原理与成立条件

可睡多久取决于最近的事件

周期采样虽然可能几分钟一次，但任何任务的短超时、软件定时器和维护工作都会缩短 **Idle** 可合并的区间。内核计算的是最早需要重新运行的调度事件，而不是最长周期任务。外设中断还可能比这个期限更早唤醒。

24 位 SysTick 的上限

SysTick 重装寄存器有效 24 位，最大可计数约 2^{24} 个输入时钟；72 MHz 直接作为输入时，约 $16,777,216/72,000,000 \approx 0.233$ s。若输入时钟选择不同，上限随之变化，还需扣移植实现的补偿边界；不能把 233 ms 当所有 Cortex-M 的固定值。

超时维护如何影响功耗

例如，一个任务为协议 TTL 或错误维护每 100 ms 醒一次，即使没有业务，也可能把空闲窗口限制在这个量级。可以改成按最近真实截止时间阻塞，但要保持协议进度、喂狗检查与异常恢复，不应为了长睡把必要状态维护完全停掉。

长期周期需要跨多次睡眠

几分钟等待可以由多次短 Tickless 区间组成；若要减少到一次长睡，需硬件计时源、任务超时和看门狗一起支持。唤醒次数下降并不等价于系统平均功耗同比下降，传感器转换、串口、稳压器和板载 LED 都可能占更大部分。

推演与反例

24 位计数器上限怎样算

SysTick 为 24 位递减计数器，最大一个周期约为 $2^{24}/f_{CLK}$ 。若用 72 MHz 时钟，约为 0.233 s；按 1 ms Tick 切分时能抑制的完整节拍数还会因取整与端口补偿略有差异。如果改用另一种时钟源，必须重新代入频率，不应把 233 ms 记成 Cortex-M 的固定常数。

最近任务期限常比硬件上限更小

一个任务每 100 ms 超时醒来检查状态，即使其他任务可睡数秒，内核也要在这个最早到期点附近恢复调度。硬件最大 233 ms、最近任务 100 ms，实际睡眠先被后者限制；更早的 UART/外部中断还能再次缩短。要延长空闲，应减少不必要轮询或使用更合适长计时源，同时保留看门狗和业务响应要求。

继续追问

将采样周期从 1 秒改为 5 分钟，是否就可睡 5 分钟？

只有不存在更早唤醒需求且硬件计时/模式允许时才可能。应列出所有等待期限和唤醒源，再计算最长空闲。

换 RTC 作唤醒就能无限睡吗？

仍受任务最早期限、RTC 精度、可用唤醒模式、时钟恢复及看门狗限制。还需用 RTC 时间正确补偿 RTOS，不能只把唤醒来源换掉。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态; Rev 7。

FreeRTOS: Low power support — Tickless 进入、唤醒与 Tick 补偿。

FreeRTOS 官方 ARM_CM3 移植源码 — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep; 上游参考，当前行为以本地快照为准。

T099 原理问答

Sleep、Stop、Standby 的保持与唤醒机制有什么区别？

Sleep 通常保留较多外设与存储状态，只停止 CPU 等部分时钟；Stop 停止更多高速时钟但保留一定状态；Standby 通常需要按复位式流程重新启动。具体保留项和唤醒源必须按芯片系列核对。

原理与成立条件

Sleep、Stop、Standby 的层次

普通 Sleep 主要停止 CPU 执行，许多外设和时钟仍可运行，响应快；Stop 关闭更多高速时钟并通常保留 SRAM，唤醒后需恢复部分时钟；Standby 关闭更多供电域，通常按复位流程恢复，普通 SRAM 与运行上下文不能假定保留。精确保留项和唤醒源以具体型号手册为准。

WFI 指令不是模式名称

WFI 让处理器等待满足条件的中断事件，实际进入普通还是深睡眠由 SCB 和芯片电源控制配置共同决定。只看到汇编里有 WFI，不能得出进入 Stop；还要看 SLEEPDEEP、电源模式位和外设时钟配置。

深睡眠如何影响通信

若 UART 或 DMA 所需时钟停止，原有接收方案可能不能在睡眠中继续工作；某些芯片支持特定低功耗唤醒，条件又受时钟源、波特率和模式限制。恢复后要检查首字节是否丢失、波特率时钟是否恢复、HAL 状态和协议是否仍一致。

看门狗与上下文恢复

IWDG 常由独立低速源驱动，在某些低功耗模式仍继续计数；允许睡眠的时间必须短于实际最小喂狗余量，或选有适当选项的硬件设计。不能为了入睡无条件停掉健康检查。Standby 恢复时协议会话与去重状态若丢失，也需要启动级恢复策略。

推演与反例

低功耗越深，恢复工作越多

Sleep 中外设可能继续工作并中断唤醒；Stop 退出后可能需要恢复 HSE/PLL、总线频率和受影响的外设时间参数；Standby 后普通 SRAM 内容可能不保留，应用不能从原任务栈直接续跑。为了接收 UART，不能只看 CPU 能否执行 WFI，还要看 UART 时钟和该模式的唤醒能力是否存在。

Tickless 与低功耗模式是正交选择

Tickless 减少周期唤醒，具体用 Sleep 还是更深模式由端口和睡眠钩子决定。设置 configUSE_TICKLESS_IDLE 不会自动打开所有芯片的深睡眠配置。实际功耗还受传感器、上拉、指示灯、调试器和电源芯片影响，降低 CPU 活跃率不等价于按同样比例降低整板电流。

继续追问

测到 WFI 后电流变化很小，先查什么？

确认实际电源模式、仍在运行的外设和外部器件、调试连接、板载灯及稳压器损耗，再用受控测量分离 MCU 与整板电流。

调试时测得功耗很低或很高都能直接作为产品值吗？

不能。调试连接和冻结选项会改变时钟、看门狗及睡眠行为；应在明确供电、外设状态和脱离调试条件下测量并记录模式。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地图表 HAL。

T100 原理问答

忙等、任务延时、硬件计时怎样选择？

忙等适合很短且不能容忍调度粒度的等待，任务延时适合毫秒以上可让出 CPU 的间隔，硬件定时/捕获适合更稳定的边沿控制。选择要同时考虑精度、CPU 占用和对其他任务的影响。

原理与成立条件

HAL_Delay 的等待模型

典型 HAL_Delay 循环检查 HAL Tick，是忙等，任务仍处于可运行状态，低优先级任务未必获得 CPU；更高优先级任务和可响应中断仍可能打断。若 Tick 不推进，等待无法结束。它适合某些启动阶段，但不应把它等同 RTOS 阻塞延时。

vTaskDelay 的等待模型

任务进入 Blocked，调度器运行其他任务，到期后再转 Ready。它节省 CPU 和功耗，却有 Tick 量化与调度抖动，适合毫秒及以上业务等待。转换成 Tick 时应使用正确单位，并考虑小于一 Tick 变为零的情况。

DWT 与硬件定时器

DWT 周期计数可实现短忙等，依赖计数器已开启、CPU 频率和回绕假设；中断会拉长实际延时。硬件定时器的输出比较/输入捕获把严格边沿和测量交给硬件，对微秒波形更稳健。软件延时再精细，也无法自动消除抢占与总线访问抖动。

依据精度与占用选择

几十微秒的协议边沿、上百毫秒的传感器转换、数秒的周期任务是不同问题。把长等待全部忙等会挤压任务，把短边沿全部用任务延时又达不到分辨率。还要检查等待期间是否持锁，释放 CPU 不代表释放该设备资源。

推演与反例

HAL_Delay 为什么可能拖住低优先级任务

典型实现轮询 HAL Tick，函数本身不会主动把当前任务置为 Blocked。中断与更高优先级任务仍可能运行，但低优先级任务可能得不到机会。vTaskDelay 则退出就绪竞争，等待 Tick 到期后再竞争 CPU；代价是节拍取整与调度抖动，不能用于几十微秒的位采样。

DWT 周期计数的精度来自哪些前提

先启用计数器，再用差值比较经过周期，并按真实 CPU 频率换算时间。频率变化、调试冻结、某些睡眠行为及计数回绕都会影响解释；超时计时本身如果停止，循环即使写了 timeout 也可能永不退出。DWT 忙等让 CPU 持续工作，精细计时不等于低功耗。

继续追问

vTaskDelay(180) 一定是 180 ms 吗？

它是 180 Tick，只有 Tick 为 1 ms 时名义上对应 180 ms，实际恢复执行还受量化与调度影响。应写清单位并使用转换宏。

用长 NOP 循环延时为何不稳？

编译优化、指令取指等待、分支和主频变化都会改变耗时。需要明确硬件计时基准或经过约束的汇编，并用波形核对，不靠循环次数猜微秒。

依据与核对范围

ST 官方 stm32f1xx HAL UART 源码 — HAL_UART_IRQHandler / ReceiveToIdle 与驱动状态；项目结论以离线本地版本为准。

FreeRTOS: vTaskDelayUntil — 相对与绝对周期延时。

Arm CMSIS: 内核寄存器 — __get_MSP / __get_PSP / __set_BASEPRI / __set_PRIMASK；具体 M3 异常帧对照 PM0056。

T101 原理问答

长时间屏蔽 SysTick，会不会丢系统时间？

长时间屏蔽节拍中断时，硬件仍可能多次到期，但挂起状态通常不能记录每一次事件。恢复后只执行一次 Handler，就会少计真实时间；这与内核主动挂起调度器后累计已执行 Tick 的情况不同。

原理与成立条件

中断 pending 通常不是无限计数器

周期定时器多次到期而 IRQ 长期被屏蔽时，硬件状态可能只保留一个待处理标志。恢复后处理一次中断并不自动知道漏了多少次，因此普通 Tick 计数可能落后真实时间。具体外设设有计数器可读取，但必须另行设计补偿，不能靠 NVIC pending 自动累计全部次数。

Tickless 为什么不同

Tickless 在已知起始计数和预定唤醒边界下有意改变计时，醒来计算经过量并补内核时间；任意长临界区没有这套记录与恢复协议。二者表面都少进几次 Tick ISR，时间语义却完全不同，不能拿“启用了 Tickless”解释所有丢 Tick 都会自动补偿。

调度器挂起又是另一种情况

如果 Tick ISR 仍执行，FreeRTOS 可累计待处理 Tick，恢复调度器后处理；如果 ISR 被屏蔽而根本未运行，就可能丢失重复发生的信息。分析时先问硬件是否计时、IRQ 是否进入、内核是否记录，再判断哪个环节能够补偿。

系统影响怎样验证

在长临界区前后翻 GPIO，用逻辑分析仪比较外部真实时间与 HAL/RTOS 计数变化；同时注入连续 UART，观察 DMA 进度、CRC 和软件丢失。这样能区分延时变长、Tick 落后与缓冲覆盖。单纯打印时间戳依赖可能已经失真的软件时钟，不够证明。

推演与反例

三毫秒为什么未必补三个 Tick

假设每 1 ms 的 SysTick 在 3.5 ms 临界区内多次到零，NVIC 主要保留一个待处理状态而不是一个计数队列。临界区退出后进入一次 SysTick Handler，内核通常只增加一次节拍，之前多次事件可能合并。COUNTFLAG 也不是无限溢出计数器，不能事后仅凭一个标志重建所有圈数。

主动 Tickless 为何能做补偿

Tickless 在关闭周期事件前保存当前相位并装载较长计数，在允许范围内根据硬件剩余值计算实际经过时间，属于受控协议。任意业务长关中断没有保存同样信息和边界，不能直接照搬补偿函数。更优先的办法是缩短屏蔽段，或让严格时序由硬件捕获承担。

继续追问

关中断 10 ms，之后 Tick 一定一次补 10 吗？

没有显式补偿机制就不能这样保证；常见结果只是处理一个待决异常。恢复多少取决于硬件与移植实现，而非通用 RTOS 特性。

RTOS Tick 少计只会影响显示时间吗？

还会影响任务延时、软件定时器、协议超时和基于 Tick 的健康判断。一个局部时序技巧可能改变整个系统的时间语义。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态；Rev 7。

FreeRTOS 官方 ARM_CM3 移植源码 — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep；上游参考，当前行为以本地快照为准。

T102 原理问答

IWDG 与 WWDG 怎样工作，超时与窗口如何计算？

IWDG 使用独立低速时钟，适合在主时钟或软件失常时触发复位；WWDG 还检查喂狗是否过早，强调允许窗口。选择依据故障模型、时钟依赖与正常抖动，不是简单互相替代。

原理与成立条件

独立看门狗如何工作

IWDG 由独立低速时钟驱动递减计数，软件在期限内重装，未重装则复位。独立时钟能覆盖部分主时钟或任务失效，但低速 RC 有误差，超时时间要按频率上下界估算。名义公式常为 $(\text{reload}+1) \times \text{prescaler} / \text{fLSI}$ ，具体范围依芯片。

窗口看门狗多检查了什么

WWDG 既限制最晚刷新，也限制太早刷新，可发现程序异常跑快或跑入无条件喂狗循环。其时钟来源和低功耗行为与 IWDG 不同。窗口要求把最短、最长合法执行周期都纳入预算，调度抖动太大时配置过窄会误复位。

喂狗点决定检测能力

在周期中断里无条件喂狗，只能说明中断还在运行；应用任务可能早已死锁。应让独立健康逻辑汇总关键任务进度，只有满足契约才刷新。窗口内完成次数、最迟成功时间和外设恢复结局比“每次循环置一位”可表达更多故障，但设计不能误把正常长操作判死。

复位后必须保留原因

启动时读取复位标志并关联上次健康记录，区分 watchdog、上电、软件复位等。调试暂停可能让 CPU 不喂狗，调试冻结选项会改变实验现象；正式运行要按真实配置验证。watchdog 是故障恢复后盾，不是用来掩盖无界阻塞或持续错误的正常控制流程。

推演与反例

超时公式与误差怎样一起算

IWDG 典型超时近似为分频值 $\times$ (重装值+1)/LSI频率。分频 64、重装 1250、假定 LSI=40 kHz 时约 2.0016 s；真实 LSI 随器件、温度、电压变化，应以规定频率范围计算最短/最长超时。软件喂狗周期必须小于最短正常允许期限，并预留中断和任务阻塞余量。

WWDG 为什么可能发现另一类故障

窗口看门狗要求在计数器降到允许范围后、又未低到复位门限前刷新，过早和过晚都可能复位，可发现跑飞后疯狂喂狗等行为。但窗口受时钟与配置限制，也更容易受合法长阻塞影响。需要精确计算正常喂狗窗口和异常覆盖，不应仅因为名字多了 window 就认为更可靠。

继续追问

1 秒检查一次健康，是否意味着 IWDG 超时是 1 秒？

不意味着。检查周期是软件策略，硬件计数期限由分频、重载和实际 LSI 决定，还应保留最坏调度与频率误差余量。

复位后怎样知道真的是看门狗？

读取并妥善清除复位原因标志，结合上次持久/保留故障信息和运行日志。调试冻结选项可能让断点期间看门狗暂停，必须把这种环境区别记录清楚。

依据与核对范围

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地 HAL。

T103 原理问答

怎样设计能检查有效进展的多任务看门狗？

多任务看门狗不应让任意一个仍活着的循环独自喂狗，而应由监督者确认关键任务在允许窗口内完成了有意义的进展。监督条件与硬件超时必须覆盖真实最长合法工作。

原理与成立条件

多任务签到要定义观察窗口

每个关键任务在完成规定进展后报告，监督者只在所有必要条件满足时喂狗，然后开始下一窗口。签到的位置非常关键：拿到任务调度就签到，无法发现业务一直失败；只有在有业务时签到，又可能把正常空闲误判为故障。应区分空闲可服务与已完成工作。

事件位为什么可能丢失代际信息

某任务两次置同一位只保留一个状态；监督者先读取全部位，稍后再清位，期间新一轮签到可能被一起清掉。原子等待并清除能减少这类分步竞态，但要核对窗口语义。需要严格世代时可用递增进度计数或带时间戳的状态，而非仅靠一组布尔位。

监控者也有调度预算

监督任务优先级太低可能被忙任务压住，太高却可能掩盖低任务一直没机会工作。硬件最短超时需大于允许的合法最长间隔，再给调度和时钟误差留余量；同时小于业务允许的故障发现时间。所有等待若没有上界，便无法严谨配置这一范围。

活性与功能健康分开

任务能循环但 UART 重启失败，是活着却不能服务；传感器断线是局部功能故障，不一定要反复重启整机。可把必需活性作为喂狗条件，把可降级功能以错误状态上报，并设置有限恢复。否则一个永久外设故障会使设备陷入无限复位循环。

推演与反例

位图签到能做到与做不到什么

每任务置自己的位，监督者看到全部位后刷新并开始下一轮，能发现某任务长期没有到达签到点。但重复置位会合并，位图不记录次数和年龄；一个任务先签到再永久卡住，仍可能帮助完成当前一轮，直到下一轮才被发现。读位和清位之间出现新签到还可能被误清，需要原子窗口协议或计数/时间戳。

有效进度应与故障模型一致

命令任务空闲时及时响应等待超时，可以说明任务尚可调度，却不能证明 **RX DMA** 还工作；发送任务循环签到也不能证明线路数据真的发出。可把调度存活与驱动健康分开监控，使用 **RX** 错误、队列消费和事务完成时间。监督者自身应避免被无限等待拖住，并且失败时留下尽可能轻量的诊断信息。

继续追问

怎样证明签到没有误放行一个卡住任务？

定义“进展”以及最大间隔，再注入分别卡在签到前、签到后和外设等待中的故障，观察喂狗停止时间；只做一次整体死循环不足以覆盖。

监督周期 **1 s**、**IWDG** 约 **2 s**，能说故障一定 **2 s** 内复位吗？

不能只这样推断。从故障发生到最后一次允许喂狗之间可能还有观察窗口，硬件又有频率容差。应从最坏故障时刻、签到语义及最晚喂狗点推导总检测时间。

依据与核对范围

FreeRTOS: **Event groups** — 事件位与等待任意/全部。

FreeRTOS 官方内核书: **Event Groups** — 第 9 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 **main** 完全相同。

排障、验证与软件工程

T104 原理问答

CPU 很高、内存增长、延迟变大分别怎么定位？

性能排障先判断系统在计算、等待 CPU、等待锁还是等待 I/O，再针对瓶颈收集证据。CPU、内存和延迟不是孤立指标，队列积压可能把三者同时推高。

原理与成立条件

先把“慢”变成明确指标

区分吞吐、响应平均值、p99、最大延迟和超时率，再标明输入负载与统计区间。端到端延迟可拆为排队、运行、I/O 等待及协议往返；CPU 利用率低只说明总体忙碌比例低，不排除某个线程长时间等待锁或某一次回调阻塞全部事件。

CPU 高时如何缩小范围

按进程、线程找到热点，再用采样看函数和调用链。热点可能是有意义计算，也可能是 EAGAIN 忙轮询、可写空转、重试风暴或日志格式化。将采样结果与每轮处理量关联；仅看到某函数采样多，不能断言它每次调用都慢，调用次数也可能很多。

内存增长如何分类

区分不可达对象泄漏、仍可达的队列积压、缓存策略和分配器保留。观察入出速率、对象数量与容量高水位，关闭负载后是否回落，再结合内存工具。RSS 不等于所有活对象字节总和，虚拟内存大小更不能直接当真实占用；不同平台要选适用统计。

优化实验怎么有说服力

固定固件/程序版本、编译参数、输入种子与环境，一次只改变关键因素，记录失败、尾延迟和资源成本。主机 Sanitizer 用于正确性，不能直接作为生产性能测量环境。MCU 通过 GPIO、DWT 或跟踪观察，注意探针与 printf 自身改变时序。

推演与反例

高 CPU 和慢响应怎样分开

用线程级 CPU、调用栈采样和系统调用观察，区分计算热点、EAGAIN 忙循环、锁竞争和频繁唤醒。CPU 不高仍慢时，检查阻塞调用耗时、调度等待、磁盘与网络队列。不要先把所有锁换无锁：若根因是数据库每行提交或串口带宽，改锁可能无益还增加复杂度。

内存增长先看对象生命周期

区分真正泄漏、队列增长、缓存保留、分配器未归还 OS 和碎片。结合活对象数量、队列深度、峰值输入及 RSS 曲线，判断是否随压力回落。记录吞吐与延迟分位数、最大值和超时率，比单个平均值更能反映突发问题；观测代码自身也应有开销预算。

继续追问

从 20% CPU 降到 10% 可以说性能翻倍吗？

不能。它只说明该条件下 CPU 占用变化，还需看吞吐、延迟、能耗和资源约束；业务响应可能完全没变，甚至因更大批量而变慢。

优化前后只看一组平均值够吗？

不够。需要相同输入、配置、环境和测量口径，并看尾延迟、资源峰值及错误率是否恶化。代码级推测不能直接写成已取得的性能提升。

依据与核对范围

Linux Kernel: [Perf events](#) — 性能事件采样与权限；性能诊断方案由本文组织。

[proc\(5\)](#) — 进程、内存与运行期诊断入口。

[strace](#) 官方文档入口 — 系统调用、信号与进程跟踪。

T105 原理问答

一个请求超时，如何分层定位并判断远端是否已执行？

请求超时的排查应沿同一请求身份逐段找最后一个确定进展点：接纳、入队、本地写出、对端接收、执行、结果返回、上层交付。每一层都要区分明确失败与结果未知。

原理与成立条件

从关联标识开始

一条命令应能在入口、接纳、发送、远端接收、执行、结果和上报各阶段被关联。记录业务 ID 与会话/序号、阶段时间和失败原因；缺少关联时，日志里看见一个“ACK”也可能是另一条或上一代命令，排障容易沿错误路径推测。

按最后确定的事实推进

若 MQTT 回调未收到，查连接、订阅和主题；收到但未接纳，查队列与窗口；已写但无节点接收，查波形、UART 错误、DMA 与解析；已接收无结果，查执行顺序、传感器等待及缓存；已收到结果但云端没有，查结果发布及其确认。每层都问“哪一步有证据”。

区分期限来源

超时可能是本地排队寿命、接收重传耗尽、业务结果期限、设备 HAL 超时或 MQTT 连接超时，含义不同。增加一个总 **timeout** 只会推迟所有错误暴露。先画实际事件时间线，核对每个计时起点及是否被重试重置，再决定阈值是否合理。

复现必须控制故障位置

有选择地丢命令、ACK 或结果，制造传感器卡线、数据库变慢和旧回调延迟，比较状态迁移是否符合预期。故障注入要限定环境并记录恢复，不能靠随机拔线后只看最终恢复就宣称所有中间结局正确。重试期间还要检查是否重复产生业务效果。

推演与反例

为什么统一请求身份很重要

串口序号、会话、云端业务 ID 与一次发送尝试不是同一个编号。若日志只写“超时 **seq=3**”，重连后另一个 **seq=3** 可能混进分析。记录身份、单调时间戳与状态变化，才能看出是排队超时、重传计时过早、节点慢业务还是结果 ACK 丢失。墙上时间用于跨系统对齐时还要考虑时钟同步误差。

定位最后一段已证实的事实

若线路上看见完整命令而节点 RX 计数不增长，优先查硬件/驱动；若解析成功但执行未开始，查窗口缺口和锁；若动作已完成而网关没结果，查结果队列、缓存和返回链。证据应逐层收窄，不要每轮同时改波特率、超时和优先级，否则即使恢复也无法知道原因。

继续追问

用户看到 **timeout** 后设备状态却已改变，是不是协议一定错了？

可能是执行成功而结果丢失，属于分布式不确定结局。协议应清楚报告这类边界并提供查询/幂等策略，不能仅凭超时保证撤销。

超时日志里最该避免哪种措辞？

把“没有收到结果”写成“设备没有执行”。前者是观察事实，后者需要额外证据；它会直接影响上层是否安全重试。

依据与核对范围

[strace](#) 官方文档入口 — 系统调用、信号与进程跟踪。

[Eclipse Mosquitto: C API](#) — [publish](#) 返回值、网络线程、回调与重连。

逻辑分析仪、示波器、GDB 各适合查什么？

逻辑分析仪看数字协议与时序，示波器看电压、边沿和模拟质量，GDB 看软件状态、调用栈和内存。要先确定问题发生在哪个层次，再选择能看到该现象的工具。

原理与成立条件

逻辑分析仪看协议，示波器看电气

逻辑分析仪适合长时间数字边沿和 UART/I2C/SPI 解码，前提是采样率、阈值与解码参数正确；示波器看上升沿、过冲、噪声、电压和时钟质量。解码乱码不必然是软件错，电气边沿不满足输入阈值也会导致同样现象。

用 GPIO 标记建立时间证据

在 ISR、临界区或任务关键阶段翻转闲置引脚，可直接观察占用和相对顺序。尽量使用低开销固定操作，并估计标记本身成本。长时间趋势可配 RAM 事件记录，不宜在微秒路径 `printf`；打印串口可能与业务串口冲突并显著改变调度。

调试器能看什么，也会改变什么

GDB/SWD 可读寄存器、栈与内存，硬件断点/观察点有数量限制。暂停 CPU 时外设、DMA 和 watchdog 是否继续运行取决于芯片与调试冻结配置；恢复后出现的超时不一定是自然运行故障。抓故障现场时记录二进制版本、优化选项和暂停条件。

Linux 工具如何配合

`strace` 看系统调用返回和等待，`perf` 看运行时热点，线程栈看等待对象，网络抓包看线上协议。它们分别观察不同层，不能用 `strace` 没有报错证明远端执行成功。工具开销与权限也影响实验，先用小范围捕获再扩大，避免采集本身造成新的超时。

推演与反例

同一 I2C 失败怎样交叉验证

逻辑分析仪可检查地址、ACK、START/STOP 和时序顺序；示波器判断上升沿是否过慢、低电平是否达标、是否有毛刺；调试器查看 HAL 状态和超时返回。数字解码器可能把边沿很差但勉强过阈值的波形解释成正确数据，因此解码成功不能证明模拟裕量充足。

调试会改变被测系统

断点暂停 CPU 时，DMA、外设和看门狗是否暂停取决于配置；原本没有的缓冲覆盖可能因断点出现，原本应复位的问题又可能被调试冻结掩盖。微秒路径适合 GPIO 打点、捕获和轻量计数，复杂日志放到事后输出。硬件观察点可定位哪条指令改坏某地址，但数量与访问能力有限。

继续追问

如何分清 UART 波特率错误和软件丢字节？

示波器先核实际位宽与电平，再比较线上完整字节序列、DMA 进度和应用接收记录；线上正确而软件缺失才把重点转向中断、缓冲与解析。

量到一次最短中断响应就能宣称最大延迟吗？

不能。要构造竞争、中断嵌套、临界区和总线负载等最坏条件，测量结果还应与分析边界结合。

依据与核对范围

GNU GDB 官方手册 — 断点、栈、寄存器、远程调试与目标现场。

ST RM0008: STM32F1 参考手册 — RCC、GPIO、DMA、TIM、USART、I2C、IWDG/WWDG 等章节；配置同时核对本地图 HAL。

T107 原理问答

测试应该分哪些层，怎样覆盖时序与并发边界？

单元测试验证局部逻辑，集成测试验证组件接口，端到端测试验证真实链路，板级测量验证硬件和时序。覆盖层次越清楚，面试时越不容易把一种证据夸大成另一种。

原理与成立条件

测试层级按隔离对象划分

单元测试验证纯函数和状态机；集成测试覆盖模块间交接及真实 I/O API；目标端测试覆盖移植、驱动和硬件时序；系统测试验证业务链与恢复。某层通过只覆盖该层使用的假设，不能从协议核心测试推出 DMA、供电和 Flash 都可靠。

好用例来自边界和不变量

对解析器测最短/最长长度、任意分块、坏校验、噪声；对状态机测重复、乱序、超时边沿、回绕和新旧会话；对队列测满、关闭与生产者并发。用例预期应描述业务现象，如“不重复执行且补发结果 ACK”，不能只断言某内部函数刚好被调用一次。

测试替身要暴露不确定性

假 HAL 若永远立即成功，会隐藏超时、短写和回调重入。替身应允许控制成功/失败、进度与事件顺序，可控时钟可避免 sleep 导致不稳定。硬件替身也要声明模拟不了的内容，例如 DMA 总线仲裁和真实中断 pending，防止测试越界背书。

报告怎样不夸大

记录构建类型、用例数、输入与环境，以及未执行板卡/耐久/性能测试。通过率不代表覆盖率，覆盖率也不证明语义正确。已有仓库测试是项目证据，个人贡献需另核实；将验证资产全写成个人设计反而会在追问时暴露责任不清。

推演与反例

怎么把测试从函数覆盖变成语义覆盖

协议应断言同一请求的执行次数、结果是否匹配、超时后资源是否释放，而不只断言某个函数被调用。并发应覆盖关闭时生产者仍在提交、回调删除自身、条件变量超时与入队竞争。错误注入应包括短写、EINTR、队列满和数据库失败，检查系统能否给出明确终态并继续处理后续工作。

确定性与真实环境怎样互补

用虚拟时间和可控 I/O 缩短边界测试并复现失败；用 PTY、真实 Broker 或真实板验证系统调用、库回调及电气接口。模拟环境不会自动体现 DMA 覆盖、LSI 误差或 DHT 波形，真机测试也不容易穷举所有状态排列。分别记录做过什么、观察到什么和未验证什么，才有可评估的工程结论。

继续追问

为什么不只做完整端到端测试？

它覆盖真实链路却定位困难、运行成本高，稀有事件顺序也难控制；小范围确定性测试与真实系统验证互补，不能互相取代。

仓库里有一套测试，能算个人贡献吗？

不能仅据存在就这样表述。应区分自己编写、维护或实际运行验证的内容，并明确其他成员或已有资产的来源。

依据与核对范围

GCC: [Instrumentation Options](#) — ASan/UBSan/TSan 的用途与限制。

CMake: [cmake-buildsystem](#) — 构建目标、依赖与使用要求。

T108 原理问答

systemd 部署、日志与自动重启怎样配合？

systemd 负责进程生命周期、启动依赖、权限与失败重启；应用仍负责识别故障、释放资源和保存必要状态。自动重启可以恢复部分服务，却不能替代正确的业务恢复协议。

原理与成立条件

服务管理器提供什么

systemd 根据 unit 描述启动命令、身份、环境和依赖，跟踪进程退出并按策略重启。Type=simple 的“已启动”不等于应用所有资源已就绪；若要精确就绪通知需相应协议。重启机制恢复进程活性，不会自动恢复内存中的未完成业务。

退出与重启策略

Restart 的选项决定哪些退出原因触发重启，RestartSec 控制间隔，启动频率限制避免快速崩溃循环。正常 stop 通常发终止信号并给停止期限，超时可能被强制杀死。应用响应信号并有限排空，不能假设有无限时间写库或等待外设。

启动顺序不是资源可用证明

After 描述相对顺序，不自动建立全部依赖或验证远端 broker 可访问；网络就绪也不证明 DNS 与业务服务可用。应用仍需要错误分类、退避重试和状态上报。WorkingDirectory、配置路径、设备权限及运行用户也是“手动能跑、服务不能跑”的常见差别。

最小权限和运维证据

给服务必要的设备与数据目录权限，避免为了一个串口直接长期用 root；资源限制与沙箱选项需与设备访问兼容。journal、退出码、重启次数和应用阶段日志一起反映健康。不要把 systemd WatchdogSec 当作 MCU IWDG，它依赖应用协议和用户态服务管理语义。

推演与反例

启动顺序不等于外部服务已经可用

After 约束启动顺序，Wants/Requires 表达不同依赖关系，network-online 也不保证某个 Broker 已可连接。应用应对连接失败有明确启动或重试策略。Type=simple 通常以进程启动作为服务开始，不表示数据库和网络都已准备好；需要精确就绪通知时应选择相应协议和配置。

最小权限如何影响部署正确性

串口访问需要合适设备权限，数据库与日志需要可写目录；ProtectSystem、DynamicUser 等限制必须与实际路径匹配。ExecReload 发送 SIGHUP 只是请求重载，是否成功生效仍需应用反馈。Restart=on-failure 加退避能避免快速重启风暴，但若每次重启都丢失未完成业务，服务“又起来了”不等于命令已可靠恢复。

继续追问

服务显示 active，能否说明数据正在正常采集？

只能说明服务管理器所观察的生命周期状态；采样、串口、数据库和云端仍需业务健康指标。

进程一直重启，第一步是什么？

先查退出码、服务日志、权限和配置路径，保留现场；不要把重启间隔调得更短来掩盖启动失败或资源依赖问题。

依据与核对范围

systemd 官方 `systemd.service` 手册源码 — 重启、停止超时与服务状态；官网手册访问受限时采用官方源码。

systemd 官方 `systemd.exec` 手册源码 — 运行身份、目录权限和沙箱配置。

T109 原理问答

怎样设计一个可移植、可测试的驱动或协议模块？

可移植模块把纯逻辑与时间、I/O、存储等环境依赖分开，接口明确数据所有权、调用上下文与错误语义。可测试性来自可控制这些输入，而不仅仅是把代码拆成多个文件。

原理与成立条件

可移植核心应该排除什么

把编码、解析和可靠协议状态机写成不依赖硬件寄存器、线程 API 和真实时钟的核心，通过参数或回调传入时间与发送/交付事件。这样同一逻辑可在 MCU 与 Linux 使用，也便于确定性测试。接口必须明确发送返回到底是接纳、提交还是线路完成。

状态与平台资源怎样分层

核心拥有协议状态和固定容量，适配层拥有 fd、DMA、RTOS 对象与缓冲交接。核心不能隐式阻塞等待平台完成，否则跨平台语义容易失真；适配层也不应偷偷改协议内部序号。用清晰的事件/状态接口保留单一所有者。

可移植不等于行为自动一致

整数宽度、端序、结构布局、编译选项和时间回绕都可能不同。使用明确宽度、显式序列化和受约束时间类型，两个平台仍需验证所有调用前提。共享源码可以减少漂移，不能替代不同 I/O 模型下的生命周期与并发检查。

版本与同步如何管理

共享库单一来源、子模块或自动同步检查各有维护成本；至少应记录版本、协议常量和固定向量。协议兼容变更与实现内部重构分开处理，跨端部署还需考虑升级次序，不能只在开发机把两个文件改得一样就认为线上节点全已更新。

推演与反例

协议内核如何脱离某个操作系统

把当前时间、发送帧和执行业务设计为回调或接口，状态机只处理有界字节与显式状态，不直接睡眠、访问 UART 或创建线程。主机测试可注入虚拟时间与记录发送结果，MCU 适配层则调用真实队列和驱动。接口需说明回调是同步还是异步、参数是否只在调用期间有效，否则移植后仍可能出现寿命错误。

分层不能把责任一起丢掉

驱动层报告搬运或硬件错误，协议层处理定界、确认与重试，业务层决定参数合法性和执行结果。每层都应保留足够的错误信息，并明确谁负责恢复；若底层只返回 `void`、上层假定成功，分层反而隐藏失败。不要为每个小函数都造抽象，而应在真实变化点和测试控制点建立边界。

继续追问

把 HAL 类型放进公共协议头有何代价？

核心会依赖节点工具链和头文件，网关测试与其他平台复用变难；可把硬件句柄留适配层，以普通数据和回调定义核心边界。

mock 全部通过就证明真驱动正确吗？

不能。mock 只能验证其建模的契约，真实中断次序、时钟、DMA 和电气行为仍需集成或板级验证；接口模型本身也可能错误。

依据与核对范围

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

CMake: `cmake-buildsystem` — 构建目标、依赖与使用要求。

T110 原理问答

手写单生产者单消费者环形队列，关键不变量是什么？

SPSC 环形队列的优势来自只有一个生产者推进写位置、一个消费者推进读位置。正确性不仅是下标取模，还包括空满判定、数据发布顺序、回收顺序和计数回绕约束。

原理与成立条件

SPSC 为什么能简化同步

单生产者只推进 `head`，单消费者只推进 `tail`，避免多个写者同时竞争同一索引。常见 `N` 槽环保留一个空槽，`head==tail` 表示空，`next(head)==tail` 表示满，实际容量 `N-1`；也可用单调计数和代数表达满空，但需处理位宽回绕与比较范围。

发布顺序是核心

生产者先写元素，再 **release** 发布新 **head**；消费者 **acquire** 看到这个 **head** 后才读元素，防止读到尚未完成的数据。消费者读完再 **release** 发布 **tail**，生产者 **acquire** 观察后才复用对应槽，防止消费者未结束就被覆盖。只让索引 **volatile** 不建立这两条 C++ 同步关系。

为什么多生产者不能直接套用

两个生产者可能同时看到同一 **head** 并写同一槽，单纯原子递增也可能出现某个槽尚未填好就被消费者看见。MPMC 常需 CAS、每槽序号和更复杂的发布/回收协议。所谓 **lock-free** 也不保证每个线程固定步数内完成，**wait-free** 是更强性质。

DMA 环与 CPU 环的差异

DMA 没有自动执行你的 C++ **release/acquire** 协议，硬件会按配置覆盖槽，消费者需要完成计数、缓存维护、借用期限或流控。索引都正确也不能恢复已被覆盖的数据。真正零拷贝还要定义块何时归还、错误取消和部分消息跨块处理。

推演与反例

发布与回收各需要一条同步链

生产者先确认有空间，再写槽内容，最后 **release** 发布新写位置；消费者 **acquire** 观察该位置后才能读对应槽。消费者读完再 **release** 发布读位置，生产者 **acquire** 观察后才可复用该槽。若先公布索引再写内容，消费者可能读取未准备好的数据；若先公布槽已空再读完，生产者可能覆盖仍在使用的数据。

容量与算法模型要说清

一种设计留一个空槽区分空满，**N** 个槽只能存 **N-1** 项；另一种用单调计数及差值表示占用，可利用全部槽，但要证明计数回绕不会产生歧义。多生产者直接改成 CAS 增写位置还不够：预留顺序和实际写完顺序可能不同，需要额外发布协议。普通 **volatile** 下标不自动提供 C++ 跨线程同步。

SPSC 环形队列的发布顺序（伪代码）

```
// 生产者，只有它写 head
h = head.load(relaxed)
next = (h + 1) % N
if next == tail.load(acquire): return FULL
slots[h] = item
head.store(next, release)

// 消费者，只有它写 tail
t = tail.load(relaxed)
if t == head.load(acquire): return EMPTY
item = slots[t]
tail.store((t + 1) % N, release)
```

这里预留一个槽，容量为 $N-1$ 。槽先写后发布，槽读完后才归还；双向 **release/acquire** 分别约束数据可见性和复用时机。假定元素存储有效、操作不抛异常且只有一个生产者和一个消费者；平台还需确认所用原子类型与执行上下文兼容。

继续追问

head 和 tail 都是原子，数据区是否就一定安全？

只有写数据→发布 head→读取，以及读取完→发布 tail→复用两条关系正确且符合 SPSC 假设时才可推导；原子变量的存在本身不是证明。

为什么不把 CPU 队列证明直接套在 DMA 环上？

DMA 不通过同一个 C++ 原子协议发布，也可能独立覆盖内存；若有非一致缓存还需维护。必须重新定义硬件生产进度、覆盖代数与 CPU 消费完成的交接机制。

依据与核对范围

C++ 工作草案：原子内存序 — **release/acquire/relaxed** 与可见性。项目按 C++17，未引入新标准专属特性。

FreeRTOS: **Stream / message buffers** — 单写者单读者与字节流语义。

FreeRTOS 官方 **StreamBuffer** 接口契约 — 单写者/单读者、**trigger level** 与 **Send/Receive** 返回值；以版本适用性为前提。

跨模块设计与定量分析

T111 原理问答

有界队列、背压与丢弃策略如何从负载推导？

队列把生产与消费在时间上解耦，却不会增加消费者的长期处理能力。设计应先估算到达、服务与最长停顿，再决定容量、接纳条件以及满时对上游反馈什么。

原理与成立条件

长期速率与短期突发

若长期到达率 λ 大于服务率 μ ，积压会持续增长，任何有限队列最终都会满； λ 小于 μ 只说明长期有排空机会，仍可能被短时突发压满。一个停顿 T 内最多到达 B 字节、此前已经积压 Q 字节，则这段至少需要容纳 $Q+B$ ，后续还需保证恢复消费能追上。平均速率不能给出最坏容量。

延迟与内存来自同一份积压

积压越多，排在后面的任务越晚开始；容量增大可少拒绝一些瞬时请求，同时扩大最坏等待与对象寿命。在稳定系统中，Little 定律把平均在系统内数量、有效到达率和平均停留时间联系为 $L=\lambda W$ ，但它不是用平均值直接证明最坏期限的公式。实时控制应同时限制队列深度和最老任务年龄。

选择丢弃需要业务语义

周期遥测可考虑保留最新值、合并或丢新数据并计数；有顺序的命令、不可重建的事件和已确认接纳的操作不能套同一策略。背压可以表现为暂停读取、返回忙、发放信用或限制在途请求。停止读取也会把压力转移到底层缓冲，必须确认该链路能否把压力传回真正生产者。

推演与反例

窗口为什么不等于字节信用

允许 8 条命令在途，只约束数量；若每条长 69 字节，它们仍可在很短时间内形成数百字节突发。接收端 64 字节 DMA 环与 64 字节软件流缓冲，不能由“窗口只有 8”直接推出安全。应计算帧长、发送节奏、ISR 延迟、执行停顿与结果占用，必要时按可用字节或块数发信用。

继续追问

不丢、不阻塞、内存有限能否对任意输入同时保证？

不能。只要输入可持续超过消费能力，就必须拒绝/减速、丢弃或增加持久存储等资源。接口应明确接受范围及失败表现，不能把无限队列当可靠性方案。

依据与核对范围

MIT 1.041/1.200: 排队模型与 Little 定律 — Spring 2026, Stationary analysis and Little's law; 平均关系不能代替最坏突发容量证明。

pthread_cond_wait(3p) — 谓词、原子解锁等待与虚假唤醒。

FreeRTOS 官方 queue.h 接口契约 — 队列元素复制、发送等待与 ISR 接口。

T112 原理问答

零拷贝为什么首先是缓冲所有权问题？

零拷贝的目标是减少数据搬运，但前提是消费者使用期间底层存储保持有效且内容稳定。只把数组换成指针队列，只减少了指针传递时的复制，并没有证明数据不会被复用或覆盖。

原理与成立条件

建立一块缓冲的完整生命周期

以块池为例，块从 **FREE** 变成 **FILLING**，生产者写完后变成 **READY**，消费者取得后进入 **PROCESSING**，最后回到 **FREE**。各状态只能有明确拥有者；入队失败、超时取消、错误恢复和退出都要规定谁归还块。引用计数可以表达多个消费者共同拥有，但要避免释放时仍有 **DMA** 或回调借用。

DMA 环为什么难以直接借用

循环 **DMA** 按硬件进度不断回写，没有等待应用归还某段内存的机制。**CPU** 取得一个环内指针后，耗时解析、传感器读取或写库都可能使 **DMA** 追上并覆盖。可改为停交接的分块设计、双缓冲配合消费预算，或让解析器及时拷贝必要数据；每一种都有吞吐、延迟和 **RAM** 代价。

重传进一步延长寿命

发送请求可能第一次失败后再发送，同一编码帧要活到最后一次可能访问结束；一份业务数据同时去数据库与 **MQTT**，也要等两条支路均不再借用才能复用。超时上报不等于底层 **I/O** 已停止，取消与完成必须有确定的最终归属。所谓零拷贝应明确节省的是哪一段，而不是宣称全链路没有复制。

推演与反例

块池耗尽时如何保持闭环

若所有块都被慢消费者占用，**DMA** 或生产者需要停接纳、启用流控、丢弃并重同步，或使用有界备用路径。没有满池策略的所有权图仍是不完整的。测收益时同时看节省的 **CPU**、增大的驻留内存和新增控制复杂度。

继续追问

小帧一定值得零拷贝吗？

不一定。几十字节的复制通常换来简单而稳定的寿命边界，管理复杂度可能比复制开销更大。先测瓶颈，再选择值得优化的交接点。

依据与核对范围

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

FreeRTOS 官方 queue.h 接口契约 — 队列元素复制、发送等待与 ISR 接口。

ST AN2548: DMA 控制器原理 — DMA 请求、总线争用与传输模式；具体通道和完成语义对照 RM0008 及本地 HAL。

T113 原理问答

状态机怎样把事件、超时和错误恢复组织清楚？

状态机用“当前状态、输入事件、保护条件、动作、下一状态”表达流程。价值在于把非法事件和异步先后关系显式化，而不只是把一组 if 换成枚举。

原理与成立条件

状态与事实要避免互相矛盾

一个请求可用排队、发送中、等接收、等结果、完成/失败来描述主要阶段，同时记录会话、尝试次数和截止时间。派生事实尽量从权威状态计算，避免多个布尔值出现“已完成但仍在等待发送”等无意义组合。每个转换应维护窗口、缓冲及回调次数等不变量。

所有事件都要有处理规则

正常 ACK、重复 ACK、结果先到、错误会话、取消、发送失败、定时到期都应明确是推进、忽略、补确认还是恢复。事件处理最好在一个所有者线程/任务中串行进行，跨线程输入先排队；如果同一状态由多个上下文改写，状态机图本身并不能消除数据竞争。

超时也是普通事件但不是远端事实

超时转换只说明本地决定不再等，它可能与完成事件同时到达。应以一次终态提交保证回调至多一次，并隔离晚到事件。若底层仍可能执行，终态可以表达“结果未知”，由业务恢复流程处理，不应伪造成“已取消远端操作”。

推演与反例

从转换表得到测试

为每个状态列出允许输入与拒绝输入，再覆盖所有边界转换；使用假时钟生成到期前、到期时和到期后的事件。检查每次转换后占用资源是否有界、终态是否单向、旧代是否能污染新代。这样测试跟随语义，而不是机械重复内部函数实现。

继续追问

状态越少越好吗？

过少会把不同责任混在一个状态里，过多又增加非法组合。应按等待的外部事实和资源责任划分，能解释每个状态存在的必要性。

依据与核对范围

MIT 6.02: 可靠传输与 ARQ 课程原稿 — 第 20 章 Stop-and-Wait、Sliding Window 与 ACK；项目窗口与缓存寿命由本地实现给出。

FreeRTOS 官方内核书: Task Management — 第 4 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

T114 原理问答

定点编码、单位、有效性与采样时间如何设计？

一个传感值至少包含数值、单位、缩放、有效性和采样时间语义。线上使用整数可简化跨平台表示，但必须先定义范围、符号和舍入规则，不能只写“乘十传过去”。

原理与成立条件

定标怎样影响范围与分辨率

用有符号 16 位整数表示 0.1 °C，量化步长为 0.1，理论编码范围约为 -3276.8 到 3276.7 °C；真实传感器范围通常更窄。若改成无符号，负温无法直接表达，不能强转后让它变成很大的正数。扩大编码小数位也不会提高传感器本身精度，它只改变数字表示粒度。

舍入必须先验证数值

对已知非负且处于范围内的值，加 0.5 再截断可实现常见四舍五入；负数、NaN、无穷大和超范围值需独立处理。两端必须约定饱和、拒绝还是报告错误，不应一端截断另一端解释为有效大数。整数字段相乘也要在足够宽的类型中完成，再检查缩窄。

无效数据不应伪装成正常零值

读取失败时返回 0 会混淆“零照度”与“传感器掉线”。可以通过独立 **valid/status** 和错误码表示，缓存旧值则应附时间或年龄。采样时刻、节点发送时刻、网关接收时刻及入库时刻不同；若只能记录接收时间，应明确它不是精确采样时间。

推演与反例

如何做一条端到端数值核对

从传感器原始字节开始，写出符号与小数解析、业务值、定点整数、线上字节序、网关还原和数据库单位。选正常小数、零、边界、负值与无效数据逐项追踪，能发现小数被截掉、单位重复缩放或状态被忽略的问题。

继续追问

温度字段多加一个小数字节就能提高测量精度吗？

不能。编码分辨率、器件分辨率和测量准确度是三件事，必须引用器件条件并对误差作合理解释。

依据与核对范围

WG14 N1570: C11 委员会草案 — 6.2/对象表示和链接、6.5/表达式、7.24/字符串内存接口。

ST STM32F103x8/xB 数据手册 — 存储规格、时钟与电气参数范围。

奥松 DHT11 V1.3 产品手册 — V1.3_20170331: 表 4/图 3-6 时序; S6 读取间隔与前次测量值; S7 温度小数及负号、湿度小数为 0。实物版本应该对。

ROHM BH1750FVI 原厂手册 (DFRobot 镜像) — 原厂 PDF, 电气特性表及 I2C Bus Access / measurement adjustment; 120 ms 典型、180 ms 最大值与默认系数。镜像为较早修订, 实物版本应该对。

T115 原理问答

错误码、异常安全与恢复重试怎样形成一致契约？

错误处理不仅是记录日志，还要告诉上层哪些动作已经发生、哪些资源仍有效，以及能否安全重试。返回值、异常与状态转移应共同表达这一契约。

原理与成立条件

先区分未开始、部分完成和结局未知

打开候选串口失败时旧端口可以完全未变；写了半帧后失败已经产生部分外部效果；数据库提交返回异常时还要确定事务状态。把这些都压成 **false** 会让上层无法决定重试或回滚。接口应尽量保留阶段、错误原因和已完成进度，避免把失败路径中的实际效果隐藏掉。

异常安全的层次

基本保证要求失败后不泄漏且对象仍保持有效不变量；强保证希望失败后观察状态不变；不抛异常保证用于特定清理路径。构造新资源再交换可实现许多局部强保证，但外部网络发送和物理动作无法靠交换指针撤销。析构通常不能抛异常穿过另一个异常展开，应采用可诊断的受控失败方式。

重试必须有分类和期限

临时不可用可重试，非法参数或永久权限错误通常不应原样无限重试。还需退避、最大次数、总截止时间和重复副作用处理；重试本身占用队列与资源，可能放大过载。恢复成功应以实际可用状态确认，不只是某个重初始化函数被调用过。

推演与反例

日志与返回值应一致

如果底层写失败只打日志仍返回成功，上层就可能删除唯一待重试副本。相反，底层已成功而上层无证据地重做，会产生重复。对重要操作，应使成功返回对应明确的完成里程碑，并为部分成功或结果未知提供独立表示。

继续追问

捕获所有异常然后继续运行更稳健吗？

只有能恢复对象不变量并给请求明确结局时才是恢复。吞掉错误后继续使用半初始化资源，往往把清晰失败变成更难定位的后续损坏。

依据与核对范围

C++ Core Guidelines — R.1/资源管理、CP/并发与接口生命周期。

write(2) — 短写、错误与完成边界。

SQLite: Transaction — 事务边界与提交失败。

T116 原理问答

固定优先级任务怎样做最坏响应时间分析？

平均 CPU 利用率不能证明实时性。对一个任务，需要把自身执行、低优先级资源阻塞和更高优先级任务/中断的干扰合起来，判断最坏响应是否小于截止时间。

原理与成立条件

一个常见分析模型

在单核、固定优先级、可抢占、独立周期/间歇任务、截止期不大于周期、无释放抖动且阻塞已被正确界定时，可迭代估计 $R_i = C_i + B_i + \sum \text{ceil}(R_i/T_j)C_j$ ，求和针对高优先级任务。 C_i 是本任务最坏执行时间， B_i 是受控阻塞上界， T_j/C_j 是高优先级任务周期与执行预算。中断、释放抖动和额外不可抢占区需要按相应模型加入；不是任何实际程序都可直接代入这个简式。

CPU 执行与等待外设必须区别

任务持锁等待 180 ms 转换时，CPU 可能在运行其他任务，但高优先级查询仍会因资源阻塞等待很久。若直接把全部等待算成纯 CPU 执行，利用率估计又会失真。需要按自挂起、服务任务或资源协议建立正确模型，再分别报告 CPU 利用和端到端完成期限。

推演与反例

用数字说明为何需要迭代

低优先级任务自身需 2 ms，阻塞上界 1 ms；高优先级任务每 5 ms 到达一次、每次执行 1 ms。从 $R=3$ 开始，代入得到 $R=3+\text{ceil}(3/5)\times 1=4$ ms，再代入仍为 4，得到该模型下的上界。如果截止时间是 3 ms，即使平均利用率看起来不高，也无法满足。增加一个中断源还需重新分析。

继续追问

哪些证据才能支持“实时响应优化了”？

同等负载下的最大中断屏蔽、任务唤醒、排队与业务完成测量，加上最坏到达和资源阻塞分析。只减少平均函数耗时或提高任务优先级不足以形成这个结论。

依据与核对范围

University of York: 固定优先级响应时间分析 — Paul Emberson 博士论文，2009，§2.2.2，式 2.3–2.5；本文示例采用单核固定优先级、无释放抖动的受限模型。

FreeRTOS 官方内核书: Task Management — 第 4 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

FreeRTOS 官方内核书: Resource Management — 第 8 章；用于通用机制。具体节点实现另对照随包内核源码，不推定与上游 main 完全相同。

FreeRTOS 移植：源码、异常与验收

T117 原理问答

把 FreeRTOS 移植到 Cortex-M3，源码和编译选项怎样选？

FreeRTOS 移植首先是把内核公共代码、与编译器和 CPU 匹配的 portable 层、恰好一种内存管理实现一起编进目标程序，再提供 FreeRTOSConfig.h。选择依据是 CPU 指令集、编译器 ABI 和项目实际使用的内核功能；把所有 portable 目录或多个 heap 文件一并编译会造成重复符号或错误的上下文切换实现。

原理与成立条件

内核公共层与硬件适配层

任务调度使用 tasks.c、list.c；使用队列、软件定时器、事件组、流缓冲时再加入 queue.c、timers.c、event_groups.c、stream_buffer.c。GCC 编译的 Cortex-M3 使用 portable/GCC/ARM_CM3/port.c 与同目录 portmacro.h。port 负责初始任务栈、SVC 启动首任务、PendSV 切换、SysTick 节拍和 BASEPRI 临界区；应用不应自行重写这套保存恢复流程。

头文件和目标架构

编译器需能找到内核 include、所选 port 目录和应用的 FreeRTOSConfig.h；链接时只有一个启动向量表和一组 FreeRTOS 异常入口。目标必须编译为 Cortex-M3 兼容指令与 ABI；芯片头、CMSIS 优先级位数、系统时钟和链接脚本应与真实器件一致。Cortex-M3 没有硬件 FPU，不应拿 M4F 或 ARMv8-M 的 port 来代替。

内存与版本可追溯

可按需求选择 heap_4.c，它用一块预留数组分配任务栈、TCB、队列等对象并合并相邻空闲块；静态创建则需另外打开相应配置并提供内核要求的存储。记录内核副本的准确来源、版本或提交、许可证和编译文件清单；开发分支快照的版本字符串不能当作正式发行标签。

推演与反例

最小编译清单怎样检查

先把 tasks.c、list.c、queue.c、匹配的 port.c、heap_4.c 和三个头文件目录接入，再只为实际开启的 timers、event groups、stream buffers 加源码。若报找不到 portmacro.h，先查移植层 include 路径；若出现多个 pvPortMalloc 或 SysTick_Handler，查是否混入两种 heap 或两套异常实现。

继续追问

使用 CubeMX 的 FreeRTOS 中间件和手动移植可以混用吗？

可以复用 HAL 与 CMSIS 基础文件，但不能同时把 CubeMX 生成的另一套 FreeRTOS/CMSIS-RTOS 包装和手动接入的内核重复编译。先确定是原生 API 还是 CMSIS-RTOS 包装，再检查生成文件、入口和链接符号。

为什么编译通过仍不能证明移植成功？

编译只检查源码和符号。向量表、NVIC 分组、SysTick 时钟、任务栈余量和 ISR 的 API 调用边界要到链接检查与板上运行阶段才能验证。

依据与核对范围

FreeRTOS: [Tasks](#) — 任务状态与调度。

FreeRTOS 官方 [ARM_CM3 移植源码](#) — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep；上游参考，当前行为以本地快照为准。

FreeRTOS: [Memory management](#) — heap_1..5 与静态分配。

ST PM0056: [Cortex-M3 编程手册](#) — 异常、寄存器、NVIC、SysTick 和故障状态；Rev 7。

T118 原理问答

SVC、PendSV、SysTick 怎样接入向量表并启动第一个任务？

Cortex-M 移植需要把向量表中的 SVC、PendSV、SysTick 入口接到 FreeRTOS port。启动调度器时内核创建 Idle 等系统任务，port 配置节拍和异常优先级，再触发 SVC 恢复第一个任务；正常 tick 由 SysTick 推进，需要换任务时挂起 PendSV，由它完成上下文切换。

原理与成立条件

从复位到任务

复位处理函数先设置 MSP、初始化数据段、执行系统初始化并进入 main；main 初始化时钟、外设和内核对象，再调用 vTaskStartScheduler。xPortStartScheduler 配置 SysTick 与异常优先级，SVC 恢复首任务保存的 R4-R11 与 PSP。此后线程态任务使用各自的 PSP 栈，异常处理器使用 MSP；不能把链接脚本中的主栈大小当作所有任务栈大小。

向量表只应指向有效实现

启动汇编通常把 SVC_Handler、PendSV_Handler、SysTick_Handler 声明为弱默认处理函数；可以通过 FreeRTOSConfig.h 宏把 port 的三个处理函数直接命名成这些符号，也可以使用正确转发入口。CubeMX 若同时生成同名空函数，应移除或禁用，否则可能重复定义，或运行时进空函数而无法调度。重新生成代码后必须复查。

三个异常为何分工

SysTick 调用内核推进时间，并在任务需要切换时设置 PendSV 挂起位；PendSV 处于最低可配置优先级，减少在其他中断中间切换上下文的风险。SVC 用于从启动代码进入第一个任务。SVC、PendSV、SysTick 是系统异常，不等同于 DMA/USART 外设 IRQ。

推演与反例

有符号还要核对向量

检查 ELF 符号和 map 文件，确认三个 Handler 来自预期 port.o，再核对启动向量表引用的是同名符号。若 SVC 保持弱默认实现，第一次启动通常进 Default_Handler；若 SysTick 被空桩占用，任务延时不会按预期到期。链接成功仍应单步观察首任务和 tick。

继续追问

为什么调度通常选 PendSV 而不在 SysTick 内直接保存恢复？

tick 应尽快完成计时和唤醒判断；PendSV 作为最低优先级的延后异常，可等更紧迫的 ISR 结束再完成切换。这样中断嵌套与上下文归属更清楚。

检查异常入口的断言能替代板上验证吗？

不能。它能发现部分向量接错，不能证明 SysTick 周期准确、栈未溢出或每条外设中断都遵守 FromISR 优先级限制。

依据与核对范围

ST PM0056: Cortex-M3 编程手册 — 异常、寄存器、NVIC、SysTick 和故障状态；Rev 7。

FreeRTOS 官方 ARM_CM3 移植源码 — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep；上游参考，当前行为以本地快照为准。

FreeRTOS: Tasks — 任务状态与调度。

T119 原理问答

FreeRTOSConfig.h 的时钟、堆栈和中断优先级怎样计算？

先从真实时钟树确定 CPU 和 SysTick 频率，再选 tick 频率；从 SRAM 预算确定堆与每个任务栈；最后按 CMSIS 实现的优先级位数计算中断阈值。任务优先级数字越大越高，Cortex-M NVIC 优先级数字越小越紧迫，两套数字不能直接比较。

原理与成立条件

时钟与任务栈单位

SysTick 与 CPU 同频时，可以不定义 configSYSTICK_CLOCK_HZ，由 port 使用 configCPU_CLOCK_HZ；普通 tick 的重装载值为 SysTick 时钟频率除以 configTICK_RATE_HZ 再减一。若 CPU 为 72 MHz、tick 为 1 kHz，则 $LOAD=71999$ 。xTaskCreate 的栈深度以 StackType_t 元素计；32 位 Cortex-M port 上 384 个元素是 1536 字节。configMINIMAL_STACK_SIZE 只直接决定 Idle 栈，其他任务需逐个指定并测高水位。

动态堆与链接内存

选择 heap_4 时，configTOTAL_HEAP_SIZE 是 FreeRTOS 堆数组的字节数，数组通常进入 .bss。任务栈、TCB、队列等动态对象从这里分配；链接脚本的 _Min_Heap_Size 和 _Min_Stack_Size 是另一层预留，不能拿来替代 RTOS 堆或任务栈预算。建任务前还要给静态数据、DMA 缓冲、主栈和中断栈留 RAM。

NVIC 逻辑值与寄存器编码

若实现 4 个优先级位，逻辑值 5 应左移 8-4 位成为 0x50，最低值 15 对应 0xF0。配置全抢占位的优先级分组后，只有逻辑优先级数字大于或等于 configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 的 ISR 可调用 FreeRTOS 的 FromISR API；数字更小的高紧迫度 ISR 不能调用。阈值不能为 0，默认优先级 0 也不能留给调用内核的 ISR。

推演与反例

计算一组可检查的参数

以 72 MHz、1 kHz tick、4 个优先级位和最大系统调用逻辑优先级 5 为例：

configCPU_CLOCK_HZ=72000000、configTICK_RATE_HZ=1000、configPRIO_BITS=4、configMAX_SYSCALL_INTERRUPT_PRIORITY=(5<<(8-4))=0x50。若 DMA IRQ 使用 CMSIS 逻辑优先级 6，就可以在 ISR 中调用相应 FromISR API；实际硬件仍须检查 NVIC 分组与所有调用路径。

继续追问

把 configTOTAL_HEAP_SIZE 调大为什么可能反而链接失败？

heap_4 的数组占用真实 SRAM；静态数据、主栈与 RTOS 堆一起超过链接脚本的 RAM 范围时，固件无法链接。调大前应看 map 文件、创建失败位置和最小剩余堆。

configMAX_PRIORITIES=7 是否限制系统只能有 7 个任务？

不是。它规定任务优先级可使用 0 到 6 七档；同一档可以有多个任务。可创建任务数受 RAM、内核配置和资源占用约束。

依据与核对范围

FreeRTOS: Cortex-M interrupt priorities — 优先级编码、BASEPRI 与可调用 ISR 范围。

FreeRTOS: Memory management — heap_1..5 与静态分配。

T120 原理问答

HAL 与 FreeRTOS 使用不同 Tick 时，怎样实现 Tickless 并验证移植？

HAL 超时与 FreeRTOS 延时可以来自不同定时器。HAL 时基应在调度器启动前就可用；RTOS 的 SysTick 则在启动调度器时由 port 接管。开启 Tickless 后，要识别哪些周期中断会把 CPU 立即唤醒，睡眠前暂停它们的中断，醒后恢复；同时明确 HAL 计数是否对睡眠时间补偿。

原理与成立条件

双时基与启动顺序

HAL_Init 和时钟初始化可能调用 HAL_GetTick 处理超时，因此不能简单在调度器未启动时就把 HAL_GetTick 改为 xTaskGetTickCount。若 HAL 另用一个 1 ms 定时器，它与 RTOS tick 是两套计数，前者供驱动等待，后者供任务调度；把 HAL_Delay 的时钟源替换掉，并不会自动让 HAL_Delay 变成可调度的任务延时。

Tickless 的具体边界

空闲任务预计可睡一段时间时，port 暂停正常节拍、重编程 SysTick、执行 WFI，醒后补偿 RTOS tick。另一个每 1 ms 触发的 HAL 定时器若不暂停其中断，会不断把 CPU 唤醒。若睡前只关闭 HAL 定时器的更新中断、醒后恢复，而没有补偿 HAL uwTick，则 HAL 计数在睡眠期间会落后于实际经过时间；不能把两个 tick 都当作连续墙上时钟。

分层验收

先确认编译与 ELF 中三个异常入口的符号来源，再观察首任务、任务阻塞和 tick 增长；接着验证每条调用 FromISR 的 IRQ 优先级以及队列/流缓冲唤醒；最后在实际板上检查睡眠唤醒、HAL 超时、任务栈高水位、堆余量、故障现场和长时间运行。构建或链接成功只覆盖前几层。

推演与反例

一个会破坏休眠的反例

RTOS SysTick 已延长到下一次 100 ms 截止期，但 HAL 的另一个 1 ms 定时器中断仍保持使能，WFI 约每毫秒就被唤醒。此时可能观察到 RTOS tick 被抑制，却没有得到预期的 CPU 连续睡眠时间。应分别看中断计数、唤醒源与 HAL Tick 的时间语义。

继续追问

睡眠时暂停 HAL Tick 会不会影响 HAL 驱动超时？

会改变 HAL_GetTick 的经过时间语义。若驱动在睡眠前后依赖绝对期限，需要明确补偿或用其他连续时钟；若任务因阻塞而空闲，也应检查唤醒后超时路径是否仍满足业务要求。

纯编译测试能证明 Tickless 降功耗吗？

不能。需要在目标板上测电流或至少测进入/唤醒次数和驻留时间；WFI 能执行不等于一定达到了预期功耗。

依据与核对范围

FreeRTOS: Low power support — Tickless 进入、唤醒与 Tick 补偿。

FreeRTOS 官方 ARM_CM3 移植源码 — SVC/PendSV、xPortSysTickHandler、vPortSuppressTicksAndSleep；上游参考，当前行为以本地快照为准。

FreeRTOS: Cortex-M interrupt priorities — 优先级编码、BASEPRI 与可调用 ISR 范围。

FreeRTOS: vTaskDelayUntil — 相对与绝对周期延时。